Go-Golf 1.0

2024 Computing Project by (Erol Halim)

STATEMENT OF ORIGINALITY

This report is submitted as part requirement for the Foundation Year in Computing Sciences course at the University of Sussex. The content of this report has not been previously submitted and is the product of my own labour except where indicated in the text

ACKNOWLEDGEMENTS

Erol Halim

SUMMARY

GoGolf is a 2D-based arcade Golf game. Old flash games and modern mobile games inspire the game with a Golf twist. Other Golf-based games have rarely gone down the 2D path and from the earliest conception of golf games for example GOLF (1984) have been a 3D environment where the player explores. Popular Golf games such as Golf with Friends (2016), Walkabout Mini Golf VR and GOLF IT are all 3D mini golf games, with online components, however, there is a lack of saturation within the arcade Golf games. This game is focused on creating a competitive single-player experience through leaderboard rankings, and player's ranks are determined through playthroughs of courses—Gameplay of the game with a 2D intractable course with semi-realistic physics involvement. The course comes with multiple holes with the goal for each course to be completed in the shortest time and fewest strokes, which game will reward the player with a higher calculated score. The more play-throughs of the game, the more rewards are showered to players through in-game currency that allows each player to have unique golf ball designs.

TABLE OF CONTENTS

STATEMENT OF ORIGINALITY....2
ACKNOWLEDGEMENTS....3
SUMMARY....4
TABLE OF CONTENTS....5
ANALYSIS....7
Introduction....7
Who is this for?....7
Problems with similar games....7
What Golf?....7
What is behind the game?....8
Aims....8
Objective....9
DESIGN....10
Background Research....10
Simulation Sports Games....10
Sports Managers....12
Sports-cade....13
Arcade Sports Games....15
Project Plan....18
Essentials....19
Golf Ball Trajectory....19
Camera, Tracking and Scrolling....21
Level Editor....21
Basics....22
Selection....22
Placement....22
Displaying....24
Saving and Loading....24
Basics....25
Game Levels....25
Tiles....26
Collisions....27
Depth....27
Gameplay....27
Gamemodes....29
Scores algorithm....29
Menus....30
Data Structures, Objects and Algorithms....32
Buttons....32
Text....33
Queues....33

Ingame Timer..... 34
Switching of Image Layers..... 34
Extras..... 35
Database..... 35
Go Golf ERD..... 36
Relations Attributes Datatyps and Constraints..... 37
Database Creation..... 38
Account Creation and Login..... 38
Creations..... 39
Log in..... 40
Encryption..... 40
Store..... 41
Leaderboard..... 42
Game objects/UMLs..... 45
Menus..... 45
Game (simple)..... 46
TEST..... 47
Type of testing..... 47
Test Plans..... 47
Logger..... 47
Editor..... 47
Game..... 49
Performance Tracking..... 52
Known Issues..... 54
EVALUATION..... 55
Objectictives Completed..... 55
System Requirements..... 57
Improvements for the Future..... 57
Conclusion..... 57
APPENDIX..... 59
References..... 59
Bibliography..... 60

ANALYSIS

Introduction

After sorting through various ideas for a project such as creating a blackjack game for practising card counting, a pvp bow game, a paint app or even a music player. I made a single-player video game called Go Golf. Go Golf is a mostly Python-based video game utilising the pygame framework. Go Golf is a 2D arcade golf-based video game, with multiple levels, game modes, a leaderboard, physics-based collisions and trajectory algorithms, GUI, an in-game store, an encrypted account creator and a level editor. All details about each account and related game information will be saved into a local database through SQL.

Who is this for?

Go Golf is an entertaining and repayable flash/mobile-inspired video game, from the Yohoho Games Red Ball, Game Pigeons Messenger Mini Golf and Shallot Games Coffee Golf. Where Go-Golf differs is through small bits of inspiration from these games and putting its twist. It has Red Balls 2D camera position, Mini Golf competitiveness through competing against other players and coffee golf multiple holes per level before completion. The game has two competitive modes, 3 holes and 5 holes, where players are given a high score for completing levels with the fewest strokes and time. Competing for high scores is
Similar to old arcade cabinet games where people will attempt multiple times to beat personal records and try to get as high as possible on a leaderboard. This game is for people who want that same thrill with a golf twist.

Problems with similar games

Go Golf differentiates itself from its competitors with its twist. All games stated beforehand in the previous subheading “Who is this for?”. These games don’t have a high score aspect to them, you are competing against various other competitors on the same local device. Recreating the same thrill and competitiveness of competing for high scores in arcade games such as Tetris. Mini Golf and Coffee Golf both don’t have this feature. Further, Go Golf can be played whenever compared to both. Mini golf needs a second active player, while Coffee golf can have only a few levels available at once. Go Golf fills this missing gap in small indie games.

What Golf?

Golf is a sport where people compete to get the fewest strokes by hitting a small ball into a sequence of various holes using a tool called a club, which can vary in use. Each course starts in a starting area called a tee box and ends when the ball falls into the hole. Courses have various hazards making it more difficult to complete this task. The course can have rivers, different roughness of grass, trees and sand-filled bunkers.

What is behind the game?

Go Golf is mostly a Python-based game, because of its simple-to-read syntax and use of an interpreter so it can check code line by line. In addition, python has one of the best error checkers compared to other languages, the language is OOP-based too, which allows the creation of objects. OOP programs are composed of several interacting objects, each responsible for its data and operations on the data. OOP allows objects to “talk” to each other by sending/receiving messages. Each object has its data (attributes) and operations that can apply to that data (methods). In adaptation, objects have ways of interacting with each other and objects do all the processing. All attributes and methods are wrapped into a single object and “hidden” (depending on the level of data/information hiding). Information/data hiding is where attributes (or sometimes methods) are hidden from the rest of the program. The benefit of information hiding is that it hides unnecessary data/behaviour that the user doesn't need to know how it works to use it. Details on how objects interact. The game also has a local database that is created on the first time every game is open, and user information is recorded, updated and retrieved through the use of SQL, a more specific SQLite module. Another big reason for the use of Python is the use of a module named Pygame. Pygame(the library) is a Free and Open Source Python programming language library for making multimedia applications like games built on top of the excellent SDL library. C, Python, Native, OpenGL”. This library allows me to create an intractable GUI window that allows me to further create my library to create my classes such as buttons, backgrounds, menus, etc. The game will also be hosted on Git Hub, where people can install the game with all its external modules, with the use of a virtual environment, meaning people only need Python to run the game. If errors occur the game has a logger to track in-game events. If something goes wrong, the logs can always be sent to me to look into.

Aims

The game aims are:

- To use OOP to create objects
- To successfully allow objects to interact with each other through “is a” and “has a” relationships
- To write clear and easy-to-read code using small methods/functions with methods to explain what the code does
- To get the most performance out of the language, because of Python being an interpreter
- Create a well-designed database using normalisation techniques, such as 2NF and 3NF
- To reuse code when necessary through the creation of libraries with different objects and functions
- To implement necessary abstract data structures such as queues or stacks when necessary
- To have a functional account creator/sign-in, with passwords that are encrypted and stored in a database
- To have accounts that can record and store their scores, coins and skins

- To have a physics-based system, that can successfully imitate ball trajectory
- To have a ball able to interact with the environment by slowing it down the ball or changing its path
- To have at least five playable levels with all their twist and turns
- To create an algorithm that can reward players who complete the level as fast as possible with the fewest strokes
- To have a clean and easy-to-navigate menu system to help users/players explore all game mechanics
- To have a camera that can follow the ball after a player makes a shot
- To have a camera that the player can use to discover the level before they take the next shot
- To create a store where players can buy skins for their ball, altering its appearance and permanently stored only for that player
- To have easy to navigate leaderboard that shows all players' scores, strokes and times for each game mode, descending from best to worst
- To implement a level creator that can create a level, alter and save levels through a tile system saved as a 2D array
- To create a level loader to load all levels from the level creator
- To reduce redundancy within created databases
- To create and find copyright assets to implement within the game

Objective

My primary objective was to create a video game that can be replayed multiple times, with a clean and friendly GUI that anyone can use. I wanted to create games that have replay value and can be changed with ease to always change up the experience, such as providing a level editor.

A secondary objective was to make a video game that used physics to create an interactive world. Allows a ball to fly across the world, and bounce off a wall into a hole. To create simple replicas of real-life physics that would be understandable to the player and that was also consistent.

A personal goal was to create a game that pushed my knowledge of Python and game development using pygame. I wanted a game source code to be readable, well-coded, and commented on. Another goal was to create a game fully utilising OOP for practically everything, getting inspiration from A level test game. I created games before that were as difficult as this game, such as UNO recreation which had dynamic CPU players, with menus and a leaderboard. However, the game source code is a confusing mess upon revisiting it. Its appearance and feel of the game were subpar and I wanted to create something that was just as hard but gave the feel of a quality mobile/indie/flash game.

DESIGN

Background Research

Golf games as a concept are nothing new. Throughout the years, video games based on sports (sports genre) have been some of the most successful. Within the sports genre, there are four major categories: simulation, manager, “sports-cade”, and arcade sports games. Each genre aims at to different target audience of various sizes, with the biggest being, the term I've coined, “sports-cade” and arcade sports games, while the niches being simulation and manager.

Simulation Sports Games

Simulation sports games are mostly racing games, such as Iracing, rFactor2, F1 series (simcade), Assetto Corsa, and Richard Burns Rally. These relate to a fanbase that only worries about realism and physics. Most of these games expect their players to come into them with a minimum of £250 to spend on entry-level equipment, such as Logitech g29, for force feedback steering wheels, pedals and shifts. This equipment can balloon to high prices for equipment to mimic real-life driving and can cause harm to people. An example of this is a direct-to-drive wheelbase (wheelbase hub/computer of the steering wheel and potential other connections. These hubs, for example, Fantect Podium Wheel Base DD2, cost **€1,499.99.** These devices can produce up to 25 Nm of torque on each side of the steering wheel. These can lead to injuries through wrist, thumb, and finger injuries from holding onto a steering wheel after a wreck. During a wreck, the steering wheel can snap side to side, based on the quickly changing position of the wheels or it freaks out to misreadings of in-game activity. These devices come with a warning saying that they are not recommended for children as quoted from Fantect ``Not **suitable for children.** A Direct Drive Wheel Base is not a toy! The fast movements of the steering wheel and strong torque can be dangerous.

Please read the safety instructions.”.



Some of these simulation games also have very high pricing for access to in-game content such as cars and tracks. The most popular example of this is Iracing, a competitive racing simulator with multiple series. After purchasing the game you have to pay a monthly subscription of $12.99, which also gives you access to a part of the game content, with further purchases still being locked behind this. However, the game is renowned by its community as being one of the best simulation games out there, further backed by real-life drivers such as 3-time F1 world champion Max Verstappen and FIA-backed series such as F4. Iracing also includes an elo-like skill system that ensures you are competing against a player with the same skill as the player. These games' esports scenes, (professional competitive racing) have led Turkish Driver Cem Bolukbasi to drive for the season in the F2 series, the closest series to F1, the pinnacle of open-wheel racing. These games are not made for casual audiences and are gated by a high sunk cost just to get into them. Developing these games takes an exorbitant amount of research, data-gathering and knowledge of physics, for example, complex tyre models that are constantly being updated.

These types of games cannot be really made for golf unless there is a complete simulator, not a game. This would just track where a golf ball goes and never be added with game-like elements. These also come with a high cost and are mainly used just to practise golf swings. The only thing I can take from these types of games is its reliance on using leaderboards and recording people's laps, to attempt them to outdo their last attempt and

beat others.



Sports Managers

Sport manager games are games that put the player into a coach, CEO or sports director-like position where the brief goal is to do all the setup for a match, race or game and let the game simulate an outcome with the decisions you came to. Examples of these games are the LMA series, Motorsport Manager, and F1 Manager with the most popular being the Football Manager series, with its latest release currently having a 24-hour peak of 70,392 players playing the game on 06/03/24.Football Manager is a game based on the sport of football/futbol/soccer, where the player simulates being manager at a football league. This allows the player a chance to experience what it is like being a manager at one of their favourite clubs and write their unique storylines, like the road to the premier league from an English national league side. The game includes real-life licensing for players, teams and leagues, with some imitation if the licence is not available.

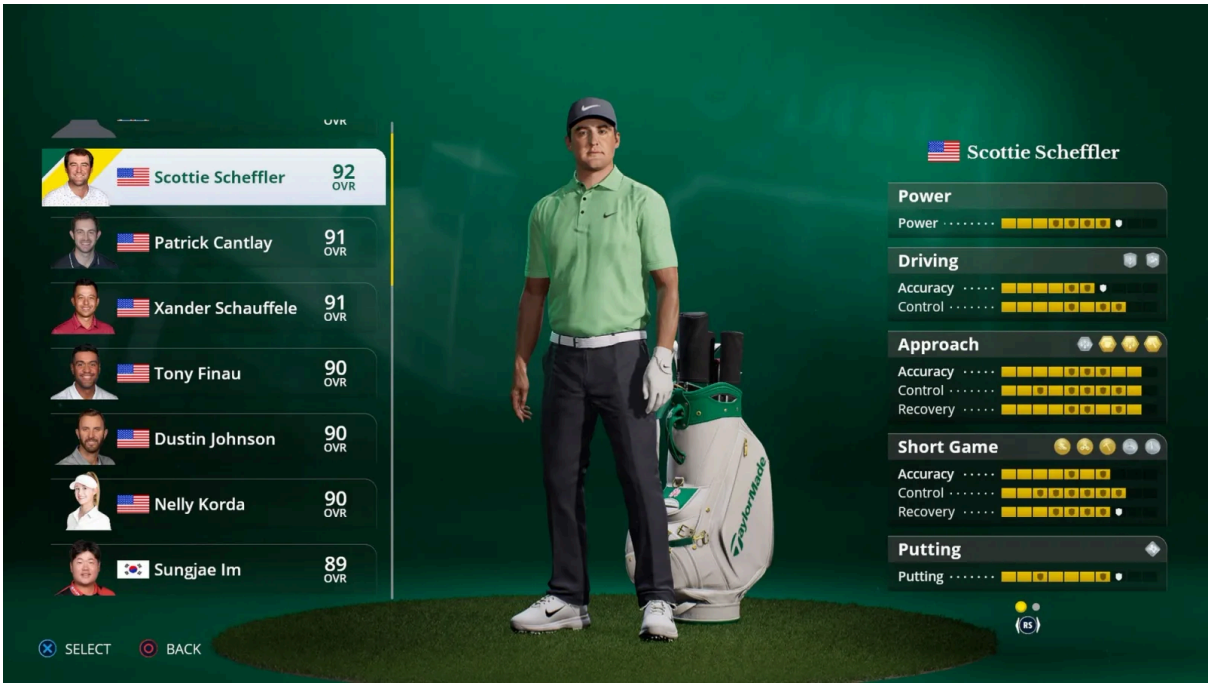
The gameplay of the games allows a player to simulate seasons, buying, selling, training, scouting and playing players while competing against CPU players whose seasons are automatically automated in doing the same thing. The player's goal is to compete against these teams in a typically 90-minute match, where the player may make tactics to score and defend against the rival team. However, unlike other football games such as FIFA or PES series, the player cannot be role/control of their players, as each player's CPU, who are given dynamic attributes on how they will react to your decisions, tactics and their fellow teammates. The game gives the same thrill as watching a real-life game on broadcast but with the ability to tell players what they should be doing. Manager-type games can vary from

quite simple to complex and overwhelming with how much control and specifics there are, such as creating recruitment strategy, worrying about financials, training, mentoring, set pieces, types of pressing, morale, tempo ect. These games have led to people getting real-life manager roles for example “the man who went from Football Manager to an actual football manager. From beating PSG on a computer game to holding them to a draw in real life, this is the incredible story of Will Still, the 30-year-old with one of the unique backgrounds in football”. These games are possible to develop with small teams and are a genre I would like to explore in the future, but I didn't think they fit the golf genre, as a sport is a more independent sport than a team sport. There is nothing I found of interest to add to my game from the genre.

Sports-cade

Sports-cade games are the name of a genre that I came up with based on the racing genre description of games such as Forza Horizon, and the F1 series sim-cade. These games are semi-realistic recreations of their sport, but with a few tweaks to make it more casual and video game-like. This would be a game like the FIFA series, PES/EFOOTBALL series, NHL series, Madden NFL series, 2K NBA series, F1 series, PGA tour, and MLB the show. Most of these games are released yearly with minor tweaks. Out of these, the most popular is the FIFA series, now known as EA Sports FC, with an all-time peak on PC of 107,109 players for the newest release in the series.

FIFA is a recreation of the sport of football/soccer/futbol where in summary a player gets to choose a team from various licence leagues, where they can change that team line up and compete against another team. The gameplay of it is that you can control all 11 players on your team and make substitution and tactic changes. You can change between selected players, when you select a player you can move that player in any direction, make them pass a ball, make tackles, shoot and even make saves if they are goalkeepers. The game pace is a lot faster than an actual football game, as players move and act in a more unrealistic manner. The match time is a lot faster than real-time, with the game average time being “12 minutes' ' when compared to the real-life match being 90 mins. Each player in the game is given attributes such as pace, attacking position, reflexes ect. These reflect the real-life abilities of players. They are then all wrapped under a single grade between 1-99, which gives players an overall score of how good they are, the higher usually the better. These normally have various game modes where matches are set against the player and either CPU or another player. The most popular mode in sports that have teams is normally card-based games, where players unpack players and use the player they got into their team. These players can normally be brought or sold through an in-game economy. The player rating can also get a very non-realistic boost, because of special cards that real-life players may earn because of a good game, season or no reason at all. This can lead to terrible players in real life having some of the best cards and being the best in the game mode.



So the PGA tour is the golf version of FIFA and the “sports cade" game. It contains similar features to real-life players and a semi-realistic recreation of the sport of golf. The game has a roster of real-life professional PGA golfers, who are each given an attribute to describe their abilities, such as power, accuracy, putting ect, leading to overall if their “combined stats '.

The game differs a bit more because of the independence of the sport of golf. On the first open you are required to create a golfer, given the name, age, country, and caddie. The game also allows you to alter the appearance of this golfer, through select presets of hair face and more. After you choose your desired swing style, alter parameters such as tempo. Giving the player a lot of freedom to create their dream golfer. This player was then given low overall, what the player meant to increase through playing with their golfer through different game modes or in-game purchases. Once created, the main menu greets the player.

The main menu greets the player with two ways to navigate it, horizontally to access different menus and vertically to scroll through menu options. On the play menu, we are given options to play different game modes like, quick play, career, and online. For an explanation of gameplay, I played quickly. Once quick play, select your given choice to play different matches, for example, The Ryder Cup, Skins, or Match Play. Once selected, the player is taken to another menu, where you get to choose a course to take a play on, with the number of holes you want to play. Then you get to select any golfer from the roster or your golfer, with the final menu before the game starts to check if you are happy with all settings and the course.

The gameplay of the PGA tour is a 3D recreation of the course with the player standing in the tee box at the start of the game. The player gets to change his club which lowers or increases the distance the ball can go, with further options with the angle the golf ball hits and what type of shot type. Where the ball will go can be seen with the zoom option which gives an area where the ball can land if a well-made shot happens. To make a swing, the

player has to pull LS (left stick) down until the club reaches the top of the arc. Then the player needs to push LS back up as straight as possible through a small mini-game.



If done correctly, a camera will follow the ball to its destination, while showing stats as did used the perfect amount of power or how well you swung the ball, based on how straight you moved the LS up. It also tells player stats about power, swing, the apex of the ball, the carry, a diagram of the arc and wind direction. This is repeated until the player uses the putter, where the mini-game changes. The game creates a path for the ball to be put to allow the ball to go into the hole. It works similarly to normal strokes but there is no arc anymore. Once putted each stroke is counted up and added to the overall. The player can not putt the ball in the amount of strokes and is stopped from finishing the hole and in the simplified version the total amount of strokes is added to the total. This repeats until a course is completed where you can compare your scores to CPU or Online players.

The game also features a store where players can buy XP boosts, new equipment, temporary stat boosts ect. What I can take from the game, is its menu style of having most of the options on the left with information being displayed in the centre. The game uses stats to allow players to know what is happening with the ball. I am also interested in how the game uses its camera to follow the ball after a certain distance has been made. Various game modes and courses also give me some interest in creating some of my own. Having a store is also an interesting feature, but having the ability to purchase boost with real currency creates p2w features I am against in video games, as it creates unfairness.

Arcade Sports Games

Arcade sports games are a genre of games that can be extremely very compared to the original sports. Examples of arcade sports would be Rocket League, Golf with Friends, Mario Kart series, and Mario Strikers. Further examples that are more golf-related are GOLF for

Famicom Disk System, Coffee Golf mobile game, Mini Golf mobile game, and WII/SWITCH sports.

Arcade sports games such as Rocket League, differ from their sport heavily. Rocket League's original sport is based on the sport of Football/Soccer, however with a big twist. Instead of football players, the game uses cars that can fly, spin and twist with the use of a booster. Each game still has a “football” and two goals for each team, but the physics, size of the pitch, goals, and arena differ completely. However, even though the game does not simulate the real-life game of football, it has created a very loyal esports scene price pool reaching $2,100,000.00.

Arcade Golf games were a genre that relates more to what the project is. The earliest version of the arcade golf game is the GOLF Famicom Disk System released in 1984. The gameplay is like similar PGA Tour games, but an even more simplified version of the game because of the limitation of hardware and lack of licensing. However, this wasn't the last attempt by Nintendo to create a game for their platforms that had golf. The game WII Sports was created by Nintendo mostly as a test demo to flex the captions of motion controls. WII Sports is, according to IGN, the fourth highest-selling video game of all time with approximately 82.9 million sales. The golf game on it is still very simple, but still similar to the PGA series of 3D gameplay, but using motion instead of a movable button. This shows how mainstream games that feature golf can become.

The smaller size of arcade sports games are the mini golf game by Game Pigeon and the game that gave me the idea to make a golf game, Coffee Golf by Shallot games for mobile devices. Coffee golf is a small daily golf game with a new course daily, with a scoring system similar to Wordle. Each course has five holes that you can complete in any order, you aim to do so with the fewest strokes possible. The game is a 3D-based camera from an angled bird's-eye view that can rotate in small angles around the ball. From that point the player can choose up top three clubs: driver (most distance), wedge (balance of power and control) or putter (catches no air and has the most control). To swing, the game uses a phone touch screen to measure the distance the player's finger is from the ball to decide how powerful the shot is (the further the more powerful the shot is). This is shown with an arc starting from the ball to the end position that casts a shadow at the end to display an area where the ball initially could land, but not the carry after. The goal of the game is to pot the ball in every hole in the course with the fewest overall strokes. The game also gives a visual of the total strokes and the amount it took for a certain hole. The game rewards players in an endgame currency after they complete a level and further awards if done in the top 25% and 10% lowest strokes. Each course has four obstacles: trees, elevation, bunkers and water that interact with the ball in different ways. Nevertheless, the game only limits you to that one course that occurs a day.



What I took from this game was the unique way each course is made, with it having multiple holes, with obstacles in between and maintaining a simple casual physics system so players don't have to worry about spin, air direction ect. In addition, its reward system for players taking part in the game is a good way to keep players playing, but it lacks fair economy within the in-game store, meaning the game rather brings it with real-life currency than the games.

Project Plan

My initial plan is to write out my dream scenario of the game, based on what I gathered from the research I did beforehand.

Dream scenario	• Account creator • Interactive GUI • Multiple Game Modes • Store • Working ball physics • Level Creator • Scoring level • Online leaderboard • Multiple levels with various of different hazards and obstacles • Multiple holes on each level, that reset the ball to its spawn • Scorekeeping system • Multiple skins for golf balls • Players earn a form of currency that they can use to buy items • Pixel art style • 2D plane game

I then separate the game into how dependent they are on the creation of the game.

Step 1(Essentials)	Step 2(Basics)	Step 3(Depth)	Step 4(Extras)
Background image that can scroll left or right with the control of the keyboard. To add a ball sprite, then make an algorithm to find the pixel distance between the user mouse and the ball. This distance will be the force on the ball. Add environmental physics such as gravity and air resistance. Level Editor with level saver and loader. Features such as dirt tiles, water tiles, and spawn locations. Allow the ball to collide with the tile, so the ball cannot fall into a void.	Background and ball move when the player shoots. Add golf holes to the game the ball falls into. With a checker to say that ball when in it. Reset the ball to the top level spawn location, if the ball falls off or there is another golf hole. Level changer feature Ball interacts with tile in a more dynamic way such as changing its path.	Create three various game modes, 3 holes, 5 holes and random Have at least 5 levels Add a menu system to navigate this menu The game has in time clock and to record strokes Score algorithm and end game screen Database creation for any further additions	Account sign-in/up that record into to database Scores, strokes and time record into user database for the user for that gamemode The game to have a currency that earned from playing games The game to have leaderboard system that can compare scores, strokes and time Skin can be brought with currencies. These skins are stored for that user and can be selected in games. Skins are only accessible by players who own the Hoster database that connects to all devices.

			Making the game leaderboard, account system, and skins online only.

For each feature, it will be: Planned -> Coded -> Tested by me -> Debug -> Tested again -> Implement -> Repeated. Each feature set was saved and uploaded to the GitHub repository.

Essentials

Golf Ball Trajectory

To start, I need to create the basic swing of a golf ball and where that swing will affect the golf ball with its environment. I wrote an initial plan on how to create an arcing golf ball on paper and then tried to implement it into the game. So I started a pygame window, where I can visualise changes for the game.

I would start by creating an object where the physics of the ball and its environment will be. I followed up by giving the object a golf ball sprite to create a visual and initially placed it within the centre of the window. I will then give the sprite a mass based on KG and added environment forces such as gravity and air resistance. I contained all these within the same class, as the golf ball was the only sprite that needed to be aware of physics rules. So when the game runs, the ball should fall straight on initialisation. So I added a condition to stop the game from running it physically until the right mouse button was clicked.

. The 1.5 acts as a multiplier, so the player does not need to move his cursor extremely far to gather power for the swing. In addition, I added a limit of force/power of 1000, to limit the player and to make the game more challenging. However, with a further multiplier, it is optional to get out of tight situations when the player cannot move the mouse far. for finding the angle the player wants to choose we use the same dx and dy with the tan equation of

```
dx = mouse_position[0] - self.__BallPostion[0]
dy = mouse_position[1] - self.__BallPostion[1]
self.__BallSpeed = sqrt(dx**2 + dy**2) * 1.5
self.__BallAngle = atan2(dy, dx)
self.__BallSpeed = self.__power_increaser()*self.__BallSpeed

if self.__BallSpeed > 1000:
        self.__BallSpeed = 1000

if degrees(self.__BallAngle) < 90 and degrees(self.__BallAngle) > -90:
        self.__direction = -1

if abs(degrees(self.__BallAngle)) > 90 :
        self.__direction = 1
```

One annoyance of pygame is how to record position (x, y), as the origin of it starts at the top left corner of the window with the y value increasing from further down you go. This means I need to make small offset changes to make sure I get my desired numbers. This becomes apparent when creating my initial velocity of x and y. Instead of the normal formula and $Uy = usin(\theta)$ I added a minus applied to both values after the formula.

```
#Initial Velocities
XVelocity = -self.__BallSpeed*cos(self.__BallAngle)
YVelocity = -self.__BallSpeed*sin(self.__BallAngle)
self.__VelocityOfBall = [XVelocity, YVelocity]
```

Afterwards we need to give the ball trajectory using these attributes. To constantly update flight we need gravity and air resistance to create the ball path. We first multiply both x and y velocities by air resistance to our own list. We also create another list variable with gravity “[0, g]”. After we add the bottom h of together (x1 + x2) and (y1 y2) into a variable called acceleration. We then add acceleration divided by the ball mass to the ball's X and Y velocities. We also apply these updates to the ball sprite position. However, without collisions and a way to stop these equations at the moment, the ball will fly into the abyss of the game. One thing left out is the use of variable update steps that are used to “slow down” the ball to make it easier to track.

```
#Calculation
        gravity_acceleration = np.array([0, self.__GRAVITY])
        air_resistance_acceleration = -self.__AirResistance *
np.array(self.__VelocityOfBall)
        acceleration = gravity_acceleration + air_resistance_acceleration

        #Updates velocity
        self.__VelocityOfBall += acceleration / self.__BALL_MASS * update_step


        #Updates Ball position
        self.__BallPostion += self.__VelocityOfBall * update_step

        #Updates Ball rectangle position
        self.__GOLFBALL_rect.x, self.__GOLFBALL_rect.y = self.__BallPostion.astype(int)
```

Camera, Tracking and Scrolling

Camera is needed in the game to track and see where the ball goes in the level. Three things the camera does in the game: scroll around the level, after the player looks to be to able snap the ball centre to the screen for the player to swing and after following the ball because each

level size is approx 7,200 in-game pixels of playable level, we need to implement a camera that scrolls. To do this I first added a background to the game and created a variable start from 0 called background scroll. This scroll will add 5 or remove 5 for each tick of the A and D key being pressed. This number will then move the background image by the same amount, however, we need to render multiple images to make it look continuous in both directions. To do this we need to make calculations to decide how many images we need to render:
Locking the camera to the centre of the screen when the player is setting its swing is to allow the ball to always be on screen when the player is swinging equally. It stops the player from being able to see always forward making it so the player has to spend more time thinking about it. To lock the camera, I'll use the enter key, with the exit being right-click to return to scrolling. Setting the camera was more complicated than expected originally. I first have to figure out the offset of x when compared with the game golf ball position, allowing the ball to always be locked in the centre when the player wants to swing.Once the ball swings the game is able to follow the ball detestation. This is done by adding a threshold. If the ball passes the threshold on either side, the game camera should start moving with it. This camera uses a player velocity divided by 115 to adjust how fast the camera should move. The result calculator is timed by the number 2.7 and used to just scroll the world, the ball position and the distance moved. A lot of numbers gathered were through trial and error to keep the ball on the screen, with the camera either too fast or slow.

Level Editor

To create levels for the game, it needed a level editor. The level editor can manipulate, save and load levels, with a level range of different tiles. To do this I was heavily inspired by a tutorial I found for creating Tile Map levels by YouTuber CodingWithRuss. This level editor uses a grid system that saves rows and columns in 2D arrays, which contain images in a grid space.

Basics

First creating the basics: I needed to add scrolling, a grid and a side menu. Adding scrolling is the same as implemented in the main game, however it will be overlaid with a menu and a grid. To draw the grid, I need to set the maximum number of columns within the level for the distance of each level, further, I need to set the amount of rows to allow a fixed amount of placements of blocks/tiles within each level. We also need to work out the size that images should be scaled up to create a constant amount of grids and scalers for images. To draw the vertical lines we start from the left of the window with a definite iterator at -2 (for no added reason later on) to the maximum number of columns we want. Each iteration draws a white line based on the number the iterator is on multiplied by the tile size added by the scroll distance of the level for the x coordinate of each line. This can then evenly move and draw a grid throughout the level that is evenly spaced. To stop the line, I set it to screen height, so it could not move further which would hurt performance. This is repeated to draw

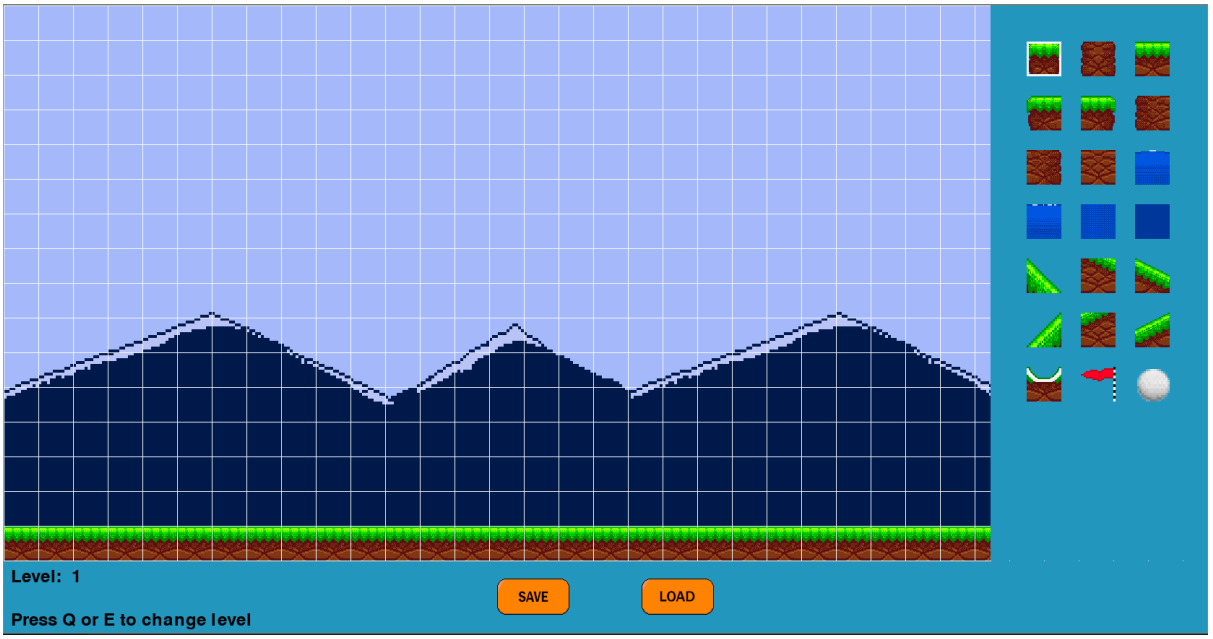
each row, but it is reversed with y axis acting being affected. The grid will then have its sides drawn over for places to add buttons, such as Save, Load and tile options.

Selection

We need to add a way to choose what tile we want to place. I implemented this with buttons that display the tile to place. This uses buttons that I created (explain later), it takes an image, coordinates to create the button and a scalar. List images are turned into buttons and given a coordinate to where they should be positioned. Each button is then drawn and can be selected. Each button can only be selected at one time, which is highlighted white outline. To figure out what button tile is selected each has its own number.

Placement

To start with placing down objects, the game will record the mouse position of the user. We then need to map and adjust these coordinates to our tile coordinates. This can be done by taking the (with change adjustment to scrolling),y coords and applying integer division with tile size. We can now use these coordinates to add and remove tiles into the world.



Each level has a 2D array (columns and rows) with data for what is in a tile space. Depending on the data within an array this used to put the tile into that space. There are up to 22 placeable tiles, including nothing. So for example blank tiles are left at -1, flat grass tiles are 2 and goals are 19.

So as we now can get the position of the cursor, if we let the user press the right mouse when a tile is selected we can use the x and y coords to select and update the information within the 2D array for the level. We can also make a left click to remove this data and set it back to -1.

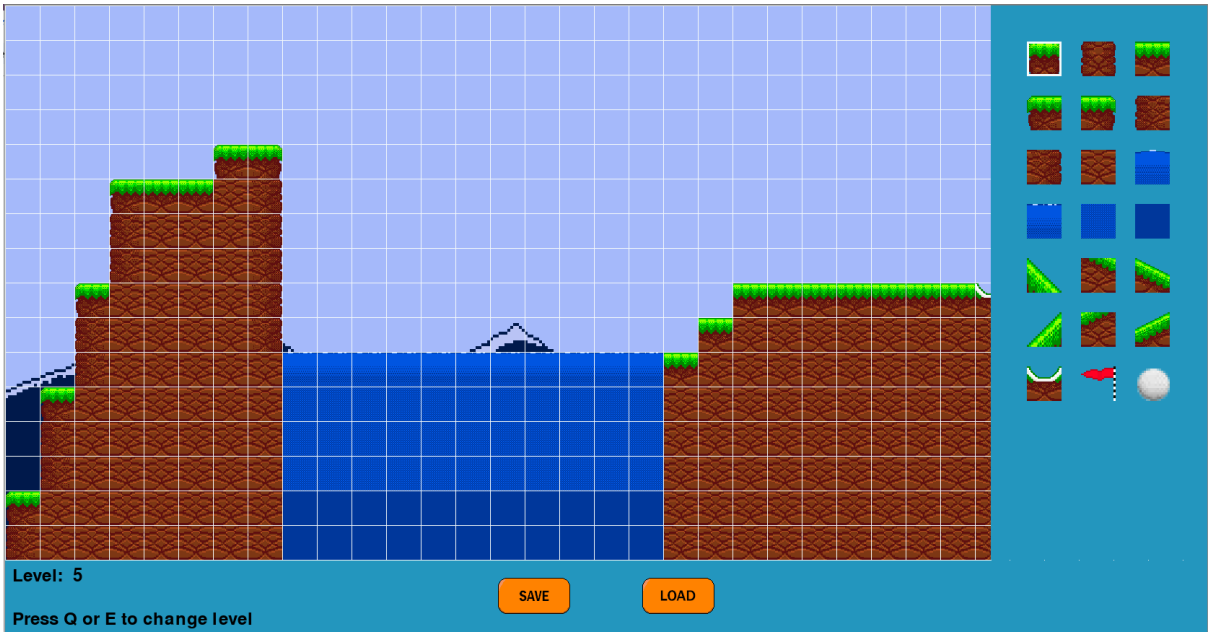
Displaying

The editor is unable to give visual updates on placement of tiles within the level. To, do this the level editor uses two definite loops, one for each row and the embedded loop being for the data with the tile. If the data within the tile is not -1, we draw the image to the right position using x coords multiplied with tile size to get tile coords (with scroll offset) and this done to y coords too. The level editor just records images/tile position and nothing more, as tile attributes and behaviours will be handled within the main game.

Saving and Loading

The editor should be able to save multiple levels and load multiple levels. The editor should be able to have multiple save lots, to select levels and from these saves, to have a confirmation to avoid accidentally overwritten data.

Editor needs to be able cycle between different levels, to do this the editor gives visual information of how to change levels (and E) and show the current level through text.



The editor only creates 8 levels, as the game does not benefit from more than that.

After selecting what level you want to save to, the user can press the save button where the editor waits for confirmation of the enter/return key. If any other key is pressed the level is not saved. To save we use Pickle, a Python module that uses object serialisation. “ Serialisation is the process of converting an object into a stream of bytes to more easily save or transmit it. The reverse process constructing a data structure or object from a series of bytes—is deserialization”. This can allow us to store data in the exact state it was before being saved and the data type of the data we are storing. This allows the editor to store each level's 2D array in a pickle-specific file.

With pickles, we can also load these levels, when selected. However, a warning is given. These files should be accessible to the game too.

Basics

Game Levels

The game contains 7 playable levels, each with its own level structure, however, the game is a way to read this data, differentiate, give purpose and be able to change/select from each level for a certain gamemode.

So when a game of golf is created, the GoGolf constructor for the game should add levels to the queue data structure based on the parameters given to it. The queue FIFO data structure is dequeued and returns the level in an order after a level has been completed. This repeats until the queue is empty. A queue will contain the name of each level 2D array. This allows the game to save the 2D array to an attribute called world data. When deque-processing this data we need three things: Golf Ball object, list of tile images and world data.

```
for y, row in enumerate(world_data):
            for x, block in enumerate(row):
                  if block >= 0:
                  img = tile_img_list[block]
                  img_rect = img.get_rect()
                  img_rect.x = x * TILE_SIZE
                  img_rect.y = y * TILE_SIZE
                  tile_data = (img, img_rect)
                  if block <= 7 or block >= 12 and block <= 17:
                        tile_data = Tile(img, img_rect, x, y, "obstacle")
                        self.obstacle_list.append(tile_data)
                  elif block >= 8 and block <= 11:
                        tile_data = Water(img, img_rect, x, y, "water")
                        self.obstacle_list.append(tile_data)
                  elif block >= 18 and block <= 19:
                        tile_data = Goal(img, img_rect, x, y, "goal")
                        self.obstacle_list.append(tile_data)
                        self.__goals_list.append(tile_data)
                  elif block == 20:
                        golfball.set_ball_postion(x*TILE_SIZE, y*TILE_SIZE
+ 28)
                        self.__ball_spawn_location = (x*TILE_SIZE,
y*TILE_SIZE + 28)
```

To process the data the game is going to loop through the levels world data, to access each tile saved .data e.g 7. The images are given exact coords by reverting from tile coordinates

and the image turns into a pygame rectangle object. The data is then sorted into four categories: Tile, Water, Goal, and Spawn location. The image, image rectangle, coordinates, and type of data are then turned into objects unless it is a golf ball or nothing. Tile, water and goal tiles will be sent to the obstacle list to be drawn (goal objects are added to the list of goals in the level). The golf ball tilesets spawn position for the ball within the level. The game also has a total travel scroll (distance moved) that uses the same keys as the background scroll, but never resets after a certain threshold. This is used to move the world data by changing x coordinates for each tile. Only tiles that are visible to the user at the time are rendered.

Tiles

Tiles are objects within the program that are used to give tile image attributes and methods. This object saves the tile image, pygame rectangle, coordinates and type. The behaviours of the object are accessors and mutators.

Tile class is a superclass for its two inherited subclasses: water and goal. The water subclass currently has no other use than having a different tile type. The goal subclass uses polymorphism to Tile class attributes and methods. The class loads the goalpost above the active hole. Once a ball enters the hole, the hole is no longer active.

```
class Goal(Tile):
    def __init__(self, img, img_rect, x, y, img_type):
        super().__init__(img, img_rect, x, y, img_type)
        self._goal_post =
pygame.image.load("Assests/tile/19.png").convert_alpha()
        self._goal_post = pygame.transform.scale(self._goal_post,
(TILE_SIZE, TILE_SIZE))
        self.__goal_active = True

    def Display_Goal(self, distance_moved):
        if self.__goal_active == True:
            screen.blit(self._goal_post, (self.get_rect().x +
distance_moved, self.get_rect().y -1*TILE_SIZE))

    def set_goal_active(self, value):
        self.__goal_active = value

    def get_goal_active(self):
        return self.__goal_active
```

Collisions

Collisions are essential to this game as They add iterable and life to the levels. Tiles need to have collisions to stop golf balls from falling through the level. Furthermore, collisions can be used to add obstacles to the game where the ball has to avoid them to get to the destination. Collisions are used to detect when the ball goes into a hole. Collisions are useful as they can tell a golf ball may not be able to travel further than a certain limit in that direction. The game additionally uses collisions to alter game ball flight, velocity and direction. This can offer ways to slow them due to friction, leading to a way to stop the ball.

Collision in the game works after the ball has been first swung. The game continuously updates at a max rate of 60FPS where the trajectory of the ball and what the ball interacts with are being constantly monitored and updated. A collision detection algorithm is run after the ball has been swung. When the golf ball is approaching a tile surface from either side, the game checks what type of tile it is approaching and if it is about to go through the tile. When the ball is about to interact with a generic tile, the algorithm checks if it is approaching its top or bottom Y position and left or right X position. For Y collisions the game checks the distance level moved with where the x position the ball is, at the same time the game checks for the y position of the ball added with the next frame position of the ball and with a buffer of 1. similar to a golf ball approaching x of a tile, however, the game will then check where the next frame x coordinate will hit.

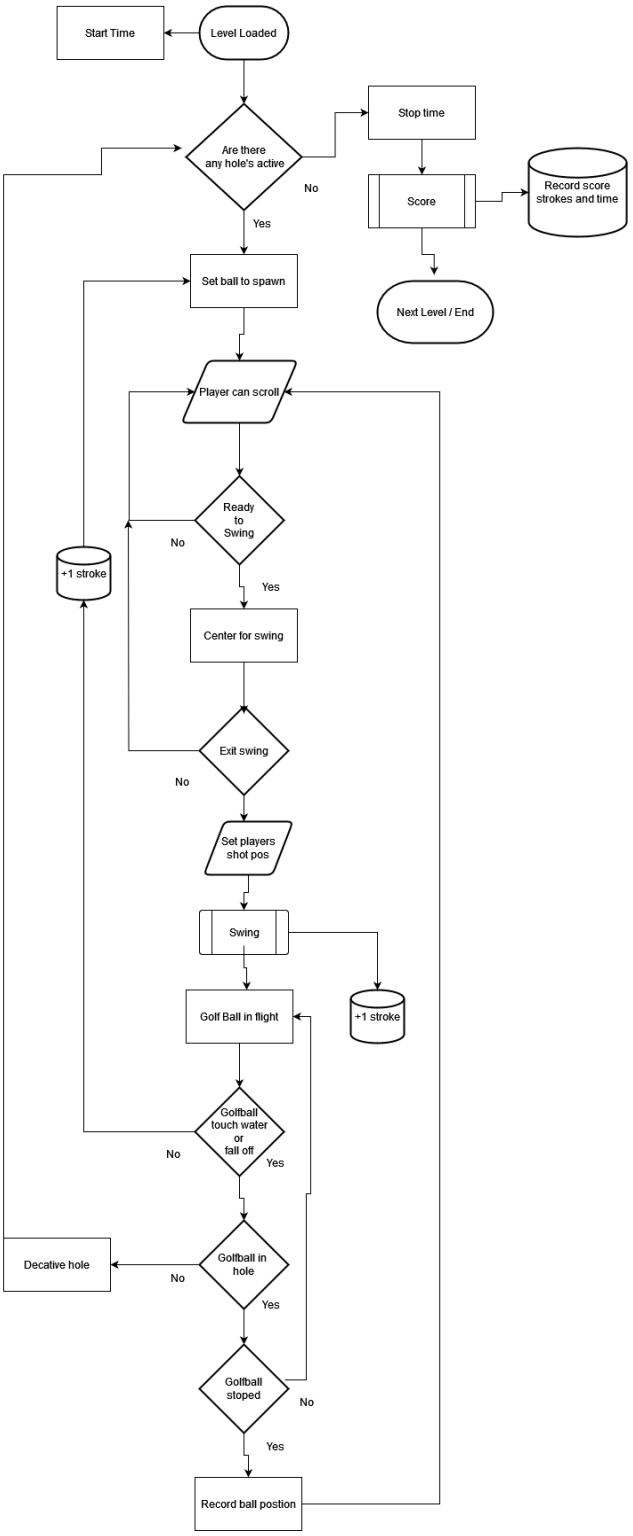
When collisions occur the game has handlers for it. The collision handler hits tile check Y positions to check how fast the ball is approaching, if fast, flip it velocity Y and divide it by number to make lose energy, if going fast enough it also removes velocity from X velocity. Depending on the velocity of the incoming golf ball the game will react to the ball in different ways. If the ball goes slow enough the game has a stop variable to tell the game to stop updating the ball, save its position and now get ready for the next swing. Allowing the player to move around in the game. X-hander for collisions is a lot simpler, as it just affects the direction variable and flips the x coordinate of the ball while removing some of its energy.

Goal tile has its type of collision, as we want to allow the ball to fall into the hole, tell the game it has been entered and take the ball back to spawn. The game does this by creating a small bit distance the ball can travel on the tile by creating a limit. Once past this limit, it falls into the tile, as the tiles are above the layer when the ball is actively moving. When the ball falls into it, when it crosses y level it will deactivate the hole and the ball will reset if there are more active holes. Water tile when hit just resets the ball straight away but punishes the player with an extra stroke of plus one. This further occurs when the ball falls off the world, after a certain y limit it will reset with the same punishment.

Depth

Gameplay

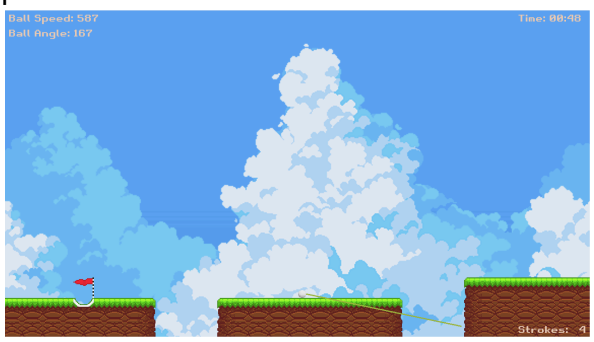
Due to having a way to stop the shot and reset for the next swing the game can now have a gameplay loop for each level. This can be represented through a flowchart below.



The player loads in the level, where the game will start a timer. The game will check for how many holes there are in the course. The game will then start checking if they are active (the ball has not gone to them yet). The game will continue this check after every shot until there are no active holes in the game, where it will stop the game, and calculate the score based on time and strokes. Score, strokes and time will be recorded for each level.

Each level will have a world spawn where the ball will start at. If the ball ever goes into a hole, touches water or falls off the world the ball will spawn back here.

Before the player swings, they are allowed to scroll through the map to plan where they want to see what hole they want to target. Once they're ready, the game locks the screen and the player is ready to swing. To swing, the player is given information about the angle of their swing and the force. This information is calculated through mouse position, with an altering colour line from the mouse to the ball to show the power too.



Once a player is happy they can click right click to shoot or left click to scroll again. Once the ball is in flight, a stroke is recorded. If the ball hits water or falls off the world, an extra stroke is added. Otherwise when the ball stops the player is allowed to get ready for another swing. If the ball goes into the hole, that hole is now not active and acts as a normal hole.

Once every hole has been entered, the game will stop. The game will then calculate a score. The strokes, time and strokes will be recorded and totalled with future or previous levels or none depending on the gamemode. This will continue until there are no more levels in the queue, where the game will save the total collected stats from each level/course.

Gamemodes

The game is planned to have three game modes, two competitive and one quick-play type mode. The competitive game modes will have stats, e.g. score records. These game modes will be 3 holes and 5 holes. Each 3 holes will play three courses/levels with score, strokes and time accumulating from each level. This also occurs in 5 holes, however additional 2 other courses are chosen. These game modes will have the same courses being played, to remove all RNG occurring during gameplay. These game modes will also have a representing leaderboard, where players can compare scores, strokes and time.

Further, the game has one additional gamemode called random. Random is a quick play mode that temporarily tracks players' scores until the end of the course. Any result is not saved to a leaderboard. How this gamemode works, it randomises a level out of 7 available courses/levels to allow the player to play.

Scores algorithm

Score systems have been around since the conception of video games, such as Space Invaders, Pac-man, Tetris and Donkey Kong. Most of their systems were used to make players want to replay the game, to try to beat the machine's highest score or their person's best score. Score systems are still heavily used to this day in games but can be used in other ways, as the Burnout 3 progression system uses scores for unlocks of levels and cars, and Resident Evil 6 Mercenaries gamemode uses to judge a player's performance with a rank. These both lead to players having reasons to continue to play.

These scores can be calculated through parameters that the game can use to judge and calculate a score. Within Go Golf three parameters are used to calculate score: strokes, time and holes. For every hole the game adds an extra 1000 score to start the course e.g. 5 holes would be 5000 starting score. This is to counter and reward players due to extra time it might take to complete these courses, however, the game will reward them with more if completed with fewer strokes and time. Strokes use brackets to decide the divider the time should be divided with.

Limits	Divider
0 to 9	14
10 to 15	10

16 to 20	8
21 to 28	6
29 to 35	2
36 and over	- amount of strokes/10

So the algorithm will take this divider and use it to divide the time by it. After the game will take a number calculated with the use of a divider and multiply it by 100. Finally, the number is divided by the starting score and rounded up. The full formula is:
$Round(((StartingScore/\ (time/divider))\ *\ 100))\ \ +\ TotalScore\ =\ Total\ Score$. The total score is the accumulated score of levels played within the game during that attempt. These scores reward fast and low-stroke games.

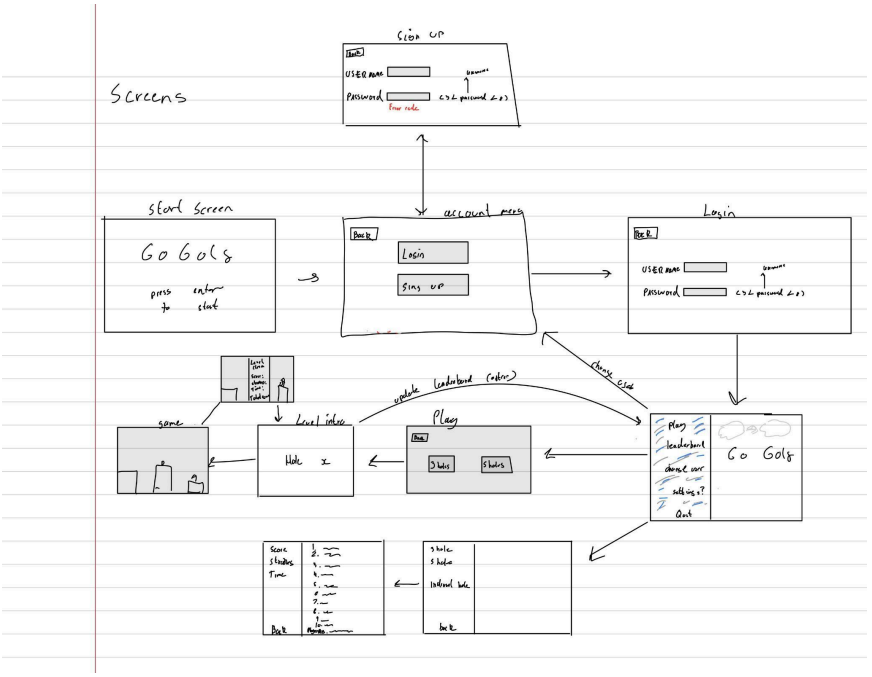
For example , one level I had the time of 78 seconds, 13 strokes which means a 10 divider and the level had 3 holes so 3000 starting points. The level rewarded me with 38,462 score of 38,462. The second level has the same amount of holes (3000 starting points), but it took me 172 seconds and I had 35 strokes. The game rewarded me with 3,488 points.

```
def score_calculator(self, amount_of_goals):
        """Calculates score"""

        starting_score = amount_of_goals * 1000
        LOGGER.info(f"starting score: {starting_score}")
        divder = self.__divider()  # Replace 1 with the default value
        LOGGER.info(f"divider: {divder} and strokes: {self.__strokes}")
        LOGGER.info(f"Time {self.__time} and score {(round(starting_score /
(self.__time / divder)))*10}")
        self.__totalscore += round(((starting_score / (self.__time /
divder))*100))
        LOGGER.info(f"Total score: {self.__totalscore}")
```

Menus

Game Menus allow the player to navigate through the game's features. I originally drew a basic



sketch of how I wanted to design the menu system and I want to keep the game as its object and the menus as their own. So each new game is a new game object.



Start Screen

Game starts with a start screen displaying a game logo, asking the player to enter a button to start the game. The logo and text oscillate up and down, using a continuous sine wave that affects the image y axis. This gives the appearance of a floating logo look. The game

will then take the player to log in and sign up to a menu where the player can make an account or log into an account.



Main Menu

Once a player logs in, they are transported to the game's main menu. The layout of all menus after this is menu options are to the left of the screen and the rest visual information of the menu, making game menus feel more seamless. The menus use button objects that return what selection they made, which will complete an action such as locating the player to the new menu, starting the game, exiting the game ect. Players on the main menu are given five options: play which allows the player to choose a gamemode to start a game, leaderboard allows the player to view all player's stats, change user to change player account, skins to change and buy golf ball colours and quit to exit the game.



Game mode choice menu

This is handled by an object called Tee Box that handles the three object menus: start screen, account creator and login in and main menu.

Data Structures, Objects and Algorithms

Buttons

Buttons is an object that returns a boolean result, depending if the user uses the object. Button object is a digital version of a physical button, it returns True if the object is “pressed down”, else it continually returns False. To create a button the object requires: position, image and scaler. The object works by monitoring the position of where the mouse cursor is. If it sees that the mouse cursor is hovering above it, the button image is brightened by brightening all RGB colour channels by a factor of 1.5. This displays to the user that the button is clickable, however the object won't return True until the button is pressed. To press the button, the user needs to press down left mouse button. The object won't return True until the left mouse button is unpressed. The issue of continuous mouse press can occur when clicking the same button object because of a user still pressing the left click.

Text

Text object allows the game to display text to GUI. Used to guide players through the game by giving them relevant information. For the object creation to display text it needs text, font,

size and position. All attributes of the object can be altered independent of time. If a text object is drawn it is rendered onto the screen for that frame, until not called.

Enter Text object allows the user to type into the GUI of the game, displaying pressed keys and recording the characters recorded. The object uses another object created by GitHub user Nearoo. The object at construction needs to position, font, size, anti-alias and validation to use the object the game will draw the text object for all characters being displayed. A method will be called to record all characters that are currently being drawn. Then it can be retrieved through a getter called access and cleared by a method called clear text. This method can be seen as being used by the game account creator.

Queues

Queue is FIFO and holds an order, linear sequence to be added to. Queues have two pointers, one pointing to the front of the queue and one to the rear. When elements are enqueued they are added to where the rear pointer is pointing to and the pointer increments to another position within an array. When elements are dequeued the front pointer returns the data in that index in an array and the front pointer points to the next item in the queue. When data is removed, data in the array doesn't move to the taken space.

The game uses the queues for the level loader. The levels added first will be the first dequeued. This queue object only exists when the game object exists meaning the game never has to worry heavily about wasting space.

```
class queue:
  """Queue data structure"""
  def __init__(self, capacity):
    self.__capacity = capacity
    self.__array = [None]*capacity
    self.__fpointer = 0
    self.__rpointer = 0

  def enqueue(self,item):
    if self.__rpointer == self.__capacity:
      if self.__fpointer == 0:
        print("Queue is full")
        return
      else:
        for i in range(self.__fpointer,self.__rpointer):
          self.__array[i - self.__fpointer] = self.__array[i]
        self.__rpointer -= self.__fpointer
        self.__fpointer = 0
    self.__array[self.__rpointer] = item
    self.__rpointer += 1
```

```
def dequeue(self):
  if self.__fpointer == self.__rpointer:
    print("Empty Queue")
  else:
    self.__fpointer += 1
    return self.__array[self.__fpointer - 1]
```

Ingame Timer

Timer is needed within the game to track how long the player is in a level. A timer is able to track a second within the game and not in real life. As frame rate can make a game technically go faster or slower when compared to real life time, which can lead to two separate games being played the same, but one will end with a better score as their frame rate was faster. In each frame the game cycles through the code, so if it has a slower frame rate, the action will be completed slowly.

The game's timer works by keeping a count of each time the game loop cycles around. This counts to be compared to pygame tick function, but not the target/max framerate. So when the count is equal or greater, the count is set to zero and the time increments by one. The game will then display this information in the form mins: seconds.

Switching of Image Layers

Similar to editing platforms where images can be overlaid on top of each other the game images have layers. Layers are used to hide, convert and display information, data and images. So the game can switch between layers to hide images behind others during a certain time or the other way around.

When the player is allowed to scroll around the level of images in this order (left lowest): background, tiles, golf ball, then UI info. When the game locked an extra image was drawn to the screen, which was inserted between tiles and a golf ball. This is a liner to help the ball aim. This allows the line to be behind the ball but over the tiles so it can be always seen. Once the ball is swung the liner is no longer rendered. The game will draw the tiles after the golf ball, this allows the ball to “fall into” the whole object.

Extras

Database

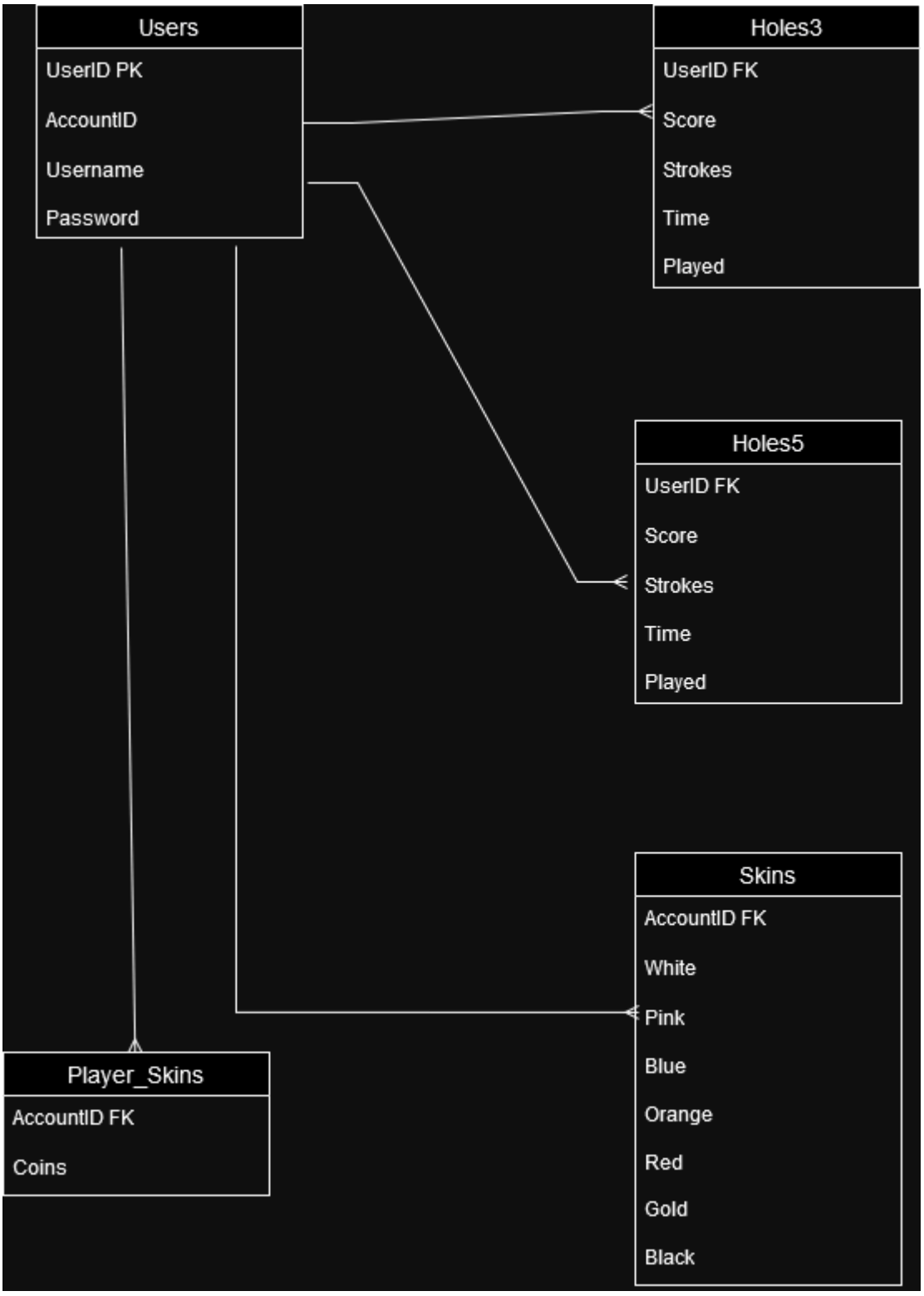
Being able to record and retrieve data from a stored collection of data created by the game is essential for a lot of the end-game features of the project. For the next three major additions, we need to be able to store data permanently. To do this the utilises the database and can hold game data forever. For DDL and DML the games use SQL through an external module named SQLlite3. SQLite3 allows programs to have pre-written/flexible queries embedded within the game.

The database needed a plan for its design before its implementation. The database design was resigned after the first implementation produced a lot of redundancy, due repetition of data, unnecessary relations, not being very readable.

So on redesigned, I knew what Entities I needed and what attributes I needed to represent each relation. Together there are five relations: Users, Holes 3, Holes 5, Account, Skins.

Entity	Attributes
Users	UserID, AccountID, Username, Passwords
Holes3 / Holes5	UserID, Score,Strokes, Time, Played
Skins	AccountID, White, Pink, Blue, Orange, Red, Gold, Black
Player_Skins	AccountID, Coins

Go Golf ERD


Users
UserID PK
AccountID
Username
Password
Holes3
UserID FK
Score
Strokes
Time
Played
Holes5
UserID FK
Score
Strokes
Time
Played
Skins
AccountID FK
White
Pink
Blue
Orange
Red
Gold
Black
Player_Skins
AccountID FK
Coins

Relations Attributes Datatyps and Constraints

Users

UserID	Primary Keys, Auto Increment, Not Null
AccountID	Int, Unique, Not Null
Username	Varchar(8) Not Null, Unique
Password	Varchar(8), Not Null

Holes3/5

UserID	Foreign Key, Int, Not Null
Score	Int
Strokes	Int
Time	Int
Played	Default Current Date

Skins

AccountID	Foreign Key, Int, Not Null
White	Bool, Default 1
Pink	Bool, Default 0
Blue	Bool, Default 0
Orange	Bool, Default 0
Red	Bool, Default 0
Gold	Bool, Default 0
Black	Bool, Default 0

Player_Skins

AccountID	Foreign Key, Int, Not Null
Coins	Int, Default 0

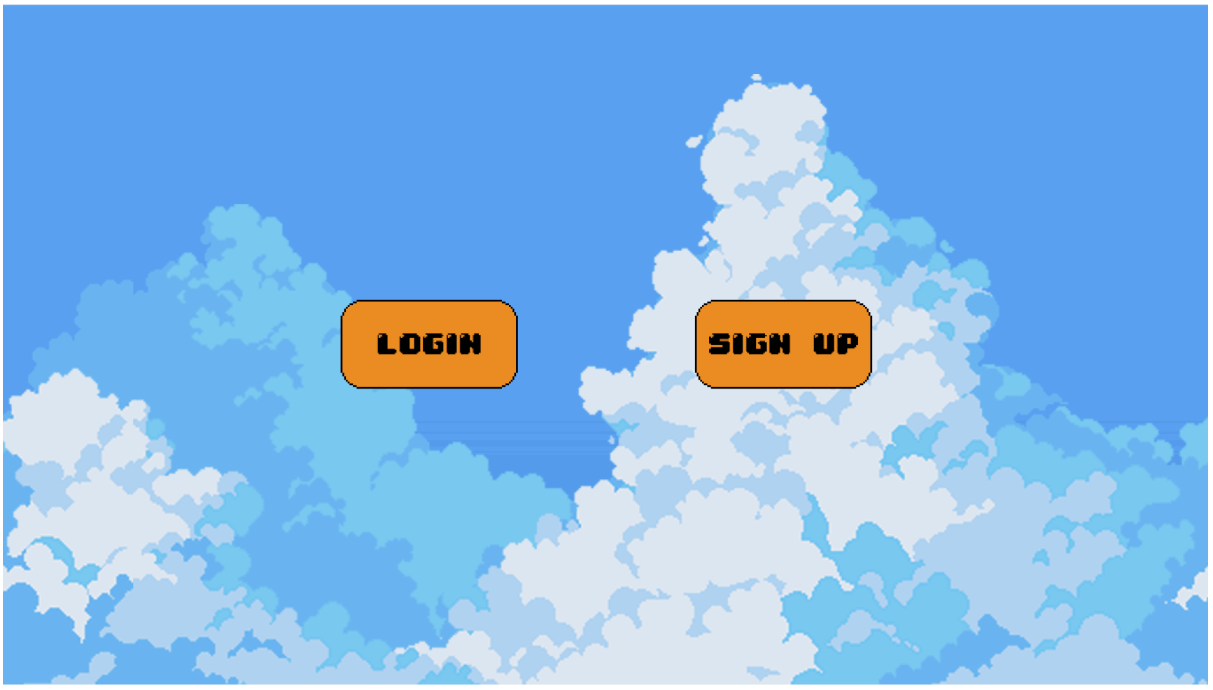
Database Creation

On first boot of the game on the player system, the game should check if the game has a working database in the game files. If not the game will create the database through pre-written DDL queries to create the database and all its tables. For example:

```
def create_register_table(conn):
      c = conn.cursor()
      c.execute("""CREATE TABLE Users (
      UserID INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
      AccountID INTEGER UNIQUE NOT NULL,
      Username VARCHAR(8) NOT NULL UNIQUE,
      Password VARCHAR(8) NOT NULL
      )""")
      conn.commit()
```

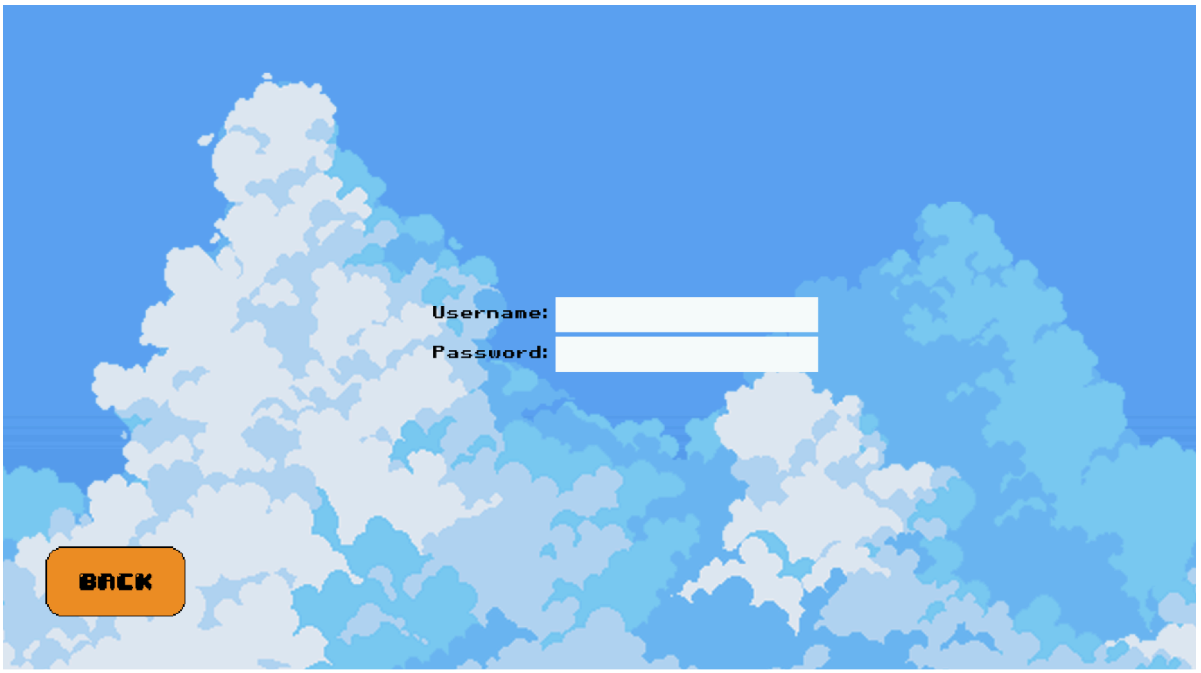
Account Creation and Login

Account menu is seen after the start screen of the game, offering the user two options to continue: Login and Sign Up. Accounts play a crucial role within the game as they are used to track user activity throughout the game, so forcing the user to have an account is necessary.



Creations

If a player does not have an account for Go Golf, to proceed to the main game they will be obliged to create one. To do this they have to click the Sign Up button. The game will display a screen asking you to type in a Username and a Password (same screen for login).



The username of player signing-in rules are that there are no spaces (they will be replaced with underscores), usernames have to be unique, usernames have at least one character including blank (will be replaced), lowercase (will replace all uppercase values) and can have up eight characters. If any rules are broken, the username will be viewed as invalid and rejected, leaving only a valid username. The password rules are a bit different: No empty spaces or space at all, all cases can be used, up to eight characters and do not need to be unique. The password is also further encrypted, through a lossy super encipherment encryption to make the password unreadable.

If the data is all valid the player details will be recorded, however, before the game further needs to create another piece of data to uniquely define a player called accountID. AccountID used to manage players' wallets and skins, as they are linked to this ID. These IDs are created by adding all the unicode values of each character. One issue with this method is words such as care and race have the same total value in unicode. So to fix this I used the method used in hash tables, by incrementing collisions/repeating data by one until the value is unique within the database. Now data can be recorded into the database and the last attribute UserID is given a unique incremented value for the player. This all records through Insert query, however through a method to avoid injection attacks. These queries are further continued by pushing data to Skins and Player_Skins tables. After the player is back to the account menu.

Log in

If a player has an account within Go Golf, they can sign in through the login button. Once pressed they are shown to the menu screen to enter their details. Once the player types their details the game will check if they are valid within the game database. If the username is not in the database it will return a "username/password is not correct" which is purposely vague. This will also be returned if the player's username doesn't match its associated password. To figure out if the password is valid, the game temporarily encrypts the signing player's password and compares it to the password stored in the database. If all are valid the player is sent to the game's main menu with all features such as wallets and skins being associated with that account being loaded.

Encryption

Passwords in the game are encrypted as a database is saved locally, people may use important passwords which can be easily accessed. To avoid passwords being leaked being saved in plain text, the game will use Super Encipherment cipher to leave data unreadable. Adds an extra layer of security, however, it still might be accessible to recover plain-text passwords.

The encryption works by passing the player's password to the encrypt function. It each letter to its Unicode value, to further convert each character's Unicode value to binary. Once converted to binary, the game applies an 8-bit xor key to each character (if the character binary value is greater than 8 bits then each value after is lost). After each character then returns to denary/unicode values. After a caesar-applies applied to each character, further scrambling it. After the value is converted back to a string and saved into the database. The only drawback of this is that due to the loss of data it can be very hard to decrypt, the xor key and Caesar key are accessible within the code as the game is local.

```
def xor_cipher(value, xor_key):
    encrypted_values = []

    for byte in value:
        encrypted_byte = ""
        for bit_index in range(len(byte)):
            #Perform XOR operation on each bit
            if byte[bit_index] == xor_key[bit_index]:
                encrypted_byte += "0"

            else:
                encrypted_byte += "1"
```

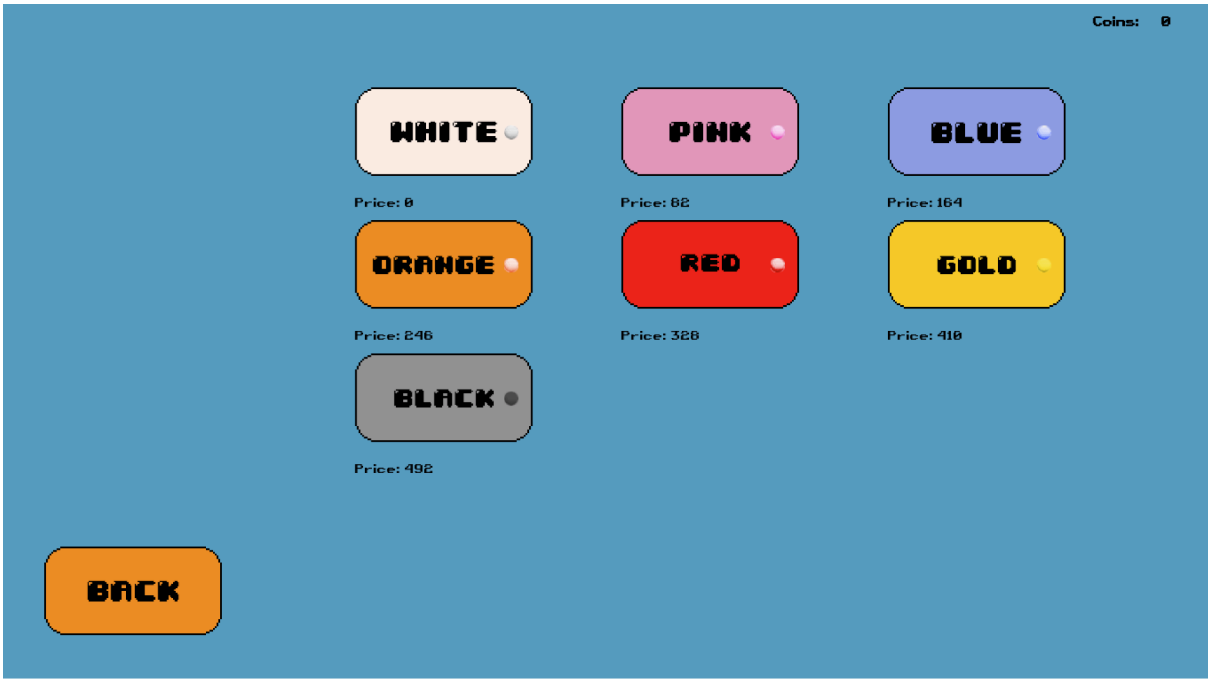
```
        encrypted_values.append(encrypted_byte)
```

Store

A store can be found in many other modern games, it allows players to progress through a game. This game store has a currency that cannot be bought with real money and is only earned through gameplay.

Each player has a wallet that stores coins linked to their accountID. These coins are earned through playing the game. The game rewards 16 coins per level completed. The 16 coins are constant per each level and the only way to get more coins is to play more levels. Caused to make purchases within the store.

The store contains golfball skins, which change the appearance of a golf ball to a different colour or image. A feature commonly found in mobile games such as Voodoos Dune. To unlock other skins the player has to purchase their accumulation of coins. Each skin has an individual price tag that increments up.



If the player has enough coins to purchase a skin, it will deduct the amount of coins the skin is worth from the account's wallet. However, the game has several checks to see if the player already owns the skin and can afford it. Each brought skin will be recorded for that player's account, allowing only people who own the skins to use it. After the player is allowed to select the skin they own. Which will be seen in the game. All this information is stored between the Player_Skins which stores the wallet and Skins which store boolean results if the player owns the skin.





Leaderboard

Leaderboards within games are used to show, compare and as an incentive for replayability because of the challenge to beat a player's PR or challenge other players' scores to come up on top. It is an easy way to keep players coming back, as arcade cabinet games such as Donkey Kong or Pacman.

The Go Golf leaderboard system records players' strokes, scores and time from a game played. The game has an infinite amount of pages to view player stats, that can be cycled

through the buttons a and d. The game will list the player's position, player name, the type of stat and data of the stat. To change what type of data to compare, the player is given three buttons to select from to change it. The game has two individual leaderboards, one for hole 3 and one for hole 5. That displays its own data. This data is created through playing each gamemode. Once played the game saves the results. The game results are set to each player's UserId, which links each User account.

```
        c.execute(f"""SELECT Users.Username,
Holes{self.__leaderboard_hole}.{self.__leaderboard_filter}
        FROM Holes{self.__leaderboard_hole} INNER JOIN Users
        ON  Holes{self.__leaderboard_hole}.UserID == Users.UserID
        ORDER BY Holes{self.__leaderboard_hole}.{self.__leaderboard_filter}
{order}
        """)

        #Player_postions is where all player stats are saves tempoarily
        player_positions = [i for i in c.fetchall()]
        leader_board = []
        conn.close()

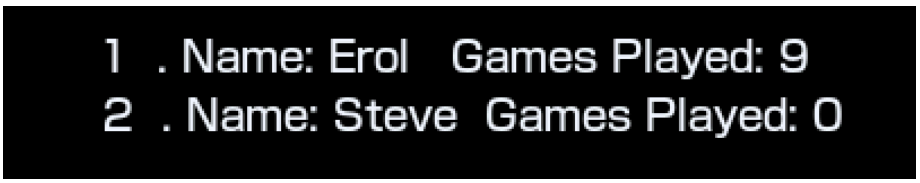
        self.__leaderboard_range_changer()

        #Sets all strings to be drawn for each player
        for i in range(0, len(player_positions)):
                pos = f"{str(i+1)}.{"":3}"
                string_pos_name = f"{player_positions[i][0]:8}"
                if not self.__leaderboard_filter == 'Time':
                string_result = f"{self.__leaderboard_filter} :
{player_positions[i][1]:6}"
                else:
                string_result = f"{self.__leaderboard_filter} :
{str(player_positions[i][1]//60).zfill(2):1}:{str(player_positions[i][1] %
60).zfill(2):1}"


                leader_board.append((pos, string_pos_name, string_result))


        self.__draw_leaderboard(player_positions, leader_board, SCREEN_WIDTH //
2, 125)
```

Issues I've come across with pygame rendering text is being able to equally space text to make perfect columns and rows. The main issue is a gap between fonts and size fonts, issues are placed evenly even when set to be equal, and end up looking awful when rendered. As seen from an example le from the previous example:



To find a workaround for this issue, I separated player position, name of position and the stats. All these in their list, which are cycled through 15 positions within that list and drawn together.

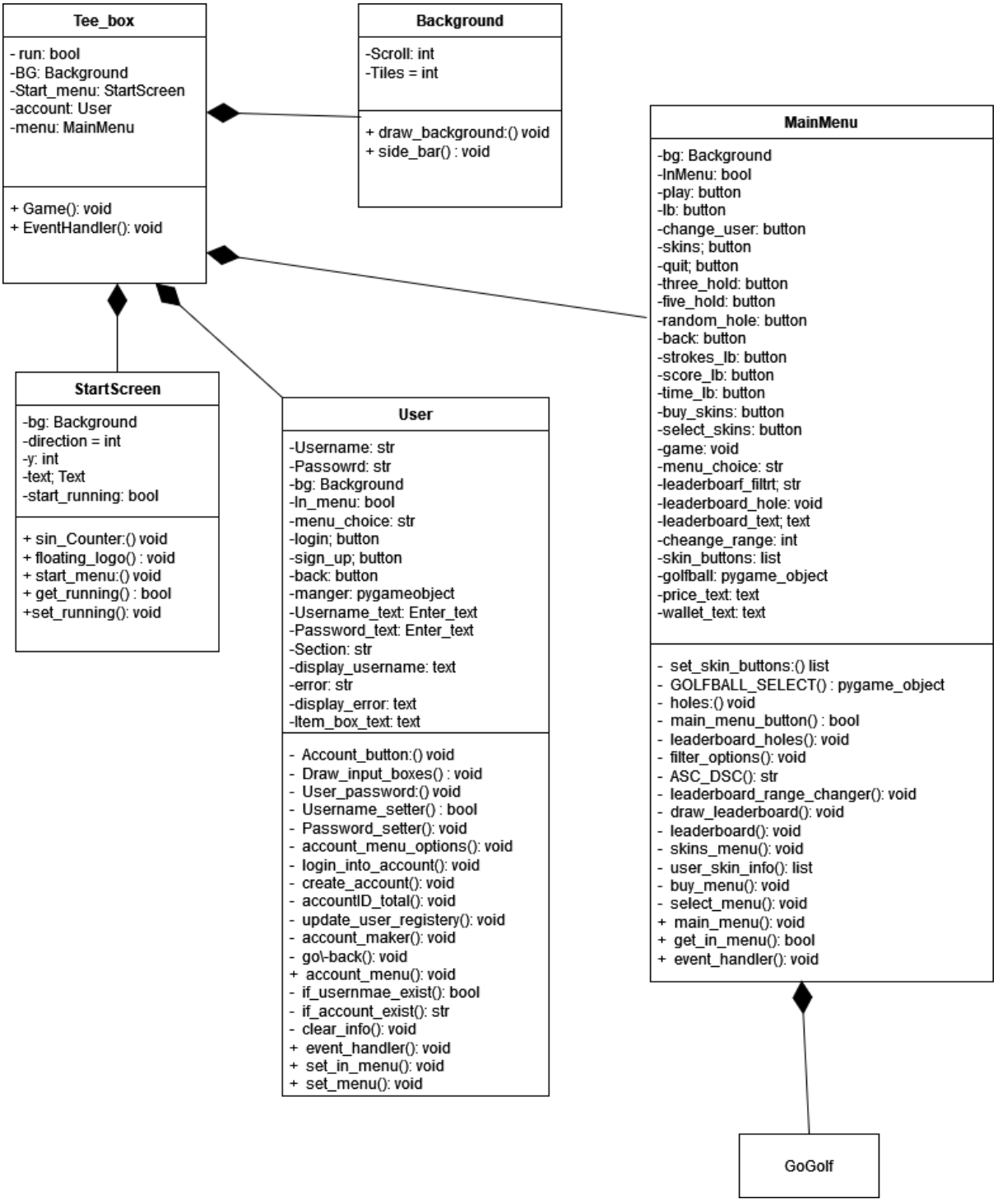
```
        for i in range(-15 + self.__change_range, self.__change_range):

            if i < len(player_positions):
                self.__leaderboard_text.set_all(leader_board[i][0],
BLACK, P2, 20, lb_x_pos, lb_y_pos)
                self.__leaderboard_text.draw_text(screen)
                self.__leaderboard_text.set_all(leader_board[i][1],
BLACK, P2, 20, lb_x_pos + 30, lb_y_pos)
                self.__leaderboard_text.draw_text(screen)
                self.__leaderboard_text.set_all(leader_board[i][2],
BLACK, P2, 20, lb_x_pos + 175, lb_y_pos)
                self.__leaderboard_text.draw_text(screen)
                lb_y_pos += 35
            else:
                break
```

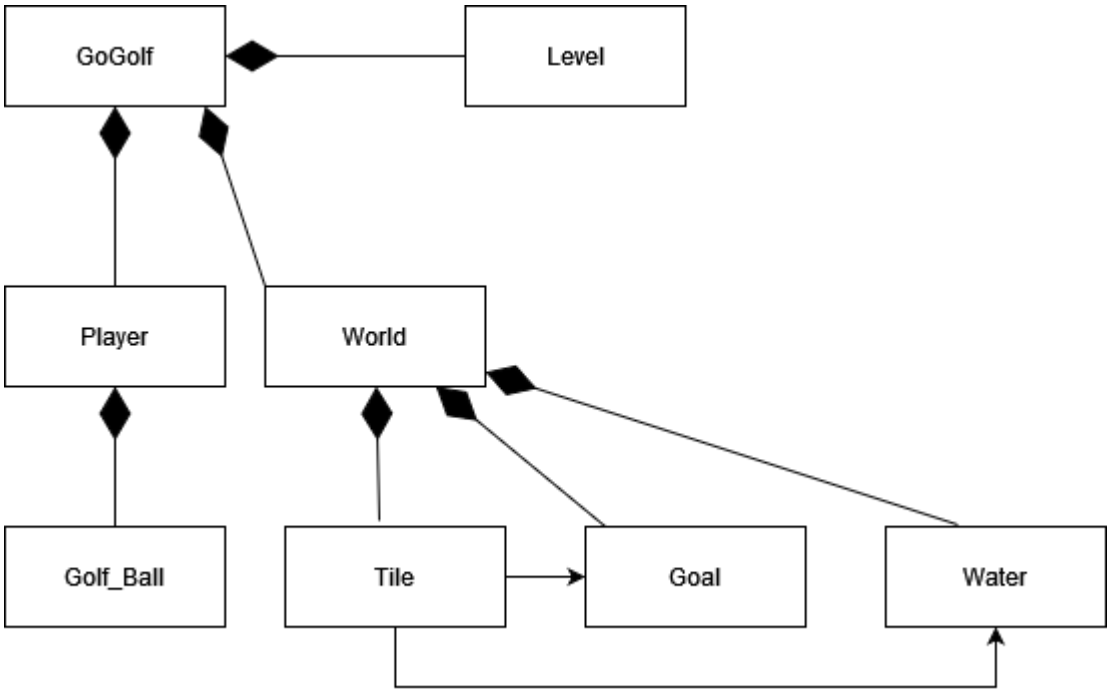


Game objects/UMLs

Menus



Game (simple)



TEST

Type of testing

Game/programs are tested in various methods such as Unit testing, Logging, performance, security and visual. Within the game I used logging, tracking performance, attempting to decrypt passwords and seeing issues arise from the visuals of the game.

Test Plans

Logger

The Logger system within the game uses an external module named Logging. The logging module creates the log file, with any name and has a simple way to designate the importance of logs, write any information to the logs, dates and times and what module/file the code is running in. Loggers can help test physics, collisions, and small errors that may arise from creating a project of this magnitude. The only drawback of this system of testing within this game is the constant amount of loops and updates the game makes per second which can lead to the same data being written 100 times. So some scenarios with this method cannot be checked and were assisted with temporary console outputs to validate data. Logger is used within both the level editor and the main game. Each new log relates to what program it is using and the time plus date of that program.

Editor

Test	Result	Proof
Application Opens	Pass	__main__ - 28-Mar-24 16:10:50 - INFO - Application Started
Game Generates World	Pass	__main__ - 28-Mar-24 16:10:50 - INFO - Blank world generated
Create List Of Image Tiles	Pass	__main__ - 28-Mar-24 16:10:50 - INFO - Create List Of Image Tiles
Setup Tile Buttons	Pass	__main__ - 28-Mar-24 16:10:50 - INFO - Setup Tile Buttons
Speed Multiplier	Pass	__main__ - 28-Mar-24

		16:10:53 - INFO - Speed Multiplied
Level Changer	Pass	__main__ - 28-Mar-24 16:10:56 - INFO - Level changed to 2
Select tiles	Pass	__main__ - 28-Mar-24 16:11:03 - INFO - Selected tile <__main__.Button object at 0x000001CBE2DD1BB0>
Tiles can be placed in grids	Pass	__main__ - 28-Mar-24 16:11:04 - INFO - Tile 3 Added in 17,5 __main__ - 28-Mar-24 16:11:04 - INFO - Tile 3 Added in 16,5
Tiles can be removed	Pass	__main__ - 28-Mar-24 16:11:03 - INFO - Tile removed in 15,6
Game Pauses for saves/loads	pass	__main__ - 28-Mar-24 16:11:27 - INFO - Game Paused __main__ - 28-Mar-24 16:11:28 - INFO - Unconfirmed __main__ - 28-Mar-24 16:11:28 - INFO - Editor Resumed __main__ - 28-Mar-24 16:11:28 - INFO - Game Resumed __main__ - 28-Mar-24 16:11:28 - INFO - Save Level
Game can save levels	Pass	__main__ - 28-Mar-24 16:11:28 - INFO - Save Level
Game can load levels	Pass	__main__ - 28-Mar-24 16:11:28 - INFO - Level 2 has been loaded

Game

Test	Result	Proof
Application opens	Pass	__main__ - 28-Mar-24 16:21:20 - INFO - Application started
Check Database Exist	Pass	
Create New Database	Pass	__main__ - 28-Mar-24 16:21:20 - INFO - Database is already created
Game In start menu	Pass	gogolf_menu - 28-Mar-24 16:21:20 - INFO - Game has open
Load Level	Pass	gogolf_gui - 28-Mar-24 16:25:29 - INFO - Level 1 has been loaded
Loads a queue of 3 Holes	Pass	gogolf_gui - 28-Mar-24 16:25:29 - INFO - Level 1 has been loaded gogolf_gui - 28-Mar-24 16:26:41 - INFO - Level 2 has been loaded gogolf_gui - 28-Mar-24 16:28:22 - INFO - Level 3 has been loaded
Loads a queue of 5 holes	Pass	gogolf_gui - 28-Mar-24 16:30:08 - INFO - Level 1 has been loaded gogolf_gui - 28-Mar-24 16:31:14 - INFO - Level 2 has been loaded gogolf_gui - 28-Mar-24 16:33:13 - INFO - Level 3 has been loaded gogolf_gui - 28-Mar-24 16:34:35 - INFO - Level 4 has been loaded gogolf_gui - 28-Mar-24 16:37:08 - INFO - Level 5 has been loaded

Set golf ball position for level spawn location (two different levels)	Pass	gogolf_gui - 28-Mar-24 16:25:29 - INFO - Level 1 has been loaded gogolf_gui - 28-Mar-24 16:25:31 - INFO - golfball x pos before 1920 gogolf_gui - 28-Mar-24 16:25:31 - INFO - Scroll Before 0 gogolf_gui - 28-Mar-24 16:25:31 - INFO - Distance camera from ball x of camera is 0 gogolf_gui - 28-Mar-24 16:26:41 - INFO - Level 2 has been loaded gogolf_gui - 28-Mar-24 16:26:42 - INFO - golfball x pos before -1191.1232563806552 gogolf_gui - 28-Mar-24 16:26:42 - INFO - Scroll Before 0 gogolf_gui - 28-Mar-24 16:26:42 - INFO - Distance camera from ball x of camera is -6327.123256380655
Scroll should start at 0	Pass	gogolf_gui - 28-Mar-24 16:26:41 - INFO - Level 2 has been loaded gogolf_gui - 28-Mar-24 16:26:42 - INFO - golfball x pos before -1191.1232563806552 gogolf_gui - 28-Mar-24 16:26:42 - INFO - Scroll Before 0
Measuring distance moved (from when ball not moved)	Pass	gogolf_gui - 28-Mar-24 16:26:42 - INFO - Scroll Before 0 gogolf_gui - 28-Mar-24 16:26:42 - INFO - Scroll after -4453.0
Measuring offset of x	Pass	gogolf_gui - 28-Mar-24

		16:26:42 - INFO - Distance camera from ball x of camera is -6327.123256380655 gogolf_gui - 28-Mar-24 16:26:42 - INFO - Offset x of camera is -4453.0 gogolf_gui - 28-Mar-24 16:26:42 - INFO - golfball x pos after -1191.1232563806552 gogolf_gui - 28-Mar-24 16:26:42 - INFO - Scroll after -4453.0 gogolf_gui - 28-Mar-24 16:26:49 - INFO - Player set his aim
Shot speed	Pass	gogolf_gui - 28-Mar-24 16:26:49 - INFO - Mouse positions: (635, 558) gogolf_gui - 28-Mar-24 16:26:49 - INFO - Ball position: [683, 460] Ball Speed: 654.7426975537795 $((\sqrt{635-683)^2+(558-460)^2})1.5)*4 = 654.742$
Shot angle	Pass	gogolf_gui - 28-Mar-24 16:26:49 - INFO - Mouse postions: (635, 558) Ball Angle: 116.09542415738835 gogolf_gui - 28-Mar-24 16:26:49 - INFO - Ball postion: [683, 460] $tan(558-460/645-683)-180 = 116$
Ball Direction based on angle	Pass	gogolf_gui - 28-Mar-24 16:26:49 - INFO - Player set his aim gogolf_gui - 28-Mar-24 16:26:49 - INFO - Mouse postions: (635, 558) gogolf_gui - 28-Mar-24 16:26:49 - INFO - Direction: 1 gogolf_gui - 28-Mar-24

		16:26:49 - INFO - Ball postion: [683, 460]
Ball enter a Hole object	Pass	gogolf_gui - 28-Mar-24 16:27:15 - INFO - Goal hit: True gogolf_gui - 28-Mar-24 16:27:15 - INFO - TILE pos: 608 BAll pos: 606
Calculate a Score	Pass	gogolf_gui - 28-Mar-24 16:31:14 - INFO - starting score: 3000 gogolf_gui - 28-Mar-24 16:31:14 - INFO - divider: 14 and strokes: 6 gogolf_gui - 28-Mar-24 16:31:14 - INFO - Time 72 and score 5830
Set score based on level (world with 3 holes)	Pass	gogolf_gui - 28-Mar-24 16:26:41 - INFO - starting score: 3000 gogolf_gui - 28-Mar-24 16:25:29 - INFO - Level 1 has been loaded
Total Score calculated	Pass	gogolf_gui - 28-Mar-24 16:29:37 - INFO - Total score: 105209 gogolf_gui - 28-Mar-24 16:33:13 - INFO - Total score: 73587 gogolf_gui - 28-Mar-24 16:26:41 - INFO - Total score: 33333

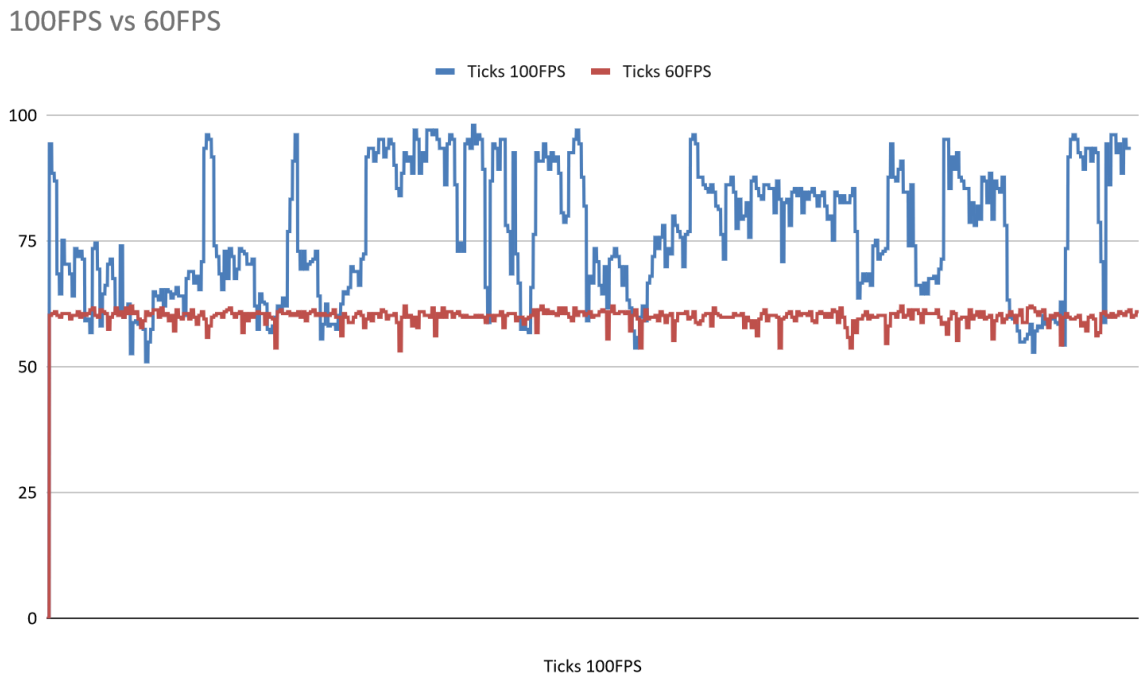
Performance Tracking

Within the game I can track the game tick rate (how fast code cycles) through calculating the game FPS. This is already done through an inbuilt pygame module named pygame.time.Clock().tick(FPS).

This allows me to get a good idea where to add the target frame rate. Originally the game was set to 100, however it led to a fluctuating frame rate which makes the game feel choppy, as the game receives and displays outputs at sharp various rates Using Level 1 as an

example at target 100 FPS vs 60 FPS, with system a 11th Gen Intel(R) Core(TM) i7-1185G7 @ 3.00GHz, 16.0 GB memory with speed of 4267 MHz and with OS Windows 11. Each FPS is saved as a list attribute and then to excel using pandas and openpyxl.

Example FPS on Level 1:



Playing the level with a max frame rate of 100, the game experiences huge dips from wide ranges of frame rates, which can create very choppy gameplay during more demanding sections of the game. When comparing it to 60 FPS, the game framerate never dips more than 7 frames, the lowest being 53.2 and the average being 59.97636002 near the target framerate. Compared with 100 FPS it never reaches its max frame rate, only reaching 98.03921509, and at its lowest being 51.02040863 creating a difference of 47 frames. At 100FPS the game average is 22 frames away from the target at 76.68151941.

Known Issues

- Collisions in y axis can sometime reverse the ball position backwards
- Collisions from a fast velocity and high height into golf hole are not as consistent
- Inactive holes can sometimes still allow the ball to fall into the hole.
- Ball flying at high velocity can officially clip into a tile, making them stuck within the tile temporarily.
- The more tiles rendering on the screen dramatically add performance onto the game
- Players can hack and alter their wallet, username, userID, account and encrypted because of due to a local database
- Multiple or Reseting text objects can stump frame rates dramatically, as seen in the leaderboard menu as the background becomes more jittery and slow.
- Some names such as race affect the placement of text in leaderboard menu

EVALUATION

Objectictives Completed

Objectives	**Implemented**
To be able to use OOP to create appropriate objects	Yes
To be able to successfully allow objects to interact with each other through “is a” and “has a” relationship	Yes
To write clear and easy-to-read code using small methods/functions with methods to explain what the code does	Yes
To get the most performance out of the language, due to Python being an interpreter	Yes
Create a well-designed database using normalisation techniques, such as 2NF and 3NF	Yes
To reuse code when necessary through the creation of libraries with different objects and functions	Yes
To be able to implement necessary abstract data structures such as queues or stacks when necessary	Yes
To have a functional account creator/sign-in, with passwords that are encrypted and stored in a database	Yes
To have accounts that can record and store their scores, coins and skins	Yes
To have a ball able to interact with the environment by slowing it down the ball or changing its path	Yes
To have at least five playable levels with all their twist and turns	Yes
To create an algorithm that can reward players who complete the level as fast as possible with the fewest strokes	Yes

To have a clean and easy-to-navigate menu system to help users/players explore all game mechanics	Yes
To have a camera that can follow the ball after a player makes a shot	Yes
To have a camera that the player can use to discover the level before they take the next shot	Yes
To create a store where players can buy skins for their ball, altering its appearance and permanently stored only for that player	Yes
To have easy to navigate leaderboard that shows all players' scores, strokes and times for each gamemode, descending from best to worst	Yes
To implement a level creator that can create a level, alter and save levels through a tile system saved as a 2D array	Yes
To create a level loader to load all levels from the level creator	Yes
To reduce redundancy within created databases	Yes
To create and find copyright assets to implement within the game	Yes

I reached my primary goal in creating a replayable game, with easy navigate menus and systems. I challenged myself to create a game type that I never made before. Having scrollable levels, small amounts of physics and a level editor. Further, I achieved my secondary objectives by having physics-based gameplay, that may not be perfect but is suitable for a game of this scale. They achieved having a ball that interacts with the world through x and y directions that work in parallel together to create an arcade ball trajectory. I think I created a game that was coded well, and was presented well, only probably needing a tiny bit more polish and maybe compiler-based programming language.

System Requirements

Please ensure that your computer uses the following or exceeds these system requirements before installing GoGolf

CPU	11th Gen Intel Core i7-1185G7
RAM	2GB (4 GB recommend)
Disk space	300 MB
Operating System	Windows 10, Windows 11, MacOS
Language	Python 3.12.1

Improvements for the Future

Some improvements for the game if I took it further would way to host an online database that syncs all players results and accounts to one location, with none of the information being stored locally and can only be accessed if connected if the device being used is on the internet, a more detailed and consistent collision model would be useful to implement more odd shaped tiles, such as a triangle. Another improvement is using a language that uses a compiler, as languages such as Python and Java byte code have to be translated during runtime using interpreters. This ends up slowing code down. A language such as C++ or Rust would be more useful. Furthermore, the game heavily relies on inbuilt methods and objects from pygame, which creates a lot of bloat. It could be a better idea to make more of the game from scratch and maybe let language like C do a lot of the more demanding tasks. A few parts of the game could do with some better names for methods and objects. The game could that input from a controller. The game could have PvP competitive multiplayer. The level editor could inbuilt a game allowing modded maps, but still separated from the official ones through check digit. Password encryption keys are not accessible to anyone with Python and IDE. The game could do with more themes and more options from the store.

Conclusion

GoGolf is another project idea that I created without knowing the basics for a game of this type. Overall, the gameplay is passable, it doesn't hold players' attention as much as some of my other projects, like UNO. This is due to games created on great ideas already being just as fun when remade anywhere. I believe I did a decent job developing the theme, menus, database, and other elements. Since I didn't see the value in adding sound effects once the core game was finished, I didn't include them in the game as I intended. The hardest part of the project was just figuring out how to use a golf ball to interact and move in the world, as before this I've never needed to add collisions to a game nor have I ever made a side scroller game before. Big fault of the game is performance. Even when I added small methods such as only rendering tiles on the screen, pygame struggled to render the mass qualities of tiles

at the same time. Pygame struggles at utilising most of the CPU which can make a considerable difference in the performance of the game. As I experienced when using AMD Ryzen 7 5700x playing the game it instantly made the game perform better, but that should be beyond overkill for such a small game. Overall, I was successful in creating my project exactly as I planned, with maybe a few minor things I could learn from for future projects.

APPENDIX

References

- GitHub. (2020). pygame/pygame. [online] Available at: https://github.com/pygame/pygame.

- Fanatec. (n.d.). CSL DD (8 Nm). [online] Available at: https://fanatec.com/eu-en/racing-wheels-direct-drive-bases/racing-wheels/csl-dd-8-nm [Accessed 6 Mar. 2024].
- steamcharts.com. (n.d.). Football Manager 2024 - Steam Charts. [online] Available at: https://steamcharts.com/app/2252570#3m [Accessed 6 Mar. 2024].
- sport.optus.com.au. (n.d.). Optus Sport. [online] Available at: https://sport.optus.com.au/news/ligue-1/os53708/ligue-1-news-2023-will-still-reims-football-manager-game-psg.
- steamcharts.com. (n.d.). EA SPORTS FCTM 24 - Steam Charts. [online] Available at: https://steamcharts.com/app/2195250.
- Google.com. (2024). Available at: https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&ved=2ahUKEwiqnNLR8d-EAxVrX0EAHQBBBxgQFnoECBwQAw&url=https%3A%2F%2Fwww.sportskeeda.com%2Fesports%2Fhow-long-game-fifa-23-1&usg=AOvVaw1idgrl0SxaYYk_NTNtO0br&opi=89978449 [Accessed 6 Mar. 2024].
- www.esportsearnings.com. (n.d.). Rocket League Top Players & Prize Pools - Esports Tracker :: Esports Earnings. [online] Available at: https://www.esportsearnings.com/games/409-rocket-league [Accessed 6 Mar. 2024].
- RealSport101. (2023). EA Sports PGA Tour Roster: Discover the best rated players in the game. [online] Available at: https://realsport101.com/golf/ea-sports-pga-tour-roster/ [Accessed 7 Mar. 2024].
- Sirani, J. (2022). The 10 Best-Selling Video Games of All Time. [online] IGN. Available at: https://www.ign.com/articles/best-selling-video-games-of-all-time-grand-theft-auto-minecraft-tetris.
- Rscar_ (2023). Coffee Golf released this week! [online] Available at: https://www.reddit.com/r/iosgaming/comments/145x2nl/coffee_golf_released_this_week/ [Accessed 7 Mar. 2024].
- Hazelcast (n.d.). What is serialization and how does it work? [online] Hazelcast. Available at: https://hazelcast.com/glossary/serialization/.

Bibliography

- GitHub. (2020). pygame/pygame. [online] Available at: https://github.com/pygame/pygame.

- Fanatec. (n.d.). CSL DD (8 Nm). [online] Available at: https://fanatec.com/eu-en/racing-wheels-direct-drive-bases/racing-wheels/csl-dd-8-nm [Accessed 6 Mar. 2024].
- iRacing.com. (n.d.). What Comes With An iRacing Membership? [online] Available at: https://www.iracing.com/what-comes-with-membership/.
- Esports driver set for Formula 2. (n.d.). BBC Sport. [online] Available at: https://www.bbc.co.uk/sport/motorsport/59973433.
- steamcharts.com. (n.d.). Football Manager 2024 - Steam Charts. [online] Available at: https://steamcharts.com/app/2252570#3m [Accessed 6 Mar. 2024].
- sport.optus.com.au. (n.d.). Optus Sport. [online] Available at: https://sport.optus.com.au/news/ligue-1/os53708/ligue-1-news-2023-will-still-reims-football-manager-game-psg.
- steamcharts.com. (n.d.). EA SPORTS FCTM 24 - Steam Charts. [online] Available at: https://steamcharts.com/app/2195250.
- Google.com. (2024). Available at: https://www.google.com/url?sa=t&rct=j&q=&esrc=s&source=web&cd=&ved=2ahUKEwiqnNLR8d-EAxVrX0EAHQBBBxgQFnoECBwQAw&url=https%3A%2F%2Fwww.sportskeeda.com%2Fesports%2Fhow-long-game-fifa-23-1&usg=AOvVaw1idgrl0SxaYYk_NTNtO0br&opi=89978449 [Accessed 6 Mar. 2024].
- www.esportsearnings.com. (n.d.). Rocket League Top Players & Prize Pools - Esports Tracker :: Esports Earnings. [online] Available at: https://www.esportsearnings.com/games/409-rocket-league [Accessed 6 Mar. 2024].
- RealSport101. (2023). EA Sports PGA Tour Roster: Discover the best rated players in the game. [online] Available at: https://realsport101.com/golf/ea-sports-pga-tour-roster/ [Accessed 7 Mar. 2024].
- www.youtube.com. (n.d.). Golf - Nintendo Famicom Disk System - VGDB. [online] Available at: https://www.youtube.com/watch?v=ga0xdbyS1Tg&ab_channel=VGDBPlay [Accessed 7 Mar. 2024].
- Sirani, J. (2022). The 10 Best-Selling Video Games of All Time. [online] IGN. Available at: https://www.ign.com/articles/best-selling-video-games-of-all-time-grand-theft-auto-minecraft-tetris.
- Rscar_ (2023). Coffee Golf released this week! [online] Available at: https://www.reddit.com/r/iosgaming/comments/145x2nl/coffee_golf_released_this_week/ [Accessed 7 Mar. 2024].
- www.datacamp.com. (n.d.). Python Pickle Tutorial with plk. [online] Available at: https://www.datacamp.com/tutorial/pickle-python-tutorial.
- Hazelcast (n.d.). What is serialization and how does it work? [online] Hazelcast. Available at: https://hazelcast.com/glossary/serialization/.

Source Code

GoGolf.py

```
import gogolf_menu
from pygame.locals import *
from Logger import *
import os
import sqlite3


#Checks if all that needed on application side to start the application
starting_check("GoGolf")

#Get name of current file being used
LOGGER = logging.getLogger(f"{__name__}")

#Starting log (info can be swaped for different use cases)
LOGGER.info(f'Application started')

LOGGER.info('Application started')

def create_register_table(conn):
      c = conn.cursor()
      c.execute("""CREATE TABLE Users (
      UserID INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL,
      AccountID INTEGER UNIQUE NOT NULL,
      Username VARCHAR(8) NOT NULL UNIQUE,
      Password VARCHAR(8) NOT NULL
      )""")
      conn.commit()


def create_holes_table(conn):
      c = conn.cursor()
      for table in ["Holes3", "Holes5"]:
      c.execute(f"""CREATE TABLE {table} (
            UserID INTEGER NOT NULL,
            Score INT,
            Strokes INT,
            Time INT,
```

```
            Played DATE DEFAULT CURRENT_DATE,
            FOREIGN KEY(UserID) REFERENCES Users(UserID)
      )""")
      conn.commit()

def create_skins_table(conn):
      c = conn.cursor()

      # Create SKINS table
      c.execute("""CREATE TABLE Skins (
      AccountID INTEGER NOT NULL,
      White BOOL DEFAULT 1,
      Pink BOOL DEFAULT 0,
      Blue BOOL DEFAULT 0,
      Orange BOOL DEFAULT 0,
      Red BOOL DEFAULT 0,
      Gold BOOL DEFAULT 0,
      Black BOOL DEFAULT 0,
      FOREIGN KEY(AccountID) REFERENCES Users(AccountID)
      )""")


      # Create PLAYER_SKINS table
      c.execute("""CREATE TABLE Player_Skins (
      AccountID INTEGER NOT NULL,
      Coins INT DEFAULT 0,
      FOREIGN KEY(AccountID) REFERENCES Users(AccountID)
      )""")
      conn.commit()



def create_database():
      LOGGER.info("Creating database")
      conn = sqlite3.connect("database/GoGolfDB.db")
      c = conn.cursor()
      create_register_table(conn)
      create_holes_table(conn)
      create_skins_table(conn)
      conn.commit()
      conn.close()
```

```
def check_database():
      if os.path.exists("database/GoGolfDB.db"):
      LOGGER.info(f"Database is already created")
      else:
      LOGGER.warning("Database does not exist")
      create_database()

if __name__ == "__main__":
      LOGGER.info('Hello')
      check_database()
      gogolf_menu.menu()
```

Gogolf_lilbray.py

```
import pygame
import os
import pygame_textinput
current_directory = os.getcwd()

class Text:
      def __init__(self):
      """Creates and display text with differnt fonts and sizes"""
      self.__text = ""
      self.__colour = ""
      self.__font = None
      self.__x = -1
      self.__y = -1

      def set_text(self, value):
      self.__text = value

      def set_colour(self, value):
      self.__colour = value

      def set_font(self, value, size):
      self.__font = pygame.freetype.Font(value, size)

      def set_x(self, value):
      self.__x = value

      def set_y(self, value):
      self.__y = value

      def set_font(self, font, font_size):
```

```
        self.__font =
pygame.freetype.Font(os.path.join(current_directory,font), font_size)

        def set_all(self , text, colour, font, font_size, x, y):
        self.set_colour(colour)
        self.set_text(text)
        self.set_font(font, font_size)
        self.set_x(x)
        self.set_y(y)

        def draw_text(self, screen):
        text_surface, rect = self.__font.render(self.__text,
pygame.Color(self.__colour))
        screen.blit(text_surface, (self.__x, self.__y))



class Button():
        """Creates Buttons that change image when the user mouse postion
is over the button"""
        def __init__(self,x,y,image,scale):
        width = image.get_width()
        height = image.get_height()
        self.image =
pygame.transform.scale(image,(int(width*scale),int(height*scale)))
        self.image_hi = self.brighten_image()
        self.rect = self.image.get_rect()
        self.rect.topleft = (x,y)
        self.clicked = False
        self.hold = False

        def brighten_image(self):
        """Brightens Image of the button """

        image_copy = self.image.copy()
        brightness_factor = 1.5

        #Iterate through pixels
        for x in range(image_copy.get_width()):
                for y in range(image_copy.get_height()):
                        #Get the color of the pixel
                        colour = image_copy.get_at((x, y))

                        #Brighten RGB channels while leaving alpha channel
unchanged
```

```
                    new_colour = (
                    min(int(colour.r * brightness_factor), 255),
                    min(int(colour.g * brightness_factor), 255),
                    min(int(colour.b * brightness_factor), 255),
                    colour.a
                    )

                    image_copy.set_at((x, y), new_colour)

        return image_copy


        def draw(self, screen):
        """Returns action and changes self.clicked if the user press the
mouse ontop of the image"""
        action = False
        pos = pygame.mouse.get_pos()

        if self.rect.collidepoint(pos):
              screen.blit(self.image_hi, (self.rect.x,self.rect.y))
              if pygame.mouse.get_pressed()[0] == 1 and self.clicked ==
False:
                    self.hold = True

        if pygame.mouse.get_pressed()[0] == 0 and self.hold:
              self.clicked = True
              action = True
              self.hold = False

        if pygame.mouse.get_pressed()[0] == 0:
              self.clicked = False

        if self.rect.collidepoint(pos):
              screen.blit(self.image_hi, (self.rect.x,self.rect.y))

        else:
              screen.blit(self.image, (self.rect.x,self.rect.y))

        return action


class Enter_Text:
        """Text that the user can type into and can be saved"""
        def __init__(self,x,y,font,font_size,anti,text_manger):
        self.font = pygame.font.SysFont(font,font_size)
        self.anti = anti
```

```
        self.cords = x, y
        self.textinput =
pygame_textinput.TextInputVisualizer(manager=text_manger,font_object=sel
f.font)
        self.textinput.antialias = self.anti
        self.name = ""

        def draw(self,events,screen,rect_x,rect_y,rect_w,rect_h):
        """Draws a box for the user to type in and to contrast with the
background"""
        self.textinput.update(events)
        input_rect = pygame.Rect(rect_x,rect_y,rect_w,rect_h)
        colour = pygame.Color("#F5FEFD")
        pygame.draw.rect(screen,colour ,input_rect)
        screen.blit(self.textinput.surface, (self.cords))

        def enter(self,events):
        """If Enter button is clicked it would save the text input"""
        if [ev for ev in events if ev.type == pygame.KEYDOWN and ev.key ==
pygame.K_RETURN]:
              self.name = str(self.textinput.value)


        def access(self):
        """Returns the user username"""
        return self.name

        def clear_text(self):
        """Clears the Current text"""
        self.name = ""
        self.textinput.value = ""


class queue:
  """Queue data structure"""
  def __init__(self, capacity):
        self.__capacity = capacity
        self.__array = [None]*capacity
        self.__fpointer = 0
        self.__rpointer = 0

  def enqueue(self,item):
        if self.__rpointer == self.__capacity:
        if self.__fpointer == 0:
        print("Queue is full")
        return
```

```
        else:
        for i in range(self.__fpointer,self.__rpointer):
              self.__array[i - self.__fpointer] = self.__array[i]
        self.__rpointer -= self.__fpointer
        self.__fpointer = 0
        self.__array[self.__rpointer] = item
        self.__rpointer += 1

    def dequeue(self):
        if self.__fpointer == self.__rpointer:
        print("Empty Queue")
        else:
        self.__fpointer += 1
        return self.__array[self.__fpointer - 1]



def merge_sort(list1):
    if len(list1) == 1:
        return list1
    left_list = list1[:len(list1)//2]
    right_list = list1[len(list1)//2:]
    left_list = merge_sort(left_list)
    right_list = merge_sort(right_list)
    results = []
    while len(left_list) > 0 and len(right_list) > 0:
        if left_list[0] < right_list[0]:
        results.append(left_list.pop(0))
        else:
        results.append(right_list.pop(0))
    results.extend(left_list)
    results.extend(right_list)
    return results



class stack:
    def __init__(self,capactiy):
        self.__capacity = capactiy
        self.__array = [None]*capactiy
        self.__pointer = 0


    def push(self,item):
        if self.__pointer < self.__capacity:
        self.__array[self.__pointer] = item
```

```
        self.__pointer += 1
        else:
        print('stack overflow')

    def pop(self):
        if self.__pointer == 0:
        print('stack undeflow')
        else:
        self.__pointer -= 1
        return self.__array[self.__pointer]


    #Create a Superencipherment cipher (cipher that encrypted twice)
    #Change letters to ASCII to binary, apply xor, convert back to denary,
ceaser cipher of one of
    #5 keys (either 5, 28, 26, 66, 92)


def convert_binary(value, length):
        binary_password = ["" for i in range(length)]

        #changes it to binary
        for i in range(length):
        while value[i]:
                if value[i] % 2 == 0:
                        binary_password[i] = "0" + binary_password[i]
                else:
                        binary_password[i] = "1" + binary_password[i]
                value[i] = value[i] //2

        return binary_password


def xor_cipher(value, xor_key):
        encrypted_values = []

        for byte in value:
        encrypted_byte = ""
        for bit_index in range(len(byte)):
                # Perform XOR operation on each bit
                if byte[bit_index] == xor_key[bit_index]:
                        encrypted_byte += "0"

                else:
                        encrypted_byte += "1"
        encrypted_values.append(encrypted_byte)
```

```
	return encrypted_values


def binary_denary_converter(value):
	ascii_encrypted = []

	for byte in value:
	ascii_value = 0
	# Start with the highest power of 2
	power = len(byte) - 1
	for bit in byte:
		# For each bit in the byte, calculate its decimal value and
add it to the total
		if bit == "1":
			ascii_value += 2 ** power
		power -= 1  # Move to the next lower power of 2
	ascii_encrypted.append(ascii_value)

	return ascii_encrypted


def ceaser_cipher(string, key):
	second_level = []

	for value in string:
	decrypted_value = value - key
	if decrypted_value <= 0:
		temp = 1110000 + decrypted_value
	else:
		temp = decrypted_value
	second_level.append(chr(temp))
	return second_level


def xor_cipher(value, xor_key):
	encrypted_values = []

	for byte in value:
	encrypted_byte = ""
	for bit_index in range(len(byte)):
		#Perform XOR operation on each bit
		if byte[bit_index] == xor_key[bit_index]:
			encrypted_byte += "0"
```

```
            else:
                  encrypted_byte += "1"
      encrypted_values.append(encrypted_byte)

      return encrypted_values

def xor_cipher_DE(value, xor_key):
      values = []
      for key in range(len(value)):
      values.append("".join(xor_cipher(value[key], xor_key[key])))

      return values



def encrypter(password):
      """Encrypts data using xor key and ceaser cipher (any mention of
ascii is
      incorrect, as values are unicode)."""

      ascii_password = [ord(letter) for letter in password] #Changes it
to ascii

      binary_password = convert_binary(ascii_password,
len(ascii_password))

      xor_key = "10101011" #AB

      #Applys xor key to all binary values
      first_level =  xor_cipher(binary_password, xor_key)

      #converts binary to denary
      ascii_encrypted = binary_denary_converter(first_level)


      key = -5 #Ceaser cipher key

      #Applies Ceaser Cipher to xor encrypted ascii values
      second_level = ceaser_cipher(ascii_encrypted, key)

      #Turns it back into a sting
      second_level = "".join(second_level)

      return second_level
```

```
#encrypted password applied with xor from a plaintext that converted to
binary of user
#typed password

#so only need xor version of password with xor with non xor version.

# def dencrypter(en_password, password):
#     #decrypt
#     xor_key_setter = "10101011"

#     password = [ord(letter) for letter in password]
#     password = convert_binary(password, len(password))
#     xor_key = xor_cipher(password, xor_key_setter)


#     print(xor_key)

#     key = 5
#     print(f"\nDecrypt: ")
#     print(f"Encrypted password: {en_password}")
#     print(f"denary {[ord(i) for i in en_password]}")

#     second_level = ceaser_cipher([ord(i) for i in en_password],
abs(key))

#     print(f"after ceaser cipher {second_level}")

#     password_denary = [ord(i) for i in second_level]
#     print(f"converted to denary {password_denary}")

#     password_bianry = convert_binary(password_denary,
len(password_denary))
#     print(f"convert to binary before xor: {password_bianry}")


#     password_bianry = xor_cipher_DE(password,  xor_key)
#     print(f"After xor {password_bianry}")
#     password_denary = binary_denary_converter(password_bianry)
#     print(f"After converted to denary {password_denary}")
#     print(f"After converted to string list {[chr(i) for i in
password_denary]}")
#     print(f"With join : {"".join([chr(i) for i in password_denary])}")
```

```
#     return "".join([chr(i) for i in password_denary])



# encrypter("steve")
# dencrypter("+&5(5", "mike")
```

Gogolf_menu.py

```
import pygame
import logging
from math import ceil, sin
import gogolf_gui
import gogolf_lilbray
import sys
import pygame_textinput
import sqlite3

pygame.init()


LOGGER = logging.getLogger(f"{__name__}")
#FPS
clock = pygame.time.Clock()
FPS = 100

icon = pygame.image.load('Assests/logo1.ico')
pygame.display.set_icon(icon)

#Image Widiths and Heights
background_1 = pygame.image.load('Background_Img/1.png').convert_alpha()
background_2 = pygame.image.load('Background_Img/2.png').convert_alpha()
background_3 = pygame.image.load('Background_Img/4.png').convert_alpha()
leaderboard_wall = pygame.image.load('Assests/leaderboard_wall.png')

logo = pygame.image.load('Assests/logo1.png').convert_alpha()

play_button = pygame.image.load('Assests/play.png')
lb_buttons = pygame.image.load('Assests/leaderboard.png')
```

```
quit_buttons = pygame.image.load('Assests/quit.png')
back_buttons = pygame.image.load('Assests/back.png')
three_hole_button = pygame.image.load('Assests/three_hole.png')
five_hole_button = pygame.image.load('Assests/five_hole.png')
score_button = pygame.image.load('Assests/score.png')
time_button = pygame.image.load('Assests/time.png')
strokes_button = pygame.image.load('Assests/strokes.png')
random_button = pygame.image.load('Assests/random.png')
change_user_button =  pygame.image.load('Assests/change_user.png')
skins_button =  pygame.image.load('Assests/skins.png')
buy_skins_button = pygame.image.load('Assests/buy_skins.png')
select_skins_button = pygame.image.load('Assests/select_skin.png')

golf_names = ["white","pink","blue","orange","red","gold","black"]

golf_skins = [pygame.image.load(f'Assests/{skin}.png') for skin  in
golf_names]

sign_up_button = pygame.image.load('Assests/sign_up.png')
login_button = pygame.image.load('Assests/login.png')

#Colours
GREY = (245, 244, 243)
BLUE = "#2596be"
WHITE = "#F5FEFD"
ORANGE = "#ff8500"
BLACK = "#000000"
RED = "#D21404"

#Backgorund widiths
bg_w = background_1.get_width()
bg_h = background_1.get_height()

#Fonts
WII_FONT = "Assests/Fonts/AOTFShinGoProMedium.otf"
PIXEL_FONT = "Assests/Fonts/04B_30__.TTF"
P2 = "Assests/Fonts/Retro Gaming.ttf"

#Display Widths and Heights
SCREEN_HEIGHT = 768
SCREEN_WIDTH = 1366
screen = pygame.display.set_mode((SCREEN_WIDTH,SCREEN_HEIGHT))



class Tee_Box():
```

```
        """Creates object for the game to start """

        def __init__(self):
        self.__run = True
        self.__BG = Background()
        self.__start_menu = StartScreen(self.__BG)
        self.__account = User(self.__BG)
        self.__menu = MainMenu(self.__BG)

        def Game(self):
        """Main Handler of the the program, where the start menu held
        and main menu"""

        LOGGER.info("Game has open")
        while self.__run:

                #Start Menu
                if not self.__start_menu.get_running():
                        self.__start_menu.start_menu()

                #Runs account menu and then the main menu
                if self.__start_menu.get_running():
                        self.__menu.main_menu(self.__start_menu,
self.__account.account_menu())
                        if not self.__menu.get_in_menu():
                        #Resets all menus
                        self.__start_menu.set_running(False)
                        self.__account.set_in_menu(True)
                        self.__account.set_menu("")
                self.EventHandler()


        def EventHandler(self):
        """Event Handler """

        events = pygame.event.get()
        for event in events:
                if event.type == pygame.QUIT:
                        LOGGER.info("Quit Application")
                        sys.exit()

        pygame.display.update()
        clock.tick(FPS)
```

```
class Background():
      def __init__(self):
      self.__scroll = 0
      self.__tiles = ceil(SCREEN_WIDTH /  bg_w) + 1

      def draw_background(self):
      """Draw menu background """

      self.__scroll -= 4
      for i in range(0, self.__tiles):
            screen.blit(background_1, (i * bg_w + self.__scroll, 0))
            screen.blit(background_2, (i * bg_w + self.__scroll, 0))
            screen.blit(background_3, (i * bg_w + self.__scroll, 0))

      if abs(self.__scroll) > bg_w:
                  self.__scroll = 0


      def side_bar(self):
      """Draw the game side bar for menu selection """
      pygame.draw.rect(screen, BLUE, (0, 0, 300, SCREEN_HEIGHT))


class StartScreen():
      def __init__(self , bg):
      self.__bg = bg
      self.__direction = 1
      self.__y = 1
      self.__text = gogolf_lilbray.Text()
      self.__start_running = False


      def sin_counter(self):
      """Changes the y coords by sin"""

      if abs(self.__y) % 90 == 0:
            self.__direction = -1
      self.__y +=0.05*self.__direction

      def floatiing_logo(self, x, y):
      """Draws Game Logo oscillating"""

      self.sin_counter()
      pos = (y) + (sin(self.__y)*10)
      screen.blit(logo, (x, pos))
      self.__text.set_all("Press Enter To Start", ORANGE, P2, 40,
```

```
SCREEN_WIDTH//2 - 260, pos + 400)


      def start_menu(self):
      """Game Start screen"""

      keys =  pygame.key.get_pressed()
      self.__bg.draw_background()
      self.floatiing_logo(SCREEN_WIDTH//2- 300, (SCREEN_HEIGHT//2 -
280))
      self.__text.set_all("Press Enter To Start", ORANGE, P2, 40,
SCREEN_WIDTH//2 - 260, (sin(self.__y)*10) + 450)
      self.__text.draw_text(screen)

      if keys[pygame.K_RETURN] or pygame.mouse.get_pressed()[0]:
            self.__start_running =  True


      def get_running(self):
      return self.__start_running

      def set_running(self, value):
      self.__start_running = value




class User():
      def __init__(self, bg):
      self.__username = ""
      self.__password = ""
      self.__bg = bg
      self.__InMenu = True
      self.__menu_choice = ""
      self.__login = gogolf_lilbray.Button(SCREEN_HEIGHT//2  ,
SCREEN_WIDTH//2 - 350, login_button, 0.5)
      self.__sign_up = gogolf_lilbray.Button(SCREEN_HEIGHT//2 + 400,
SCREEN_WIDTH//2- 350, sign_up_button, 0.5)
      self.__back = gogolf_lilbray.Button(50, SCREEN_HEIGHT-150,
back_buttons, 0.4)
      self.__manager = pygame_textinput.TextInputManager(validator =
lambda input: len(input) <= 8)
      self.__username_text = gogolf_lilbray.Enter_Text(SCREEN_WIDTH//2-
45, SCREEN_HEIGHT//2- 45, "cambria", 22, True, self.__manager)
      self.__password_text = gogolf_lilbray.Enter_Text(SCREEN_WIDTH//2-
45, SCREEN_HEIGHT//2, "cambria", 22, True, self.__manager)
```

```
        self.__section = "user"
        self.__display_username = gogolf_lilbray.Text()
        self.__error = ""
        self.__display_error = gogolf_lilbray.Text()
        self.__item_box_text = gogolf_lilbray.Text()

        def __account_buttons(self, events):
        """Draws the game Login and Sign up buttons and return an input
        to menu choice varibale """

        if self.__login.draw(screen):
                self.__username_text.clear_text()
                self.__password_text.clear_text()
                self.__menu_choice = "login"

        if self.__sign_up.draw(screen):
                self.__username_text.clear_text()
                self.__password_text.clear_text()
                self.__menu_choice = "sign_up"


        def __draw_input_boxes(self):
        """Draws input boxes for user type his username and password"""

        pygame.draw.rect(screen, WHITE, (SCREEN_WIDTH//2- 50,
SCREEN_HEIGHT//2 - 50, 300, 40))
        pygame.draw.rect(screen, WHITE, (SCREEN_WIDTH//2- 50,
SCREEN_HEIGHT//2 - 5, 300, 40))
        self.__item_box_text.set_all("Username:", BLACK, P2, 20,
SCREEN_WIDTH//2 - 190, SCREEN_HEIGHT//2 - 40)
        self.__item_box_text.draw_text(screen)
        self.__item_box_text.set_all("Password:", BLACK, P2, 20,
SCREEN_WIDTH//2- 190, SCREEN_HEIGHT//2 + 5)
        self.__item_box_text.draw_text(screen)



        def __user_password(self, events):
        """Sets username and passwords inputs"""

        self.__draw_input_boxes()
        if self.__section == "user":
                self.__username_setter(events)

        elif self.__section == "pass":
                self.__password_setter(events)
```

```
      #Create a Superencipherment cipher (cipher that encrypted twice)
      #Change letters to ASCII to binary, apply xor, convert back to
denary, ceaser cipher of one of
      #5 keys (either 5, 28, 26, 66, 92)


      def __username_setter(self, events):
      """Sets username"""

      self.__display_username.set_text("")
      self.__username_text.draw(events, screen, SCREEN_WIDTH//2- 50,
SCREEN_HEIGHT//2 - 50, 300, 40)

      self.__username_text.enter(events)

      #If Username not blank
      if not self.__username_text.access() == "":
            self.__username = self.__username_text.access().lower()
            self.__section = "pass"
            self.__username_text.clear_text()
            self.__display_username.set_all(self.__username, BLACK, P2 ,
22, SCREEN_WIDTH//2- 45, SCREEN_HEIGHT//2- 40)


      def __password_setter(self, events):
      """Sets password"""

      self.__display_username.draw_text(screen)
      self.__password_text.draw(events, screen, SCREEN_WIDTH//2- 50,
SCREEN_HEIGHT//2 - 5, 300, 40)
      self.__password_text.enter(events)

      #If Password not blank
      if not self.__password_text.access() == "":
            self.__password = self.__password_text.access()
            self.__username_text.clear_text()
            self.__password_text.clear_text()
            self.__section = "user"
            self.__account_menu_options()

      def __account_menu_options(self):
      """Depending on if user attmepting sign up or login it will
      change the mneu choice after user enter there inputs correctly
      accordingly"""
```

```
if self.__menu_choice == "login":
      self.__menu_choice = "game"

if self.__menu_choice == "sign_up":
      self.__menu_choice = "creation"



def __login_into_account(self, events):
"""Draws back button and inputs for login in """

self.__go_back()
self.__user_password(events)

def __create_account(self, events):
"""Draws back button and inputs for account creation"""

self.__go_back()
self.__user_password(events)

def __accountID_total(self, conn):
"""Creates account ID"""

c = conn.cursor()
count = 1
total = 0
for letter in self.__username:
      total += ord(letter)
total += count

while True:
      try:
            c.execute(f"""
            SELECT AccountID
            FROM Users
            WHERE AccountID = {total}""")
            existing_account = c.fetchone()
            if existing_account:
            #AccountID already exists increment total
            total += 1
            else:
            #AccountID doesn't exist break the loop
            break
      except Exception as e:
            print(f"Error occurred: {e}")
            break
```

```
        return total


        def __update_user_registery(self, conn):
        """Updates the table with user infomation"""

        c = conn.cursor()
        accountID = self.__accountID_total(conn)
        self.__password = gogolf_lilbray.encrypter(self.__password)
        c.execute("INSERT INTO Users (AccountID, Username, Password)
VALUES (?, ?, ?)", (accountID, self.__username.lower(),
self.__password))
        last_inserted_id = []

        # Insert into SKINS table
        c.execute(f"""INSERT INTO Skins (AccountID)
                    VALUES ({accountID})""")
        last_inserted_id.append(c.lastrowid)


        # Insert into PLAYER_SKINS table
        c.execute(f"""INSERT INTO Player_Skins (AccountID)
                    VALUES ({accountID})""")


        def __account_maker(self):
        """Checks if user has followed the account creation rules
        and add there data to user registery """

        conn = sqlite3.connect("database/GoGolfDB.db")
        c = conn.cursor()
        #Removes spaces out of user name and repalces them with underscore
        self.__username = self.__username.replace(" ", "_")

        #Checks for spaces within the passwords
        for i in range(len(self.__password)):
              if self.__password[i] == " ":
                    self.__error = "Password cannot contain spaces"
                    self.__menu_choice = ""
                    return
        try:
              username_sql = c.execute(f'SELECT Username FROM Users WHERE
Username IS "{self.__username}"')
              username_sql = str("".join(c.fetchone()))
        except TypeError:
```

```
            pass #Username not in the database


      if username_sql == self.__username:
            self.__error = "Username taken"

      elif self.__username == "":
            self.__error = "Username was blank"

      else:
            self.__update_user_registery(conn)

      self.__clear_info()
      conn.commit()
      conn.close()
      self.__menu_choice = ""



      def __go_back(self):
      if self.__back.draw(screen):
            self.__menu_choice = ""


      def account_menu(self):
      while self.__InMenu:
            self.__bg.draw_background()
            events = pygame.event.get()

            self.__display_error.set_all(self.__error, RED, P2, 30, 30,
30 )
            self.__display_error.draw_text(screen)

            if self.__menu_choice == "":
                  self.__account_buttons(events)

            if self.__menu_choice == "login":
                  self.__login_into_account(events)

            if self.__menu_choice == "sign_up":
                  self.__create_account(events)

            if self.__menu_choice == "creation":
                  self.__account_maker()

            if self.__menu_choice == "game":
```

```
                    if self.__if_account_exist():
                    return self.__username
                    else:
                    self.__menu_choice = ""
              self.EventHandler(events)



      def __if_username_exist(self, conn):
      """Checks if username exist within the database, if it doesnt
      an error flag will shown on the screen """

      c = conn.cursor()
      username_sql = c.execute(f'SELECT Username FROM Users WHERE
Username IS "{self.__username}"')

      try:
            username_sql =  str("".join(c.fetchone())).lower()
      except TypeError:
            self.__clear_info()
            self.__error = "username does not exist"
            conn.close()
      return username_sql


      def __if_account_exist(self):
      """Check if user trying to login infomation is correct.
      This consit of checking for user username and password"""

      conn = sqlite3.connect("database/GoGolfDB.db")
      c = conn.cursor()
      username_sql = self.__if_username_exist(conn)

      #Check if Username vaild
      if username_sql == self.__username:
            password_sql = c.execute(f'SELECT Password FROM Users WHERE
Username IS "{self.__username}"')
            try:
                  password_sql =  str("".join(c.fetchone()))
                  self.__password =
gogolf_lilbray.encrypter(self.__password)
            except TypeError:
                  pass

            #Check password valid
            if password_sql == self.__password:
```

```
                    conn.close()
                    return True
              else:
                    self.__clear_info()
                    conn.close()
        self.__error = "username/password is not correct"
        conn.close()
        return False

        def __clear_info(self):
        """Sets the username abd password of the object to blank"""

        self.__username = ""
        self.__password = ""


        def EventHandler(self, events):
        for event in events:
              if event.type == pygame.QUIT:
                    LOGGER.info("Quit Application")
                    sys.exit()

        pygame.display.update()
        clock.tick(FPS)


        def set_in_menu(self, value):
        self.__InMenu = value

        def set_menu(self, value):
        self.__menu_choice = value


class MainMenu():
        def __init__(self, bg):
        self.__bg = bg #Backgorund
        self.__InMenu = True #If in menu
        self.__play = gogolf_lilbray.Button(50, 50, play_button, 0.5)
        self.__lb = gogolf_lilbray.Button(50, 200, lb_buttons, 0.5)
        self.__change_user = gogolf_lilbray.Button(50, 350,
change_user_button, 0.5)
        self.__skins = gogolf_lilbray.Button(50, 500, skins_button, 0.5)
        self.__quit = gogolf_lilbray.Button(50, 650, quit_buttons, 0.5)
        self.__three_hole = gogolf_lilbray.Button(50, 50,
three_hole_button, 0.5)
        self.__five_hole = gogolf_lilbray.Button(50, 200,
```

```
five_hole_button, 0.5)
      self.__random_hole = gogolf_lilbray.Button(50, 350, random_button,
0.5)
      self.__back = gogolf_lilbray.Button(50, SCREEN_HEIGHT-150,
back_buttons, 0.5)
      self.__strokes_lb = gogolf_lilbray.Button(50, 50, strokes_button,
0.5)
      self.__score_lb = gogolf_lilbray.Button(50, 200, score_button,
0.5)
      self.__time_lb = gogolf_lilbray.Button(50, 350, time_button, 0.5)
      self.__buy_skins = gogolf_lilbray.Button(50, 50, buy_skins_button,
0.5)
      self.__select_skins = gogolf_lilbray.Button(50, 200,
select_skins_button, 0.5)
      self.__game = None #GoGolf Object
      self.__menu_choice = "" #Which menu to access
      self.__leaderboard_filter = "Score" #Filter for leaderbaord,
defaults to SCORE
      self.__leaderboard_hole = None #Which collections of holes results
to display
      self.__leaderboard_text = gogolf_lilbray.Text()
      self.__change_range = 15 #Range of user to display
      self.__skin_buttons = self.__set_skin_buttons()
      self.__GOLFBALL = self.__GOLFBALL_SELECT("white")
      self.__prices_text = gogolf_lilbray.Text()
      self.__wallet_text = gogolf_lilbray.Text()

      def __set_skin_buttons(self):
      """Creates list of golf skins buttons """

      x = 400
      y = 100
      col_count = 0
      temp_skins_buttons = []
      for i in range(len(golf_skins)):
            temp_skins_buttons.append(gogolf_lilbray.Button(x, y,
golf_skins[i], 0.5))
            x+=300
            col_count+=1

            #Starts a new row
            if col_count > 2:
                  x = 400
                  y+= 150
                  col_count = 0
      return temp_skins_buttons
```

```
      def __GOLFBALL_SELECT(self, value):
      """Selects the correct image of the GOlFBAll image"""

      return pygame.image.load(f'Assests/{value}_ball.png')

      def __holes(self, username):
      """Game Select for what type for game the user want to play """

      if self.__three_hole.draw(screen):
                  self.__game = gogolf_gui.GoGolf(username, 3, False,
self.__GOLFBALL)
                  self.__game.PlayGame(3)

      if self.__five_hole.draw(screen):
                  self.__game = gogolf_gui.GoGolf(username, 5, False,
self.__GOLFBALL)
                  self.__game.PlayGame(5)

      if self.__random_hole.draw(screen):
            self.__game = gogolf_gui.GoGolf(username, 1, True,
self.__GOLFBALL)
            self.__game.PlayGame(1)

      if self.__back.draw(screen):
            self.__menu_choice = ""



      def __main_menu_button(self):
      """Main menu buttons to play the game, access the leaderboard,
quit """

      if self.__play.draw(screen):
            self.__menu_choice = "play"

      if self.__lb.draw(screen):
            self.__menu_choice = "leaderboard_hole_select"

      if self.__change_user.draw(screen):
            self.__menu_choice = "back"

      if self.__skins.draw(screen):
```

```
            self.__menu_choice = "skins"

        if self.__quit.draw(screen):
            sys.exit()

        def __leaderboard_holes(self):
        """Leaderboard game mode button select """

        if self.__three_hole.draw(screen):
            self.__menu_choice = "leaderboard"
            self.__leaderboard_hole = 3


        if self.__five_hole.draw(screen):
            self.__menu_choice = "leaderboard"
            self.__leaderboard_hole = 5

        if self.__back.draw(screen):
            self.__menu_choice = ""

        def __filter_options(self):
        """Filter options buttons for the leaderboard """

        if self.__strokes_lb.draw(screen):
            self.__leaderboard_filter = "Strokes"

        if self.__score_lb.draw(screen):
            self.__leaderboard_filter = "Score"

        if self.__time_lb.draw(screen):
            self.__leaderboard_filter = "Time"

        if self.__back.draw(screen):
            self.__menu_choice = "leaderboard_hole_select"


        def __ASC_DSC(self):
        """If data should repsented ascending or descending """

        if self.__leaderboard_filter == "Strokes" or
self.__leaderboard_filter == "Time":
            return "ASC"
        else:
            return "DESC"

        def __leaderboard_range_changer(self):
```

```
        """Changes the range of data to be drawn on the screen"""

        keys =  pygame.key.get_pressed()
        if keys[pygame.K_d]:
                self.__change_range += 15

        if keys[pygame.K_a] and self.__change_range > 15:
                self.__change_range -= 15


        def __draw_leaderboard(self, player_positions, leader_board,
lb_x_pos, lb_y_pos):
        """Display Users leaderboard results onto the GUI """

        for i in range(-15 + self.__change_range, self.__change_range):

                if i < len(player_positions):
                        self.__leaderboard_text.set_all(leader_board[i][0],
BLACK, P2, 20, lb_x_pos, lb_y_pos)
                        self.__leaderboard_text.draw_text(screen)
                        self.__leaderboard_text.set_all(leader_board[i][1],
BLACK, P2, 20, lb_x_pos + 30, lb_y_pos)
                        self.__leaderboard_text.draw_text(screen)
                        self.__leaderboard_text.set_all(leader_board[i][2],
BLACK, P2, 20, lb_x_pos + 175, lb_y_pos)
                        self.__leaderboard_text.draw_text(screen)
                        lb_y_pos += 35
                else:
                        break


        def __leaderboard(self):
        """Leaderboard menu"""

        self.__filter_options()
        screen.blit(leaderboard_wall, (SCREEN_WIDTH//2 - 50, 75))
        conn = sqlite3.connect("database/GoGolfDB.db")  # Fix the path to
the database

        order = self.__ASC_DSC()

        c = conn.cursor()


        c.execute(f"""SELECT Users.Username,
Holes{self.__leaderboard_hole}.{self.__leaderboard_filter}
```

```
      FROM Holes{self.__leaderboard_hole} INNER JOIN Users
      ON  Holes{self.__leaderboard_hole}.UserID == Users.UserID
      ORDER BY
Holes{self.__leaderboard_hole}.{self.__leaderboard_filter} {order}
      """)

      #Player_postions is where all player stats are saves tempoarily
      player_positions = [i for i in c.fetchall()]
      leader_board = []
      conn.close()

      self.__leaderboard_range_changer()

      #Sets all strings to be drawn for each player
      for i in range(0, len(player_positions)):
            pos = f"{str(i+1)}.{"":3}"
            string_pos_name = f"{player_positions[i][0]:8}"
            if not self.__leaderboard_filter == 'Time':
                  string_result = f"{self.__leaderboard_filter} :
{player_positions[i][1]:6}"
            else:
                  string_result = f"{self.__leaderboard_filter} :
{str(player_positions[i][1]//60).zfill(2):1}:{str(player_positions[i][1]
% 60).zfill(2):1}"


            leader_board.append((pos, string_pos_name, string_result))


      self.__draw_leaderboard(player_positions, leader_board,
SCREEN_WIDTH // 2, 125)


      def __skins_menu(self):
      """Skins menu"""

      if self.__select_skins.draw(screen):
            self.__menu_choice = "select_skins"

      if self.__buy_skins.draw(screen):
            self.__menu_choice = "buy_skins"

      if self.__back.draw(screen):
            self.__menu_choice = ""

      def __user_skin_info(self, conn, username):
```

```
        c = conn.cursor()

        c.execute(f"""SELECT Player_Skins.Coins, Users.AccountID
                    FROM Player_Skins INNER JOIN Users
                    ON Users.AccountID == Player_Skins.AccountID
                    WHERE Users.Username = '{username}'""")

        return c.fetchone()



        def __buy_menu(self, username):
        """Allows a user to purchase skins with there coins """


        screen.fill(BLUE) #Hides lags due to text

        if self.__back.draw(screen):
              self.__menu_choice = "skins"

        conn = sqlite3.connect("database/GoGolfDB.db")
        c = conn.cursor()
        AccountID = self.__user_skin_info(conn, username)


        x = 400
        y = 225
        count = 0 #Amount coloums allowed

        #Draws each price of every button and button
        for button_index in range(len(self.__skin_buttons)):
              self.__prices_text.set_all(f"Price: {button_index*82}",
BLACK, P2, 14, x, y)
              x+= 300
              count += 1

              #next rows
              if count > 2:
                    x = 400
                    y+= 150
                    count = 0

              self.__prices_text.draw_text(screen)

              #Checks if pressed
```

```
            if self.__skin_buttons[button_index].draw(screen):
                  c.execute(f"""SELECT
{golf_names[button_index].capitalize()}
                  FROM Skins
                  WHERE AccountID IS {AccountID[1]}""")
                  skinid = c.fetchone()
                  skinid = skinid[0]

                  #Checks if player has enough funds to buy it or
already owns it
                  if (AccountID[0] - (button_index*82)) > 0 and skinid
== 0:

                  #Pays for skins
                  c.execute(f"""UPDATE Player_Skins
                        SET Coins = {AccountID[0] - (button_index*82)}
                        WHERE AccountID = {AccountID[1]}""")
                  conn.commit()

                  #Allows the player to select the skin now
                  c.execute(f"""UPDATE Skins
                  SET {golf_names[button_index]} = 1
                  WHERE AccountID = {AccountID[1]}""")
                  conn.commit()

            self.__wallet_text.set_all(f"Coins: {AccountID[0]:5}",
BLACK, P2, 15, SCREEN_WIDTH - 135, 20)
            self.__wallet_text.draw_text(screen)
      conn.close()




      def __select_menu(self, username):
      """Selects skin that the current player/user owns """

      if self.__back.draw(screen):
            self.__menu_choice = "skins"

      conn = sqlite3.connect("database/GoGolfDB.db")
      c = conn.cursor()

      #Selects all data in SKINS table for that player
      c.execute(f"""SELECT Skins.*
            FROM Player_Skins
```

```
            INNER JOIN Skins ON Player_Skins.AccountID = Skins.AccountID
            INNER JOIN Users ON Users.AccountID = Player_Skins.AccountID
            WHERE Users.Username = '{username}'
      """)
      skins_owned = c.fetchone()

      #Draws the buttons
      for button_index in range(1, len(self.__skin_buttons) + 1):
            if skins_owned[button_index] == 1:
                  if self.__skin_buttons[button_index-1].draw(screen):
                  self.__GOLFBALL =
self.__GOLFBALL_SELECT(golf_names[button_index-1])

      conn.commit()
      conn.close()


      def main_menu(self, logo, username):
      """Main Menu"""

      self.__InMenu = True
      self.__menu_choice = ""
      while self.__InMenu:
            events = pygame.event.get()

            self.__bg.draw_background()
            self.__bg.side_bar()

            if not self.__menu_choice in ["leaderboard", "buy_skins",
"select_skins"]:
                  logo.floatiing_logo(SCREEN_WIDTH//2 - 120,
SCREEN_HEIGHT//2 - 200)

            if self.__menu_choice == "":
                  self.__main_menu_button()

            if self.__menu_choice == "play":
                  self.__holes(username)

            if self.__menu_choice == "leaderboard_hole_select":
                  self.__leaderboard_holes()

            if self.__menu_choice == "leaderboard":
                  self.__leaderboard()

            if self.__menu_choice == "skins":
```

```
                self.__skins_menu()

            if self.__menu_choice == "buy_skins":
                self.__buy_menu(username)

            if self.__menu_choice == "select_skins":
                self.__select_menu(username)

            if self.__menu_choice == "back":
                self.__InMenu = False


            self.EventHandler(events)


        def get_in_menu(self):
        return self.__InMenu

        def EventHandler(self, events):
        for event in events:
            if event.type == pygame.QUIT:
                LOGGER.info("Quit Application")
                sys.exit()

        pygame.display.update()
        clock.tick(FPS)



def menu():
        Tee = Tee_Box()
        Tee.Game()
```

Gogolf_gui.py

```
import pygame
import logging
from math import ceil, degrees, sqrt, cos, atan2, sin
import sys
import pickle
import gogolf_lilbray
```

```
import sqlite3
from random import randint
import numpy as np
import pandas as pd
import openpyxl

#pygame.init()

LOGGER = logging.getLogger(f"{__name__}")

#Create display window
SCREEN_HEIGHT = 768
SCREEN_WIDTH = 1366
screen = pygame.display.set_mode((SCREEN_WIDTH,SCREEN_HEIGHT))


#FPS
clock = pygame.time.Clock()
FPS = 60 #100

pygame.display.set_caption("Go Golf")
#icon = pygame.image.load('Assets/uno_icon.ico')
#pygame.display.set_icon(icon)
pygame.key.set_repeat(0,0)

#Load Images

GOLFBALL_IMG = pygame.image.load('Assests/white_ball.png')


#Image Widiths and Heights
background_1 = pygame.image.load('Background_Img/1.png').convert_alpha()
background_2 = pygame.image.load('Background_Img/2.png').convert_alpha()
background_3 = pygame.image.load('Background_Img/4.png').convert_alpha()


GOLFBALL_WIDITH = GOLFBALL_IMG.get_width()
GOLFBALL_HEIGHT = GOLFBALL_IMG.get_height()


#Colours
BLUE = "#2596be"
WHITE = "#F5FEFD"
ORANGE = "#edc9af"

#Fonts
```

```
WII_FONT = "Assests/Fonts/AOTFShinGoProMedium.otf"
P2 = "Assests/Fonts/Retro Gaming.ttf"

#Grid
ROWS = 16
MAX_COLS = 150
TILE_SIZE = SCREEN_HEIGHT // ROWS
TILE_TYPES = 21


class GoGolf():
      def __init__(self, username, level, random, BALL_SKIN):
      self.__game_running = True #is the game running
      self.__current_player = Player(username, BALL_SKIN) #Current
player
      self.__currentlevel = Level(1) #Current level
      self.__muitplier = 1.5 #Muilplier
      self.__distance_moved = 0 #Distance moved from the start
      self.__world = World()
      self.__player_text = gogolf_lilbray.Text()
      self.__count = -10  #Offset
      self.__random = random #If the game should select levels at random
      self.__world_queue = self.__create_world_queue(level)
      self.__skin_of_golfball = BALL_SKIN #Colour of ball
      self.ticks = ticks_data = [] #Test FPS by addding all recored
Frame rates to list

      def __create_world_queue(self, level):
      """Creates a Queue that hold each level"""

      worlds = gogolf_lilbray.queue(level) #Holds a queue of each level
loaded
      for i in range(level):

            if self.__random:
                  #Picks a random level
                  pickle_in = open(f'Levels/level{randint(1, 7)}_data',
'rb')

            else:
                  #Finds the level
                  pickle_in = open(f'Levels/level{i+1}_data', 'rb')

            #Load the level
            worlds.enqueue(pickle.load(pickle_in))
```

```
        return worlds

        def __UI_playerinfo(self):
        """Displays UI elements such as amount of strokes and
        score"""

        self.__player_text.set_all(f"Strokes:
{self.__current_player.get_strokes():3}", ORANGE, P2, 22, SCREEN_WIDTH -
167, SCREEN_HEIGHT - 30)
        self.__player_text.draw_text(screen)
        self.__player_text.set_text(f"Time:
{str(self.__current_player.get_time()//60).zfill(2):1}:{str(self.__curre
nt_player.get_time() % 60).zfill(2):1}")
        self.__player_text.set_y(10)
        self.__player_text.draw_text(screen)

        def __prepare_next_level(self):
        """Reset level conditions and changes the current level"""


self.__current_player.score_calculator(len(self.__world.get_goal_list())
)
        self.__current_player.reset()
        self.__world = World()
        self.__current_player.new_ball(self.__skin_of_golfball)
        self.__currentlevel.set_World_data([])
        self.__currentlevel.next_level(1)

        def __change_level(self):
        """Proccess new data for the new level and reset level postions"""

        self.__world.proccess_data(self.__currentlevel.get_world_data(),
self.__currentlevel.get_block_list(),
self.__current_player.get_golfball())

self.__current_player.get_golfball().set_spawn_location(self.__world.get
_spawn_location())

self.__current_player.get_golfball().set_ball_postion(self.__world.get_s
pawn_location()[0] + self.__distance_moved,
self.__world.get_spawn_location()[1])
        self.__currentlevel.set_control_background(True)


        def __timer(self):
        """Timer that uses FPS as timer, meaning timer is linked
```

```
        to the game and not the computer"""

        self.__count += 1
        if self.__count > clock.get_fps():
                self.__current_player.add_to_time(1)
                self.__count = 0

        def __coin_update(self, value):
        conn = sqlite3.connect("database/GoGolfDB.db")
        c = conn.cursor()

        #Finds the amount of coins and the AccountID for that user
        c.execute(f"""SELECT Player_Skins.Coins, Player_Skins.AccountID
        FROM Player_Skins INNER JOIN Users
        ON Player_Skins.AccountID == Users.AccountID
        WHERE Users.Username =
'{self.__current_player.get_username()}'""")
        AccountID = c.fetchone()

        #Updates the amount of coins for that account id
        c.execute(f"""UPDATE Player_Skins
                SET Coins = {AccountID[0] + (16* value)}
                WHERE AccountID = {AccountID[1]}""")

        conn.commit()
        conn.close()




        def __save_game_records(self, level):
        """Records game results"""

        conn = sqlite3.connect("database/GoGolfDB.db")
        c = conn.cursor()

        #Finds the UserID
        c.execute(f"SELECT UserID FROM Users WHERE Username =
'{self.__current_player.get_username()}'")
        UserID = c.fetchone()

        #Inserts Data to the game DB
        c.execute(f"""INSERT INTO Holes{level} (UserID, Score, Strokes,
Time)
        VALUES ('{UserID[0]}','{self.__current_player.get_total_score()}',
'{self.__current_player.get_total_stokes()}',
```

```
        '{self.__current_player.get_total_time()}')""")

        conn.commit()
        conn.close()


        def __is_level_completed(self, level):
        """Checks to either to load level or
        to save user data"""

        if self.__world.level_completed():
                self.__prepare_next_level()

                #Checks if the current level exceeds the amount of levels
for the gamemode
                if not self.__currentlevel.get_level() > level:
                        self.LoadWorld()
                        self.__change_level()
                else:
                        #Game does not save random gamemode results
                        if not self.__random:
                        self.__save_game_records(level)
                        self.__coin_update(level)
                        self.__scoreboard()
                        self.__game_running = False


        def __scoreboard(self):
        """Displays end of game results """

        self.__currentlevel.set_scroll_background(2000) #Still loaded on
the level however far away from the level platforms
        time = 5
        count = 0
        x_pos = -300
        game_result = gogolf_lilbray.Text()

        #Time Limit For Scoreboard
        while time < 15:
                count += 1
                if count > clock.get_fps():
                        self.__current_player.add_to_time(1)
                        count = 0
                        time+=1

                self.__currentlevel.create_background()
```

```
self.__currentlevel.set_scroll_background(self.__currentlevel.get_scroll
_background() + 5)

            #Draws and moves texts accross the screen
            x_pos +=2
            game_result.set_all(f"Score:
{self.__current_player.get_total_score()}", "#ff8500", P2, 30, x_pos,
200)
            game_result.draw_text(screen)
            game_result.set_all(f"Strokes:
{self.__current_player.get_total_stokes()}", "#ff8500", P2, 30, x_pos,
400)
            game_result.draw_text(screen)
            game_result.set_all(f"Time:
{str(self.__current_player.get_total_time()//60).zfill(2):1}:{str(self._
_current_player.get_total_time() % 60).zfill(2):1}", "#ff8500", P2, 30,
x_pos , 600)
            game_result.draw_text(screen)
            self.EventHandler()



      def PlayGame(self, level):
      """Main Game Loop """

      #Load first level
      self.LoadWorld()
      self.__world.proccess_data(self.__currentlevel.get_world_data(),
self.__currentlevel.get_block_list(),
self.__current_player.get_golfball())
      #Sets spawn location

self.__current_player.get_golfball().set_spawn_location(self.__world.get
_spawn_location())

      while self.__game_running:

            self.__timer()
            self.__is_level_completed(level)
            self.__currentlevel.create_background()

            #Before the player sets up the shot
            if self.__currentlevel.get_control_backgorund():
```

```
                self.level_controls() #Can control the camera
                self.__current_player.get_golfball().display_ball()
                self.__world.draw(self.__distance_moved)
                if not self.__currentlevel.get_control_backgorund():
                self.set_camera_pos()

            #Player wants to shoot
            else:
                self.__world.draw(self.__distance_moved)
                self.player_can_shoot(self.__current_player)
                self.__current_player.get_golfball().display_ball()
                self.__player_text.draw_text(screen)

                #If the player does not want to take the shot anymore
                if self.__current_player.get_golfball().get_stop():
#Checks the player wants to stop wanting to shoot
                self.__current_player.get_golfball().set_stop(False)

                #Activates the control of the camera again
                self.__currentlevel.set_control_background(True)

self.__current_player.get_golfball().set_clicked(False)

            #If the player shot
            if  self.__current_player.get_golfball().get_clicked():
                self.__current_player.get_golfball().display_ball()
                self.__world.draw(self.__distance_moved)
                self.__player_text.draw_text(screen)

            self.__UI_playerinfo()
            self.EventHandler()


      def EventHandler(self):
      """Event Handler """

      events = pygame.event.get()
      for event in events:
            if event.type == pygame.QUIT:
                LOGGER.info("Quit Application")

                #temp #Test FPS
                data = {'Ticks': ticks_data}
                df = pd.DataFrame(data, index=range(len(self.ticks)))
# Specify index as a range
                df.to_excel('ticks_data.xlsx', index=False)
```

```
                  #Test FPS


                  sys.exit()

            if event.type == pygame.KEYDOWN:
                  if event.key == pygame.K_ESCAPE:

                  #temp #Test FPS
                  data = {'Ticks': self.ticks}
                  df = pd.DataFrame(data, index=range(len(self.ticks)))
#Specify index as a range
                  df.to_excel('ticks_data.xlsx', index=False)
                  #Test FPS

                  self.__game_running = False

      pygame.display.update()
      clock.tick(FPS)
      self.ticks.append(clock.get_fps()) #Test FPS



      def update_camera_postion(self):
      """Follows the ball in the air """

      player_ball = self.__current_player.get_golfball()
      ball_postion = player_ball.get_postion_of_ball()
      speed_movement = player_ball.get_x_velocityofball()//115 #120
      self.__muitplier = 2.7

      #Forwards movement
      if ball_postion[0] > self.__currentlevel.get_threshold_left() and
self.__current_player.get_golfball().get_direction() == 1:

            offset_x = self.__muitplier * speed_movement
            self.__currentlevel.update_scroll_right(offset_x)
            player_ball.update_ball_to_right(offset_x)
            self.__distance_moved -= offset_x

      #Backwards movement
      elif ball_postion[0] < self.__currentlevel.get_threshold_right()
and self.__current_player.get_golfball().get_direction() == -1:

            offset_x = self.__muitplier * speed_movement
```

```
            self.__currentlevel.update_scroll_right(offset_x)
            player_ball.update_ball_to_right(offset_x)
            self.__distance_moved -= offset_x



    def set_camera_pos(self):
    """Centers the ball in middle of the screen """

    golfball = self.__current_player.get_golfball()
    golfball_x_pos = golfball.get_postion_of_ball()[0]

    LOGGER.info(f"golfball x pos before {golfball_x_pos} ")
    LOGGER.info(f"Scroll Before
{self.__currentlevel.get_scroll_background()}")

    #Calculate the offset to center the golf ball on the screen
    offset_x = -(golfball.get_postion_of_ball()[0] - SCREEN_WIDTH //
2) + self.__distance_moved
    offset_y = golfball.get_postion_of_ball()[1]

    LOGGER.info(f"Distance camera from ball x of camera is
{self.__distance_moved} ")
    LOGGER.info(f"Offset x of camera is {offset_x} ")

    #Center the golf ball on the screen
    golfball.set_ball_postion(SCREEN_WIDTH//2 , offset_y)

    #Update the background scroll to center the camera on the golf
ball
    self.__currentlevel.set_scroll_background(offset_x)

    self.__distance_moved = (offset_x)
    LOGGER.info(f"golfball x pos after {golfball_x_pos} ")
    LOGGER.info(f"Scroll after
{self.__currentlevel.get_scroll_background()}")


    def player_can_shoot(self, player):
    """Everything to do with stroke attempt """

    player_ball = self.__current_player.get_golfball()

    #If player wants to set up a new shot
    if not self.__currentlevel.get_control_backgorund():
```

```
            if pygame.mouse.get_pressed()[0] and not
player_ball.get_clicked():
                  self.__currentlevel.set_control_background(True)



            #Let player set up the shot and shoot it
            if not player_ball.get_clicked():
                  player_ball.aim(player)
                  player_ball.swing()

            #While ball is not stoped
            if not player_ball.get_stop():

player_ball.trajecotry(self.__world.get_obstacle_list(),
self.__distance_moved, self.__current_player)
                  if player_ball.get_clicked():
                  self.update_camera_postion()



      def level_controls(self):
      """Level control for scrolling left and right, ready to shoot and
a muitlplier
      to move faster """

      if self.__currentlevel.get_control_backgorund():

            keys = pygame.key.get_pressed()
            spped_muiltplayer = 1

            #Muiltplier
            if keys[pygame.K_LSHIFT]:
                  spped_muiltplayer = 5

            speed = 5*spped_muiltplayer

            #PLayer moves to the right
            if keys[pygame.K_d] and
abs(self.__distance_moved)//TILE_SIZE < 150:
                  self.__currentlevel.update_scroll_right(speed)

self.__current_player.get_golfball().update_ball_to_right(speed)
                  self.__distance_moved -= speed
```

```
            #PLayer moves to the left
            if keys[pygame.K_a] and self.__distance_moved //TILE_SIZE <
2:
                  self.__currentlevel.update_scroll_left(speed)

self.__current_player.get_golfball().update_ball_to_left(speed)
                  self.__distance_moved += speed

            #Player ready to swing
            if keys[pygame.K_RETURN]:
                  self.__currentlevel.set_control_background(False)



      def LoadWorld(self):
      """Load selected level world"""

      #Create empty world /2d array
      world_data = [[-1] * 150 for _ in range(16)]

      LOGGER.info("Save Level")

      #Finds the level
      level_in = self.__world_queue.dequeue()

      #Reset World
      self.__currentlevel.set_scroll_background(0)
      self.__currentlevel.set_World_data(world_data)

      #Load the level
      self.__currentlevel.set_World_data(level_in)
      LOGGER.info(f"Level {self.__currentlevel.get_level()} has been
loaded ")



class World():
      def __init__(self):
      self.obstacle_list = [] #All tile/blocks in the wrold
      self.__ball_spawn_location = [0, 0]
      self.__goals_list = []

      def proccess_data(self, world_data, tile_img_list, golfball):
      """Proccess the world data and gives all tiles it own
properties"""
```

```
        for y, row in enumerate(world_data):
                for x, block in enumerate(row):
                        if block >= 0:
                        img = tile_img_list[block]
                        img_rect = img.get_rect()
                        img_rect.x = x * TILE_SIZE
                        img_rect.y = y * TILE_SIZE
                        tile_data = (img, img_rect)

                        #Normal Block
                        if block <= 7 or block >= 12 and block <= 17:
                                tile_data = Tile(img, img_rect, x, y, "obstacle")
                                self.obstacle_list.append(tile_data)

                        #Water Block
                        elif block >= 8 and block <= 11:
                                tile_data = Water(img, img_rect, x, y, "water")
                                self.obstacle_list.append(tile_data)

                        #Goal Block
                        elif block >= 18 and block <= 19:
                                tile_data = Goal(img, img_rect, x, y, "goal")
                                self.obstacle_list.append(tile_data)
                                self.__goals_list.append(tile_data)

                        #Golfball Postion
                        elif block == 20:
                                golfball.set_ball_postion(x*TILE_SIZE,
y*TILE_SIZE + 28)
                                self.__ball_spawn_location = (x*TILE_SIZE,
y*TILE_SIZE + 28)


        def draw(self, distance_moved):
        """Draws game tile """

        #Turns the level block/tile location back to pygame coords in
numpy array
        x_coordinates = np.array([block.get_rect().x // TILE_SIZE for
block in self.obstacle_list])

        #Checks for blocks that range in coords that would be dispalyed on
screen at that time
        mask = (x_coordinates * TILE_SIZE + distance_moved >= -TILE_SIZE)
& (x_coordinates * TILE_SIZE + distance_moved <= SCREEN_WIDTH +
TILE_SIZE)
```

```
        visible_blocks = np.array(self.obstacle_list)[mask]

        #Renders the visbale blocks
        for block in visible_blocks:
                temp = block.get_rect().x
                screen.blit(block.get_img(), (temp + distance_moved,
block.get_rect().y))
                if block.get_block_type() == "goal":
                        block.Display_Goal(distance_moved)


        def level_completed(self):
        """Checks if all goals have been hit"""

        holes = 0 #Holes starts at zero because every level should have
one or more holes
        for goal in self.__goals_list:
                if not goal.get_goal_active():
                        holes += 1 #Adds everyhole that been entered

        if holes == len(self.__goals_list):
                return True #If the holes enteted is same amount as goals
then the level is completed
        return False


        def get_obstacle_list(self):
        return self.obstacle_list

        def get_goal_list(self):
        return self.__goals_list

        def get_spawn_location(self):
        return self.__ball_spawn_location



class Tile():
        def __init__(self, img, img_rect, x, y, img_type):
        self.__img = img #Tile Image
        self.__rect = img_rect #Tile Rectangle
        self.__rect.midtop = (x + TILE_SIZE //2, y + (TILE_SIZE -
self.__img.get_height())) #Changes midtop postion
        self.__rect.x = x * TILE_SIZE #Tile Rectangle x postion
        self.__rect.y = y * TILE_SIZE #Tile Rectangle y postion
        self.__type = img_type #Image Type
```

```
	def get_img(self):
	return self.__img

	def get_rect(self):
	return self.__rect

	def get_block_type(self):
	return self.__type

	def set_rectx(self ,v):
	self.__rect.x = v

	def set_recty(self, v):
	self.__rect.y = v

	def set_rect(self, distnace):
	self.__rect.x += distnace



class Water(Tile):
	def __init__(self, img, img_rect, x, y, img_type):
	super().__init__(img, img_rect, x, y, img_type)



class Goal(Tile):
	def __init__(self, img, img_rect, x, y, img_type):
	super().__init__(img, img_rect, x, y, img_type)
	self._goal_post =
pygame.image.load("Assests/tile/19.png").convert_alpha()
	self._goal_post = pygame.transform.scale(self._goal_post,
(TILE_SIZE, TILE_SIZE))
	self.__goal_active = True

	def Display_Goal(self, distance_moved):
	"""Displays that the goal is active"""

	if self.__goal_active == True:
		screen.blit(self._goal_post, (self.get_rect().x +
distance_moved, self.get_rect().y -1*TILE_SIZE))

	def set_goal_active(self, value):
	self.__goal_active = value
```

```
        def get_goal_active(self):
        return self.__goal_active


class Player():
        def __init__(self, username, GOLFBALL):
        self.__username = username #Player Name
        self.__totalstrokes = 0 #Total stokes
        self.__totalscore = 0 #Total score
        self.__totaltime = 0
        self.__playergolfball = Golf_Ball(GOLFBALL) #Player ball
        self.__time = 0
        self.__strokes = 0

        def __divider(self):
        """Diver is way of measuring rewaring a player
        with higher divider for fewer strokes"""

        if self.__strokes >= 0 and self.__strokes <= 9:
                return 14
        elif self.__strokes >= 10 and self.__strokes <= 15:
                return 10
        elif self.__strokes >= 16 and self.__strokes <= 20:
                return 8
        elif self.__strokes >= 21 and self.__strokes <= 28:
                return 6
        elif self.__strokes >= 29 and self.__strokes <= 35:
                return 2
        return -(self.__strokes/10)

        def add_to_totalstokes(self, strokes):
        self.__totalstrokes += strokes

        def add_to_stokes(self, strokes):
        self.__strokes += strokes

        def add_to_time(self, value):
        self.__time += value

        def add_to_total_time(self, value):
        self.__time += value

        def new_ball(self, GOLFBALL):
        self.__playergolfball = Golf_Ball(GOLFBALL)

        def reset(self):
```

```
        """Reset strokes and time"""

        self.__totalstrokes += self.__strokes
        self.__totaltime += self.__time
        self.__strokes = 0
        self.__time = 0

        def score_calculator(self, amount_of_goals):
        """Calculates score"""

        starting_score = amount_of_goals * 1000 #Starting Score
        LOGGER.info(f"starting score: {starting_score}")
        divder = self.__divider()  #Divider
        LOGGER.info(f"divider: {divder} and strokes: {self.__strokes}")
        LOGGER.info(f"Time {self.__time} and score {(round(starting_score
/ (self.__time / divder)))*10}")
        self.__totalscore += round(((starting_score / (self.__time /
divder))*100)) #Algorithm
        LOGGER.info(f"Total score: {self.__totalscore}")


        def get_username(self):
        return self.__username

        def get_total_stokes(self):
        return self.__totalstrokes

        def get_strokes(self):
        return self.__strokes

        def get_golfball(self):
        return self.__playergolfball

        def get_time(self):
        return self.__time

        def get_total_time(self):
        return self.__totaltime

        def set_time(self):
        return self.__time

        def get_total_score(self):
        return self.__totalscore

        def set_username(self, username):
```

```
        self.__username = username

        def set_totalstrokes(self, strokes):
        self.__totalstrokes = strokes

        def set_strokes(self, strokes):
        self.__strokes = strokes



class Level():
        def __init__(self, levelname):
        self.__level = levelname #Current level
        self.__backgroundscroll = 0 #Background scroll starts at zero
        self.__backgrond_widith = background_1.get_width()
        self.__backgrond_height = background_2.get_height()
        self.__background_threshold_right= 1000 #Camera threshold to the
right
        self.__background_threshold_left = 100  ##Camera threshold to the
left
        self.__background_threshold_middle = SCREEN_WIDTH//2 #Camera
threshold to the middle
        self.__tiles = ceil(SCREEN_WIDTH /  self.__backgrond_widith) + 1
#how many images needed to display background
        self.__control_background = True #If the user allowed use scroll
        self.__block_list = self.__CreateBlockList() #Creates a list of
all images of tiles/blocks
        self.__world_data = [] #All infomation about the current world


        def __CreateBlockList(self):
        """Creates a list of all images of tiles/blocks """

        LOGGER.info("Create List Of Image Tiles")
        tile_list = []
        for x in range(TILE_TYPES):
                img = pygame.image.load(f'Assests/tile/{x}.png') #Loads the
image
                img = pygame.transform.scale(img, (TILE_SIZE, TILE_SIZE))
#Scales it
                tile_list.append(img) #Add to list
        return tile_list

        def draw_background(self):
        """Draw the background for the game """
```

```
        #Draw all three layers of images
        for i in range(0, self.__tiles):
                screen.blit(background_1, (((i * self.__backgrond_widith) +
self.__backgroundscroll) , 0))
                screen.blit(background_2, (((i * self.__backgrond_widith) +
self.__backgroundscroll), 0))
                screen.blit(background_3, (((i * self.__backgrond_widith) +
self.__backgroundscroll), 0))

        def update_backgorund(self):
        """Add a new tile in either vertical direction"""

        #Reset Scroll (For going forward)
        if abs(self.__backgroundscroll) > self.__backgrond_widith:
                self.__backgroundscroll = 0

        #Reset Scroll (For going backward)
        if abs(self.__backgroundscroll) < self.__backgrond_widith and
self.__backgroundscroll > 0:
                self.__backgroundscroll = self.__backgroundscroll -
self.__backgrond_widith

        def create_background(self):
        """Creates the game backgorund and updates it """
        self.draw_background()
        self.update_backgorund()

        def update_scroll_right(self, number):
        """Scroll backgorund right """
        self.__backgroundscroll -= number

        def update_scroll_left(self, number):
        """Scroll backgorund left """
        self.__backgroundscroll += number

        def next_level(self, value):
        self.__level += value

        def get_block_list(self):
        return self.__block_list

        def get_control_backgorund(self):
        return self.__control_background

        def get_scroll_background(self):
```

```
        return self.__backgroundscroll

        def get_threshold_right(self):
        return self.__background_threshold_right

        def get_threshold_left(self):
        return self.__background_threshold_left

        def get_threshold_middle(self):
        return self.__background_threshold_middle

        def get_level(self):
        return self.__level

        def get_world_data(self):
        return self.__world_data

        def set_control_background(self, locked):
        self.__control_background = locked

        def set_scroll_background(self, number):
        self.__backgroundscroll = number

        def set_level(self, level):
        self.__level = level

        def set_World_data(self, value):
        self.__world_data = value



class Golf_Ball():
        def __init__(self, GOLFBALL):
        self.__GOLFBALL_image =
pygame.transform.scale(GOLFBALL,(int(GOLFBALL.get_width()*1.3),
        int(GOLFBALL.get_height()*1.3)))
        self.__BallPostion = [0,0] #[x,y] Where image of the ball
postioned within the game
        self.__GOLFBALL_rect = GOLFBALL_IMG.get_rect() #Rectangle for
collison for golfball
        self.__GOLFBALL_rect.x = self.__BallPostion[0] #Rectangele x
postion
        self.__GOLFBALL_rect.y = self.__BallPostion[1] #Rectangele y
postion
        self.__GRAVITY = 10 #Gravity
        self.__BALL_MASS = 0.037 #The weight of the ball
```

```
        self.__AirResistance = 0.005 #The amount air resitance
        self.__BallSpeed = 0 #The power of the swing
        self.__BallAngle = 0 #The angle of the swing
        self.__clicked = False #If the player has swang
        self.__stop = False #If the turn is over
        self.__display_shot_info = gogolf_lilbray.Text() #Create text
object to display text
        self.__direction = 1 #Direction the ball is going in the x
directions
        self.__r_colour = 247 #Colour of line when aiming from a cetian
distnace
        self.__ball_spawn_location = (0, 0) #Ball spawn location
        self.__ball_pre_pos = (0, 0) #Set old postion of ball location


        def __shot_infomation(self):
        """Displays infomation about the shot, including speed and
angle"""
        self.__display_shot_info.set_all(f"Ball Speed:
{round(self.__BallSpeed)} ", ORANGE, P2, 22, 10, 10)
        self.__display_shot_info.draw_text(screen)
        self.__display_shot_info.set_text(f"Ball Angle:
{abs(round(degrees(self.__BallAngle))-180)}")
        self.__display_shot_info.set_y(45)
        self.__display_shot_info.draw_text(screen)


        def __power_increaser(self):
        """Allows the user to have sharper increase in power to helps in
tight situations"""
        keys = pygame.key.get_pressed()
        if keys[pygame.K_LCTRL]:
                return 4
        return 1


        def __shot_calculator(self, mouse_position):
        """Calculates the power/angle of the user shot based on distance
from distance and the user angle postion from the ball.
        In additon also sets the direction the ball intially travelling"""

        #Difference between mouse pos and ball pos
        dx = mouse_position[0] - self.__BallPostion[0]
        dy = mouse_position[1] - self.__BallPostion[1]

        #Finds the distance
```

```
        self.__BallSpeed = sqrt(dx**2 + dy**2) * 1.5

        #Finds the angle
        self.__BallAngle = atan2(dy, dx) #Radians
        self.__BallSpeed = self.__power_increaser()*self.__BallSpeed

        #Cap for distance
        if self.__BallSpeed > 1000:
                self.__BallSpeed = 1000

        #Checks for intial direction
        if degrees(self.__BallAngle) < 90 and degrees(self.__BallAngle) >
-90:
                self.__direction = -1

        if abs(degrees(self.__BallAngle)) > 90 :
                self.__direction = 1


        def __line_up(self, mouse_position):
        """Displays line to help player player see where the ball will go
and the amount power with
        change the line colour as another indaicator"""
        self.__r_colour = self.__BallSpeed//4
        pygame.draw.line(screen, (self.__r_colour, 180, 47),
(self.__BallPostion[0] + 8, self.__BallPostion[1] + 8), mouse_position,
3)


        def __handle_y_collsion(self, tile, distance):
        """Handle y collsions of the ball with tiles """

        if abs(self.__VelocityOfBall[1]) > 75 and
abs(self.__VelocityOfBall[0]) > 0.1:
                #Affects the game y ball velocity (flips it to going
opposite way and losing energy)
                self.__VelocityOfBall[1] = -self.__VelocityOfBall[1]  / 4

                #Affects the game x ball velocity
                if abs(self.__VelocityOfBall[0]  / 6)*self.__direction >
0.1:
                        self.__VelocityOfBall[0] -=
abs(self.__VelocityOfBall[0]  / 6)*self.__direction


        elif abs(self.__VelocityOfBall[1]) > 0.2 and
```

```
abs(self.__VelocityOfBall[0]) > 0.1:

            #Affects the game y ball velocity (flips it to going
opposite way and losing energy)
            self.__VelocityOfBall[1] = -self.__VelocityOfBall[1]  / 3

            #Affects the game x ball velocity
            if abs(self.__VelocityOfBall[0]  / 6)*self.__direction >
0.1:
                  self.__VelocityOfBall[0] -=
abs(self.__VelocityOfBall[0]  / 5.5)*self.__direction


            LOGGER.info(f"hit y {tile.get_rect().y}")

      else:
            #Ball has stop moving, so this tell the program that
            self.__stop = True
            self.__ball_pre_pos = (self.__BallPostion[0] - distance,
tile.get_rect().y - 20)
            #Set ball postion above tile to help ball be above the ties
            self.set_ball_postion(self.__BallPostion[0],
tile.get_rect().y - 20)


      def __handle_x_collsion(self):
      """Handle x collsions of the ball with tiles """

      self.__direction = self.__direction*-1
      self.__VelocityOfBall[0] = -self.__VelocityOfBall[0]  / 2.5



      def __handle_goal_collsion(self, tile, distance, update_step):
      """Handle goal collsions of the ball with goal tiles """

      #Checks if ball gone in the hole
      if tile.get_rect().y - TILE_SIZE*0.12 < self.__GOLFBALL_rect.y and
tile.get_goal_active():

            tile.set_goal_active(False)
            self.__reset_ball_location(distance)


      def __reset_ball_location(self, distance):
      """Reset ball location to back to the starting point """
```

```
      self.__VelocityOfBall[0] = 0
      self.__VelocityOfBall[1] = 0
      self.set_ball_postion(distance + self.__ball_spawn_location[0],
self.__ball_spawn_location[1])


      def __collison(self, tile, distance ,update_step ,player):
      """Checks for collisons"""

      #CHECK ball fell of the map or touches water
      if self.__BallPostion[1] > SCREEN_HEIGHT + 50 or
(tile.get_block_type() == "water" and
tile.get_rect().colliderect(-distance + self.__GOLFBALL_rect.x ,
      self.__GOLFBALL_rect.y + (self.__VelocityOfBall[1]*update_step),
self.__GOLFBALL_image.get_width(), self.__GOLFBALL_image.get_height())):

            #Kill ball velocity
            self.__VelocityOfBall[0] = 0
            self.__VelocityOfBall[1] = 0

            #Set ball postion
            if not self.__ball_pre_pos == (0,0):
                  self.set_ball_postion(self.__ball_pre_pos[0] +
distance, self.__ball_pre_pos[1])
            else:
                  self.__reset_ball_location(distance)
            player.add_to_stokes(1)

      #Checks if ball hit a goal tile/block
      if tile.get_block_type() == "goal" and
tile.get_rect().colliderect(-distance + self.__GOLFBALL_rect.x ,
      self.__GOLFBALL_rect.y, self.__GOLFBALL_image.get_width(),
self.__GOLFBALL_image.get_height()):

            self.__handle_goal_collsion(tile, distance, update_step)
            LOGGER.info(f"Goal hit: {tile.get_rect().y - 1*TILE_SIZE//3
> self.__GOLFBALL_rect.y}")
            LOGGER.info(f"TILE pos: {tile.get_rect().y - 1*TILE_SIZE//3}
BAll pos: {self.__GOLFBALL_rect.y}")

      #If ball dosent hit a goal tile
      else:

            #CHECK Y collsions
            if tile.get_rect().colliderect(-distance +
```

```
self.__GOLFBALL_rect.x , self.__GOLFBALL_rect.y +
(self.__VelocityOfBall[1]*update_step) + 1,
self.__GOLFBALL_image.get_width(), self.__GOLFBALL_image.get_height()):
                    self.__handle_y_collsion(tile, distance)

            #CHECK X collsions
            if tile.get_rect().colliderect(-distance +
self.__GOLFBALL_rect.x + (self.__VelocityOfBall[0] *update_step),
self.__GOLFBALL_rect.y, self.__GOLFBALL_image.get_width(),
self.__GOLFBALL_image.get_height()):
                    self.__handle_x_collsion()


      def aim(self, player):
      """Use to set the force of ball (ball speed) and the ball angle"""

      mouse_position = pygame.mouse.get_pos()
      self.__line_up(mouse_position)
      self.__shot_infomation()
      self.__shot_calculator(mouse_position)

      if pygame.mouse.get_pressed()[2] and self.__clicked == False:
            self.__clicked = True
            LOGGER.info("Player set his aim")
            LOGGER.info(f"Mouse postions: {mouse_position}")
            LOGGER.info(f"Direction: {self.__direction}")
            LOGGER.info(f"Ball postion: {self.__BallPostion}")
            LOGGER.info(f"Ball Speed: {self.__BallSpeed} Ball Angle:
{degrees(self.__BallAngle)}")
            player.add_to_stokes(1)


      def swing(self):
      """Sets velocitys in x and y"""

      #Intial Velocitys
      XVelocity = -self.__BallSpeed*cos(self.__BallAngle)
      YVelocity = -self.__BallSpeed*sin(self.__BallAngle)
      self.__VelocityOfBall = [XVelocity, YVelocity]


      def update_flight(self, update_step):
      """Update the ball flight path"""

      #Calculation forr flight [x,y]
      gravity_acceleration = np.array([0, self.__GRAVITY])
```

```
      air_resistance_acceleration = -self.__AirResistance *
np.array(self.__VelocityOfBall) #Apply AirResistance to the x, y
velcoity
      acceleration = gravity_acceleration + air_resistance_acceleration
#Finds total accleration

      #Updates velocity
      self.__VelocityOfBall += acceleration / self.__BALL_MASS *
update_step


      #Updates Ball position
      self.__BallPostion += self.__VelocityOfBall * update_step

      #Updates Ball rectangle position
      self.__GOLFBALL_rect.x, self.__GOLFBALL_rect.y =
self.__BallPostion.astype(int)

      def trajecotry(self, obstacles, distance, player):
      update_step = 0.028 #Slows the ball movemnet

      if self.__clicked:
            self.update_flight(update_step)

            if not self.__stop:
                  for tile in obstacles:
                  self.__collison(tile, distance, update_step, player)


      def display_ball(self):
      """Displays ball on screen """
      screen.blit(self.__GOLFBALL_image, (int(self.__BallPostion[0]),
int(self.__BallPostion[1])))

      def update_ball_to_right(self, number):
      """Moves ball to right within world movemnet"""
      self.__BallPostion[0] -=number

      def update_ball_to_left(self, number):
      """Moves ball to left within world movemnet"""
      self.__BallPostion[0] +=number

      def get_stop(self):
      """Returns stop"""
      return self.__stop
```

```
    def get_clicked(self):
    """Returns clicked """
    return self.__clicked

    def get_x_velocityofball(self):
    """Returns velocity x"""
    return self.__VelocityOfBall[0]

    def get_y_velocityofball(self):
    """Returns velocity y"""
    return self.__VelocityOfBall[1]

    def get_postion_of_ball(self):
    """Returns ball postion"""
    return self.__BallPostion

    def get_direction(self):
    """Returns direction"""
    return self.__direction

    def get_ball_speed(self):
    """Returns ball power/speed"""
    return self.__BallSpeed

    def set_air_resistance(self, Resistance):
    """Set air restiacne value"""
    self.__AirResistance = Resistance

    def set_stop(self, value):
    """Set stop value"""
    self.__stop = value

    def set_clicked(self, value):
    """Change clicked value """
    self.__clicked = value

    def set_spawn_location(self, location):
    """Set spawn location, location value needs to be a tuple/list"""
    self.__ball_spawn_location = location

    def set_ball_postion(self, x, y):
    """Sets ball postion postion and the golfball rectangle"""
    self.__BallPostion = [x, y]
    self.__GOLFBALL_rect.x = x
    self.__GOLFBALL_rect.y = y
```

Logger.py

```
import logging
import os
import datetime


def logger_setup(name):
      """Create the Logger for the application"""

      #Create the name of the log
      time = datetime.datetime.now()
      time =f'{name}-' + str(time.strftime("%x")) + " T-"+
str(time.strftime("%X")) + '.log'
      time = str(time).replace("/","-").replace("
","--").replace(":","-")

      #Creates the file within Logs directory
      logging.basicConfig(filename=f"Logs/{time}",filemode='w',
      format='%(name)s - %(asctime)s - %(levelname)s - %(message)s',
      datefmt='%d-%b-%y %H:%M:%S',level=logging.INFO)


def starting_check(name):
      """checks if all that needed on application side to start the
application"""

      if not os.path.exists("Logs"):
      os.mkdir("Logs")
      logger_setup(name)
```

LevelEditor.py

```
import pygame
import sys
import math
import os
```

```
import pickle

sys.path.append('../GoGolf')
from Logger import *


pygame.init()

# Logger
LOGGER = logging.getLogger(f"{__name__}")
starting_check("Level_Editor")

# Game window
SCREEN_HEIGHT = 768
SCREEN_WIDTH = 1366
LOWER_MARGIN = 100
SIDE_MARGIN = 300

#Grid
ROWS = 16
MAX_COLS = 150
TILE_SIZE = SCREEN_HEIGHT // ROWS
TILE_TYPES = 21

#FPS
FPS = 60
clock = pygame.time.Clock()

#Colours
BLUE = "#2596be"
WHITE = "#F5FEFD"

pygame.key.set_repeat(0, 0)
screen = pygame.display.set_mode(((SCREEN_WIDTH + SIDE_MARGIN),
(SCREEN_HEIGHT + LOWER_MARGIN)))
pygame.display.set_caption("GoGolf Level Editor")


# Images
current_directory = os.getcwd()
print(current_directory)

background_img = pygame.image.load('bg.png').convert_alpha()
save_img = pygame.image.load(os.path.join(current_directory,
'Assests','save.png')).convert_alpha()
load_img = pygame.image.load(os.path.join(current_directory,
```

```
'Assests','load.png')) .convert_alpha()

#Fonts
WII_FONT = os.path.join(current_directory,"Assets", "Fonts",
"AOTFShinGoProMedium.otf")



class LevelEditor():
      def __init__(self):
      self.__editor_running = True
      self.__current_level = Level()
      self.__save_button = Button(SCREEN_WIDTH // 2, SCREEN_HEIGHT +
LOWER_MARGIN - 75, save_img, 1)
      self.__load_button = Button(SCREEN_WIDTH // 2 + 200, SCREEN_HEIGHT
+ LOWER_MARGIN - 75, load_img, 1)
      self.__speed_muiltplier = 1
      self.__pause = False
      self.__confirm = True
      self.__total_scroll = 0


      def __UpdateSpeedMuilplier(self, keys):
      """Speed up camera"""

      if keys[pygame.K_LSHIFT]:
            LOGGER.info('Spped Muiltplied')
            self.__speed_muiltplier = 5
      else:
            self.__speed_muiltplier = 1


      def LevelControls(self, keys):
      """Level controls, that move the camera"""
      # Modify background_scroll based on user input or scrolling
mechanism
      # For example, you can use the arrow keys:

      scroll_speed = 5*self.__speed_muiltplier
      if keys[pygame.K_a]:
            if not self.__total_scroll > (2*TILE_SIZE):
                  self.__current_level._UpdateScrollLeft(scroll_speed)
                  self.__total_scroll += scroll_speed


      if keys[pygame.K_d]:
```

```
            if abs(self.__total_scroll) < (MAX_COLS*TILE_SIZE) -
SCREEN_WIDTH:
                  self.__current_level._UpdateScrollRight(scroll_speed)
                  self.__total_scroll -= scroll_speed


      def __DisplayLevelInfomation(self):
      """Displays UI elemetns such as text promt and current level"""

      #Text
      text = Text()
      text.set_all(f'Level: {self.__current_level.Get_Level():2}',
"Black", WII_FONT, 35, 10 , SCREEN_HEIGHT + LOWER_MARGIN - 90 )
      text.draw_text()
      text.set_text(f'Press Q or E to change level')
      text.set_y(SCREEN_HEIGHT + LOWER_MARGIN - 30)
      text.draw_text()


      def __DrawEditor(self):
      """Draws world, grid, and UI"""

      self.__current_level._DrawBackground()
      self._DrawWorld()
      self._DrawGrid()
      pygame.draw.rect(screen, BLUE, (SCREEN_WIDTH, 0, SIDE_MARGIN,
SCREEN_HEIGHT))


      def __EventHandler(self):
      events = pygame.event.get()
      for event in events:
            if event.type == pygame.QUIT:
                  LOGGER.info("Quit Application")
                  sys.exit()

            #Addiotnal Level Controls
            if event.type == pygame.KEYDOWN:
                  if not self.__pause:
                  self.__ChangeLevel(event)
                  LOGGER.info(f'Level changed to
{self.__current_level.Get_Level()}')

                  #Confirms if editor wants to save
                  else:
                  if event.key == pygame.K_RETURN:
```

```
                        self.__confirm = True
                        self.__pause = False
                        LOGGER.info('Confirmed')

                  elif event.type == pygame.KEYDOWN and not event.key ==
pygame.K_RETURN:
                        self.__pause = False
                        LOGGER.info('Unconfirmed')

                  LOGGER.info('Editor Resumed')



      pygame.display.update()
      clock.tick(FPS)


      def __ChangeLevel(self, event):

      if event.key == pygame.K_e and not
self.__current_level.Get_Level() >= 8:
            self.__current_level.UpdateSelectedLevel(1)
      if event.key == pygame.K_q:
            if not self.__current_level.Get_Level() <= 1:
                  self.__current_level.UpdateSelectedLevel(-1)


      def Editor(self):
      """Editor loop (main loop)"""

      LOGGER.info('In editor loop')

      while self.__editor_running:
            keys = pygame.key.get_pressed()
            self.__UpdateSpeedMuilplier(keys)
            self.LevelControls(keys)

            self.__DrawEditor()

            button_list = self.__current_level.get_button_tile_list()
            self._DrawTileButtons(button_list)
            pygame.draw.rect(screen, WHITE,
button_list[self.__current_level.Get_ButtonTileCount()].rect, 3)

            self.__DisplayLevelInfomation()
            self.EditorControls()
```

```
            if self.__save_button.draw():
                  self.SaveWorld()

            if self.__load_button.draw():
                  self.LoadWorld()

            self.__EventHandler()



      def _DrawTileButtons(self, button_list):
      """Draw all tile buttons"""

      #Tile buttons
      #Selecting  a tile
      for i in range(0, TILE_TYPES):
            if button_list[i].draw():
                  self.__current_level.set_button_count(i)
                  LOGGER.info(f'Selected tile {button_list[i]}')
                  break


      def _DrawGrid(self):
      """Draw grid"""

      # Vertical lines
      for c in range(-2 ,MAX_COLS + 1):
            pygame.draw.line(screen, WHITE, (c * TILE_SIZE +
self.__total_scroll, 0), (c * TILE_SIZE +  self.__total_scroll,
SCREEN_HEIGHT))

      # Horizontal lines
      for c in range(ROWS + 1):
            pygame.draw.line(screen, WHITE, (0, c * TILE_SIZE),
(SCREEN_WIDTH, c * TILE_SIZE))

      #Function for drrawing the world tiles dictionary #EEE
      def _DrawWorld(self):
      """Draw world"""

      for y, row in enumerate(self.__current_level.Get_WorldData()):
            for x, tile in enumerate(row):
                  if tile >= 0:
                  screen.blit(self.__current_level.Get_TileList()[tile],
```

```
(x * TILE_SIZE + self.__total_scroll, y *TILE_SIZE))


      def EditorControls(self):
      """Editor controls"""

      pos = pygame.mouse.get_pos()
      x = (pos[0] - self.__total_scroll) // TILE_SIZE
      y = pos[1] // TILE_SIZE

      #check if cordiantes are in tile area sp mouse clicks are only
recorded in game area
      if pos[0] < SCREEN_WIDTH and pos[1] < SCREEN_HEIGHT:
            self.__current_level._AddTiles(x,y)

            if pygame.mouse.get_pressed()[0] == 1:
                  self.__current_level._RemoveTiles(x,y)


      def PasueEditor(self, select):
      """Pause the game"""

      self.__pause = True
      LOGGER.info("Game Paused")
      while self.__pause:
            self.__DrawEditor()
            text = Text()
            text.set_all(f"PRESS RETURN TO CONFIRM {select} LEVEL
{self.__current_level.Get_Level()}", "Red", WII_FONT, 50,
SCREEN_WIDTH//2 - SIDE_MARGIN, SCREEN_HEIGHT//2)
            text.draw_text()
            self.__EventHandler()
      LOGGER.info("Game Resumed")



      def SaveWorld(self):
      """Save world data"""

      self.PasueEditor("SAVE")
      LOGGER.info("Save Game")
      if self.__confirm:
            self.__confirm = False
            pickle_out =
open(f'Levels/level{self.__current_level.Get_Level()}_data', 'wb')
            pickle.dump(self.__current_level.Get_WorldData(),
```

```
pickle_out)
            pickle_out.close()
            LOGGER.info(f"Level {self.__current_level.Get_Level()} has
been saved ")


      def LoadWorld(self):
      """Load world data"""

      self.PasueEditor("LOAD")
      LOGGER.info("Save Level")
      if self.__confirm:
            self.__confirm = False
            try:
                  pickle_in =
open(f'Levels/level{self.__current_level.Get_Level()}_data', 'rb')
                  self.__total_scroll = 0
                  self.__current_level.Set_WorldData([])

self.__current_level.Set_WorldData(pickle.load(pickle_in))
                  LOGGER.info(f"Level {self.__current_level.Get_Level()}
has been loaded ")
            except FileNotFoundError:
                  LOGGER.info(f"Level {self.__current_level.Get_Level()}
does not exist ")
                  text = Text()
                  text.set_all(f"LEVEL
{self.__current_level.Get_Level()} DOES NOT EXIST", "Red", WII_FONT, 50,
SCREEN_WIDTH//2 - SIDE_MARGIN + 100, SCREEN_HEIGHT//2)
                  self.__DrawEditor()
                  text.draw_text()
                  self.__EventHandler()
                  pygame.time.wait(500)




class Level():
      def __init__(self):
      self.__level_background_img =
pygame.image.load('bg.png').convert_alpha()
      self.__tiles_background = math.ceil((SCREEN_WIDTH - SIDE_MARGIN) /
self.__level_background_img.get_width()) + 1
      self.__world_data = self.__CreateBlankWorld()
      self.__tile_list = self.__CreateTileList()
```

```
        self.__background_widith = background_img.get_width()
        self.__background_scroll = 0
        self.__level = 1
        self.__button_tile_list = self.__SetupTileButtons()
        self.__button_tile_count = 0


        def __SetupTileButtons(self):
        """Set up tile button"""

        LOGGER.info("Setup Tile Buttons")
        # Make button list
        button_tile_list = []
        button_tile_col = 0
        button_tile_row = 0
        for i in range(0, TILE_TYPES):
              tile_button = Button(
                    SCREEN_WIDTH + (75 * button_tile_col) + 50,
                    75 * button_tile_row + 50,
                    self.__tile_list[i],1)

              button_tile_list.append(tile_button)
              button_tile_col += 1
              if button_tile_col > 2:
                    button_tile_col = 0
                    button_tile_row += 1
        return button_tile_list



        def __CreateTileList(self):
        LOGGER.info("Create List Of Image Tiles")
        tile_list = []
        for x in range(TILE_TYPES):
              img = pygame.image.load(os.path.join(current_directory,
'Assests', 'tile',f'{x}.png'))
              img = pygame.transform.scale(img, (TILE_SIZE, TILE_SIZE))
              tile_list.append(img)

        return tile_list


        def __CreateBlankWorld(self):
        LOGGER.info("Blank world generated")
        #Create emptylist 2d array
        world_data = [[-1] * 150 for _ in range(16)]
```

```
        #Ground
        world_data[-1] = [2 for i in world_data[-1]]

        return world_data


        def __ResetBackgroundScroll(self):
        #For going forwards
        if abs(self.__background_scroll) > self.__background_widith:
                    self.__background_scroll = 0


        #For going backward
        if abs(self.__background_scroll) < self.__background_widith and
self.__background_scroll > 0:
              self.__background_scroll = self.__background_scroll -
self.__background_widith



        def _AddTiles(self, x, y):
        if pygame.mouse.get_pressed()[2] == 1:
              if self.__world_data[y][x] != self.__button_tile_count:
                    LOGGER.info(f"Tile {self.__button_tile_count} Added in
{x},{y} ")
                    self.__world_data[y][x] = self.__button_tile_count

        def _RemoveTiles(self, x, y):
        self.__world_data[y][x] = -1
        LOGGER.info(f"Tile removed in {x},{y} ")


        def _UpdateScrollRight(self, value):
        self.__background_scroll -= value

        def _UpdateScrollLeft(self, value):
        self.__background_scroll += value



        #Draw background
        def _DrawBackground(self):
        screen.fill(BLUE)
        for i in range(0, self.__tiles_background):
```

```
            screen.blit(self.__level_background_img, (i *
self.__background_widith + self.__background_scroll, 0))

        self.__ResetBackgroundScroll()


        def Get_CurrentLevel(self):
        return self.__level

        def Set_CurrentLevel(self, value):
        self.__level = value

        def Get_WorldData(self):
        return self.__world_data

        def Set_WorldData(self, value):
        self.__world_data = value

        def Get_TotalScroll(self):
        return self.__total_scroll

        def Get_BackgroundScroll(self):
        return self.__background_scroll

        def UpdateSelectedLevel(self, value):
        self.__level += value

        def Get_TileList(self):
        return self.__tile_list

        def Get_ButtonTileCount(self):
        return self.__button_tile_count

        def Get_Level(self):
        return self.__level

        def get_button_tile_list(self):
        return self.__button_tile_list

        def set_button_tile_list(self, value):
        self.__button_tile_list = value

        def set_button_count(self, value):
        self.__button_tile_count = value
```

```
class Button():
      """Creates Buttons that change image when the user mouse postion
is over the button"""
      def __init__(self,x,y,image,scale):
      width = image.get_width()
      height = image.get_height()
      self.image =
pygame.transform.scale(image,(int(width*scale),int(height*scale)))
      self.image_hi = self.brighten_image()
      self.rect = self.image.get_rect()
      self.rect.topleft = (x,y)
      self.clicked = False

      def brighten_image(self):
      # Convert image to a copy
      image_copy = self.image.copy()
      brightness_factor = 1.5

      # Iterate through pixels
      for x in range(image_copy.get_width()):
            for y in range(image_copy.get_height()):
                  # Get the color of the pixel
                  colour = image_copy.get_at((x, y))

                  # Brighten RGB channels while leaving alpha channel
unchanged
                  new_colour = (
                  min(int(colour.r * brightness_factor), 255),
                  min(int(colour.g * brightness_factor), 255),
                  min(int(colour.b * brightness_factor), 255),
                  colour.a
                  )

                  # Set the new color to the pixel
                  image_copy.set_at((x, y), new_colour)

      return image_copy


      def draw(self):
      """Returns action and changes self.clicked if the user press the
mouse ontop of the image"""
      action = False
      pos = pygame.mouse.get_pos()
```

```
        if self.rect.collidepoint(pos):
                screen.blit(self.image_hi, (self.rect.x,self.rect.y))
                if pygame.mouse.get_pressed()[0] == 1 and self.clicked ==
False:
                        self.clicked = True
                        action = True


        if pygame.mouse.get_pressed()[0] == 0:
                self.clicked = False


        if self.rect.collidepoint(pos):
                screen.blit(self.image_hi, (self.rect.x,self.rect.y))

        else:
                screen.blit(self.image, (self.rect.x,self.rect.y))
        return action


class Text:
        def __init__(self):
        """Creates and display text with differnt fonts and sizes"""
        self.__text = ""
        self.__colour = ""
        self.__font = None
        self.__x = -1
        self.__y = -1

        def set_text(self, value):
        self.__text = value

        def set_colour(self, value):
        self.__colour = value

        def set_font(self, value):
        self.__font = value

        def set_x(self, value):
        self.__x = value

        def set_y(self, value):
        self.__y = value

        def set_font(self, font, font_size):
```

```
        self.__font = pygame.font.SysFont(font, font_size)

        def set_all(self , text, colour, font, font_size, x, y):
        self.set_colour(colour)
        self.set_text(text)
        self.set_font(font, font_size)
        self.set_x(x)
        self.set_y(y)


        def draw_text(self):
        text_surface = self.__font.render(self.__text, True,
pygame.Color(self.__colour))
        screen.blit(text_surface, (self.__x, self.__y))



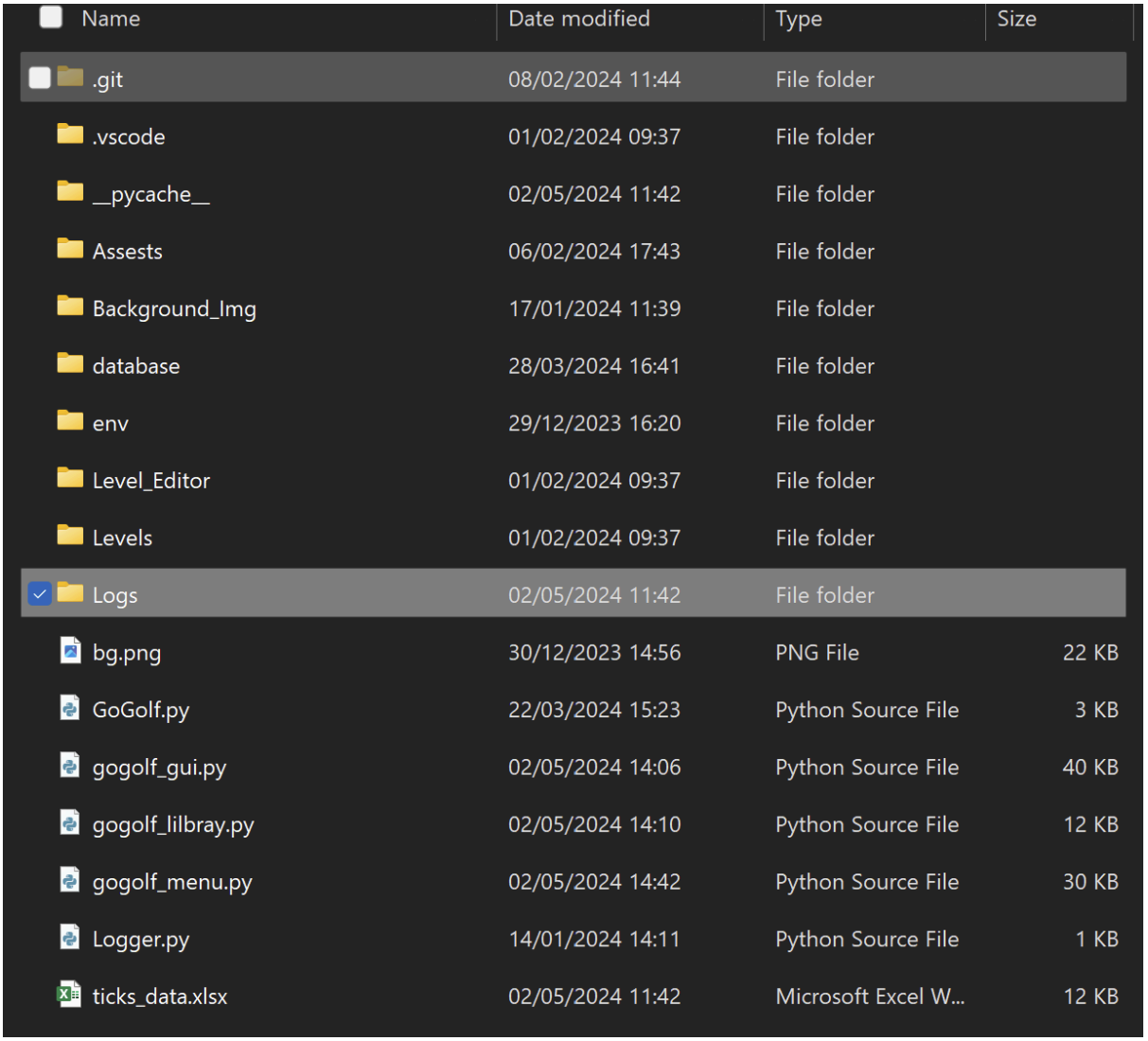

if __name__ == "__main__":
        LOGGER.info('Application Started')
        level_editor = LevelEditor()
        LOGGER.info('LevelEditor object create')
        level_editor.Editor()
```

Logs

Due to logs being very long I added them to a google drive to be downloaded:
https://drive.google.com/drive/folders/1DhFJR6zz61n0-PMc39BsMbk5Snlbq9mM?usp=sharing

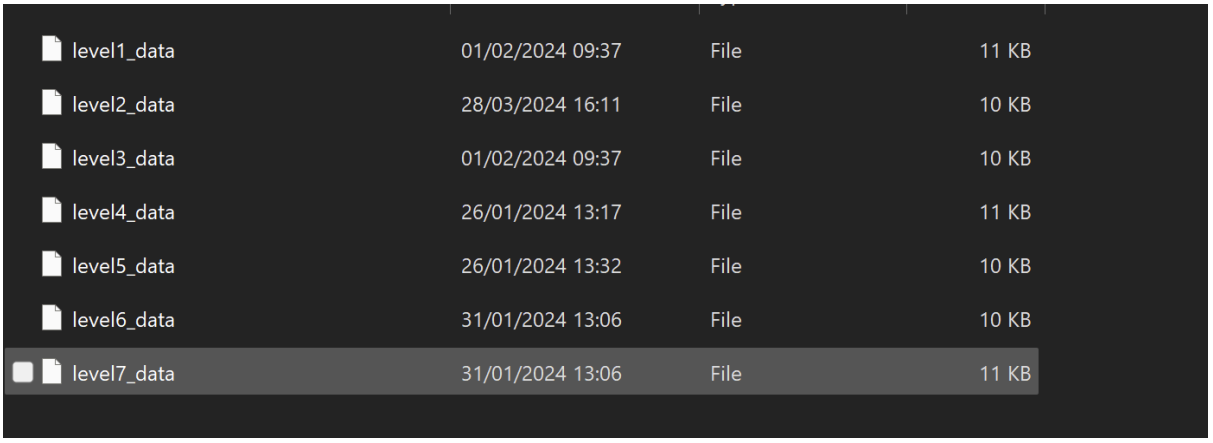
Directory

Main



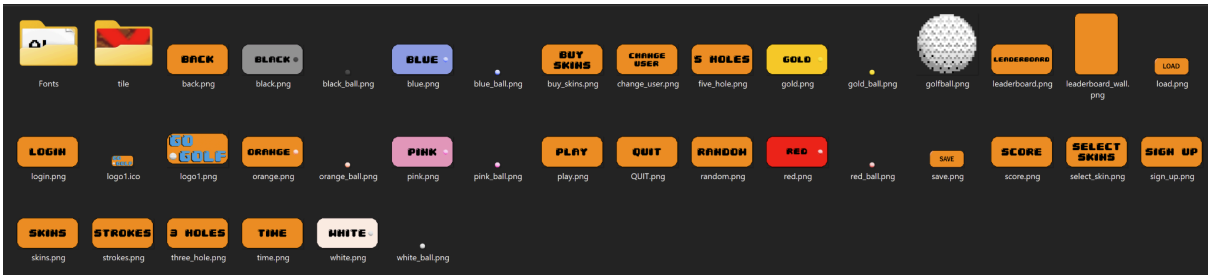
Name	Date modified	Type	Size
.git	08/02/2024 11:44	File folder	
.vscode	01/02/2024 09:37	File folder	
__pycache__	02/05/2024 11:42	File folder	
Assests	06/02/2024 17:43	File folder	
Background_Img	17/01/2024 11:39	File folder	
database	28/03/2024 16:41	File folder	
env	29/12/2023 16:20	File folder	
Level_Editor	01/02/2024 09:37	File folder	
Levels	01/02/2024 09:37	File folder	
Logs	02/05/2024 11:42	File folder	
bg.png	30/12/2023 14:56	PNG File	22 KB
GoGolf.py	22/03/2024 15:23	Python Source File	3 KB
gogolf_gui.py	02/05/2024 14:06	Python Source File	40 KB
gogolf_lilbray.py	02/05/2024 14:10	Python Source File	12 KB
gogolf_menu.py	02/05/2024 14:42	Python Source File	30 KB
Logger.py	14/01/2024 14:11	Python Source File	1 KB
ticks_data.xlsx	02/05/2024 11:42	Microsoft Excel W...	12 KB

Levels



level1_data	01/02/2024 09:37	File	11 KB
level2_data	28/03/2024 16:11	File	10 KB
level3_data	01/02/2024 09:37	File	10 KB
level4_data	26/01/2024 13:17	File	11 KB
level5_data	26/01/2024 13:32	File	10 KB
level6_data	31/01/2024 13:06	File	10 KB
level7_data	31/01/2024 13:06	File	11 KB

Assets



Database Data

Users

	UserID	AccountID	Username	Password
	Filter	Filter	Filter	Filter
1	1	434	erol	B9>A=
2	2	551	steve	=A7B&↔
3	3	101	e	B
4	4	774	chicken1	&9<+
5	5	317	bay	BA
6	6	332	elz	<15
7	7	412	race	;9,5
8	8	413	care	'?'
9	9	96	_	BA
10	10	646	zellam	C59@

Skins

	AccountID	White	Pink	Blue	Orange	Red	Gold	Black
	Filter	Filter	Filter	Filter	Filter	Filter	Filter	Filter
1	96	1	0	0	0	0	0	0
2	101	1	0	0	0	0	0	0
3	317	1	0	0	0	0	0	0
4	332	1	0	0	0	0	0	0
5	412	1	0	0	0	0	0	0
6	413	1	0	0	0	0	0	0
7	434	1	0	1	0	0	0	0
8	551	1	0	0	0	0	0	0
9	646	1	1	0	0	0	0	0
10	774	1	0	0	0	0	0	0

Player_Skins

	AccountID	Coins
	Filter	Filter
1	434	245
2	551	0
3	101	0
4	774	48
5	317	0
6	332	0
7	412	80
8	413	0
9	96	0
10	646	62

Holes3

	UserID	Score	Strokes	Time	Played
	Filter	Filter	Filter	Filter	Filter
1	1	94182	60	226	2024-03-22
2	1	44183	83	318	2024-03-22
3	4	100572	57	263	2024-03-22
4	10	105209	51	247	2024-03-28

Holes5

	UserID	Score	Strokes	Time	Played
	Filter	Filter	Filter	Filter	Filter
1	7	129382	113	495	2024-03-23
2	1	141328	102	485	2024-03-23
3	10	143840	97	495	2024-03-28

Data Structure

Name	Type	Schema
Tables (6)		
Holes3		CREATE TABLE Holes3 (UserID INTEGER NOT NULL, Score INT, Strokes INT, Time INT, Played DATE DEFAULT CURRENT_DATE, FOREIGN KEY(UserID) REFERENCES Users(UserID))
UserID	INTEGER	"UserID" INTEGER NOT NULL
Score	INT	"Score" INT
Strokes	INT	"Strokes" INT
Time	INT	"Time" INT
Played	DATE	"Played" DATE DEFAULT CURRENT_DATE
Holes5		CREATE TABLE Holes5 (UserID INTEGER NOT NULL, Score INT, Strokes INT, Time INT, Played DATE DEFAULT CURRENT_DATE, FOREIGN KEY(UserID) REFERENCES Users(UserID))
UserID	INTEGER	"UserID" INTEGER NOT NULL
Score	INT	"Score" INT
Strokes	INT	"Strokes" INT
Time	INT	"Time" INT
Played	DATE	"Played" DATE DEFAULT CURRENT_DATE
Player_Skins		CREATE TABLE Player_Skins (AccountID INTEGER NOT NULL, Coins INT DEFAULT 0, FOREIGN KEY(AccountID) REFERENCES Users(AccountID))
AccountID	INTEGER	"AccountID" INTEGER NOT NULL
Coins	INT	"Coins" INT DEFAULT 0
Skins		CREATE TABLE Skins (AccountID INTEGER NOT NULL, White BOOL DEFAULT 1, Pink BOOL DEFAULT 0, Blue BOOL DEFAULT 0, Orange BOOL DEFAULT 0, Red BOOL DEFAULT 0, Gold BOOL DEFAULT 0, Black BOOL DEFAULT 0, FORE
AccountID	INTEGER	"AccountID" INTEGER NOT NULL
White	BOOL	"White" BOOL DEFAULT 1
Pink	BOOL	"Pink" BOOL DEFAULT 0
Blue	BOOL	"Blue" BOOL DEFAULT 0
Orange	BOOL	"Orange" BOOL DEFAULT 0
Red	BOOL	"Red" BOOL DEFAULT 0
Gold	BOOL	"Gold" BOOL DEFAULT 0
Black	BOOL	"Black" BOOL DEFAULT 0
Users		CREATE TABLE Users (UserID INTEGER PRIMARY KEY AUTOINCREMENT NOT NULL, AccountID INTEGER UNIQUE NOT NULL, Username VARCHAR(8) NOT NULL UNIQUE, Password VARCHAR(8) NOT NULL)
UserID	INTEGER	"UserID" INTEGER NOT NULL
AccountID	INTEGER	"AccountID" INTEGER NOT NULL UNIQUE
Username	VARCHAR(8)	"Username" VARCHAR(8) NOT NULL UNIQUE
Password	VARCHAR(8)	"Password" VARCHAR(8) NOT NULL
sqlite_sequence		CREATE TABLE sqlite_sequence(name,seq)
name		"name"
seq		"seq"